

Universidad Nacional Autónoma de México

Facultad de Ciencias

Título de la Tesis

Tesis presentada para obtener el grado de

Licenciado en Matemáticas

Presentado por:

Hector Mauricio Salazar Ledesma

Director de Tesis:

Nombre del Director

Ciudad Universitaria, México

3 de septiembre de 2026

Índice general

1. Introducción	4
2. Gráficas y Superficies	5
2.1. Superficies	5
2.1.1. Espacio Topológico	6
2.1.2. Espacio de Hausdorff	8
2.1.3. Compacidad	9
2.1.4. Espacios numerables	11
2.1.5. Variedades topológicas	11
2.1.6. Complejos simpliciales y triangulaciones	12
2.2. Gráficas	14
2.2.1. Gráficas simples y propiedades básicas	14
2.2.2. Representación de gráficas	17
2.2.3. Subgráficas	18
2.2.4. Caminos y ciclos	20
2.2.5. Árboles	21
3. Redes Neuronales Gráficas	25
3.1. Redes Neuronales	26
3.1.1. Conceptos básicos	26
3.1.2. Entrenamiento de redes neuronales	29
3.1.3. Backpropagation	32
3.2. Redes convolucionales CNN	35
3.2.1. Operación de convolución	36
3.2.2. Propiedades de convolución	38
3.2.3. Filtros y mapas de características	39
3.2.4. Arquitectura de una CNN	42
3.3. Redes Neuronales Gráficas	43
3.3.1. Encoders y Decoders	44
3.3.2. Propagación de Mensajes Neuronales	45
3.3.3. La GNN fundamental	47
3.4. Redes Gráficas Convolucionales (GCNs)	48
4. Construcción de algoritmo de muestreo	50
4.1. Construcción de superficies a partir de símlices	50
4.2. Transformación superficie a gráfica	52
4.3. Muestreo por árbol generador mínimo	53
4.4. Muestreo por Esferas	55
4.5. Reasignación de adyacencias	62
4.6. Método general para la reducción de nodos	64
4.7. Análisis de Complejidad	65

5. Resultados	67
5.1. Arquitecturas utilizadas	67
5.1.1. Fourier Neural Operator	68
5.1.2. Dynamic Graph CNN	69
5.1.3. PointNet	69
5.1.4. PointNet++	71
5.2. Análisis de resultados	72
5.3. Conclusiones	73
A. Definiciones básicas	75
B. Código implementado en Python	76

Capítulo 1

Introducción

Aquí va a ir la introducción

Capítulo 2

Gráficas y Superficies

La teoría de gráficas y el estudio de superficies constituyen dos áreas fundamentales de las matemáticas que, en principio, podrían parecer independientes. Sin embargo, ambas se encuentran estrechamente relacionadas cuando una superficie se describe mediante triangulaciones y complejos simpliciales. Esta relación permite traducir ciertos problemas topológicos al lenguaje combinatorio de las gráficas, lo cual resulta especialmente útil para el desarrollo de métodos algorítmicos orientados al análisis y clasificación de superficies. En el contexto de esta tesis, dicha conexión adquiere un papel central, ya que el objetivo principal es desarrollar un algoritmo de optimización de gráficas para el estudio del género de superficies asociadas a nudos. Para ello, es necesario establecer primero las nociones básicas que permiten pasar de un objeto geométrico o topológico, como una superficie, a una representación discreta mediante vértices y aristas. Esta representación facilita el uso de herramientas provenientes de la teoría de gráficas, así como su posterior incorporación en modelos computacionales y redes neuronales gráficas. De manera general, una **gráfica** puede entenderse como un conjunto de **vértices** conectados mediante **aristas** [8]. Por otro lado, una **superficie** es un objeto que localmente se comporta como el plano ($\mathbb{R}^2$) [12], como ocurre con ejemplos clásicos tales como la esfera o el toro. A su vez, un **complejo simplicial** puede describirse como una colección de vértices junto con ciertos subconjuntos de estos, llamados **símplices**, que satisfacen propiedades específicas [10]. Estas tres nociones serán desarrolladas con mayor precisión a lo largo del capítulo. La importancia de los complejos simpliciales radica en que permiten triangular superficies y, a partir de dicha triangulación, inducir una gráfica sobre ellas. Esta representación discreta facilita el estudio de propiedades topológicas mediante herramientas combinatorias y resulta especialmente relevante para esta tesis, ya que permite analizar superficies asociadas a nudos de Lissajous y aplicar algoritmos de optimización orientados al estudio de su género. Por ello, este capítulo introduce los conceptos fundamentales de superficies, complejos simpliciales, triangulaciones y teoría de gráficas, los cuales servirán como base para justificar y construir el algoritmo propuesto en capítulos posteriores. La primera parte del capítulo, dedicada a conceptos topológicos y complejos simpliciales, se basa principalmente en los textos de [11], [3] y [10]. La segunda parte, enfocada en definiciones, propiedades y algoritmos de teoría de gráficas, toma como referencia los textos de [8], [4] y [12]. A lo largo del capítulo, las definiciones se presentarán de manera formal y rigurosa, acompañadas de comentarios explicativos que permitan interpretar su significado y su utilidad dentro del desarrollo general de la tesis.

2.1. Superficies

Para comprender adecuadamente el objeto de estudio que se construye a lo largo de esta tesis, comenzaremos por introducir uno de los conceptos fundamentales que utilizaremos desde el inicio: el de superficie. Sin embargo, para poder definirlo con rigor, es necesario

establecer previamente una serie de conceptos sobre los cuales se apoya esta definición. Algunas definiciones no se mencionarán en este capítulo ya que podría considerarse que el lector está familiarizadas con ellas, pero para mantenerlas en contexto se incluyen en el apéndice A.

2.1.1. Espacio Topológico

De manera intuitiva, un **espacio topológico** puede entenderse como un conjunto al que se le ha asignado una colección especial de subconjuntos, llamados **conjuntos abiertos**. Esta colección no es arbitraria, pues debe satisfacer condiciones específicas que permiten estudiar nociones como continuidad, vecindad y proximidad, sin necesidad de recurrir a una medida de distancia. Formalmente, dicha colección debe contener al conjunto vacío y al conjunto total, ser cerrada bajo uniones arbitrarias y bajo intersecciones finitas.

Definición 1 (Espacio topológico). *Sea X un conjunto. Una **topología** sobre X es una colección $\mathcal{T}$ de subconjuntos de X que satisface las siguientes axiomas:*

1. $\emptyset \in \mathcal{T}$ y $X \in \mathcal{T}$.
2. Si $\{U_i\}_{i \in I}$ es una familia arbitraria de elementos de $\mathcal{T}$, entonces

$$\bigcup_{i \in I} U_i \in \mathcal{T}.$$

3. Si $U_1, \dots, U_n \in \mathcal{T}$, entonces

$$\bigcap_{j=1}^n U_j \in \mathcal{T}.$$

A la pareja $(X, \mathcal{T})$ se le denomina **espacio topológico**, y a los elementos de $\mathcal{T}$ se les llama **conjuntos abiertos**.

La presente definición nos proporciona un marco general para trabajar con nociones geométricas de manera más flexible, lo cual resulta útil en contextos que, a simple vista, podrían parecer triviales.

A continuación, presentaremos un ejemplo que, a pesar de su simplicidad aparente, ilustra de una manera adecuada la estructura de un espacio topológico y revela propiedades interesantes.

Ejemplo 1. *Sea X un conjunto de dos elementos $X = \{x, y\}$, podemos definir una topología sobre este conjunto de la siguiente forma:*

Tomemos los conjuntos abiertos: $\mathcal{T} = \{X, \emptyset, \{x\}\}$

Entonces observamos que:

$$\{x\} \cup X = X, \{x\} \cup \emptyset = \{x\}, X \cup \emptyset = X, \{x\} \cap X = \{x\}, \{x\} \cap \emptyset = \emptyset, X \cap \emptyset = \emptyset$$

Por tanto $(X, \mathcal{T})$ es un espacio topológico, en específico se le conoce como espacio topológico de Sierpiński.

Este espacio, aunque pequeño, nos muestra propiedades útiles en el estudio de conceptos como separación y continuidad, además su importancia radica en que sirve como el ejemplo más sencillo de un espacio no trivial que distingue entre puntos, y es frecuentemente utilizado

como caso de prueba en demostraciones, en la caracterización de funciones continuas, y en demás propiedades.

Otro de los conceptos en el que nos auxiliaremos para construir y tratar las superficies trabajadas es el concepto de **conjunto cerrado**, el cual no es más que la contraparte de la definición de conjunto abierto. Notaremos que la relación entre ambos es directa: un conjunto cerrado es aquel cuyo complemento es un conjunto abierto.

Definición 2 (Conjunto cerrado). *Sea X un espacio topológico. Decimos que un subconjunto A de X es **cerrado** si el conjunto $X - A$ es abierto.*

El siguiente ejemplo es bastante útil visualmente ya que se muestra explícitamente cómo se aplica la definición de conjunto cerrado en un caso concreto.

Ejemplo 2. *Sea $A = \{x \times y | x \geq 0 \text{ e } y \geq 0\} \subset \mathbb{R}^2$, entonces el conjunto A es cerrado.*

Demostración. Observamos que el complemento de A es:

$$\{x \times y \mid x < 0, y \geq 0\} \cup \{x \times y \mid x \geq 0, y < 0\} \cup \{x \times y \mid x < 0, y < 0\}$$

es decir

$$\{x \times y \mid x < 0, y \in \mathbb{R}\} \cup \{x \times y \mid x \in \mathbb{R} \times y < 0\}$$

lo cual es equivalente a

$$(-\infty, 0) \times \mathbb{R} \cup \mathbb{R} \times (0, \infty)$$

, como cada uno de ellos es abierto y la unión de abiertos es abierta, entonces el complemento de A es abierto y por lo tanto A es un conjunto cerrado. $\square$

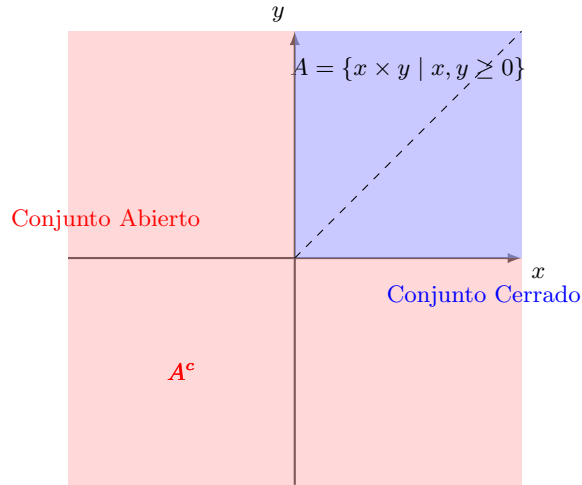


Figura 2.1: Ilustración del suconjunto A

De la mano de las definiciones de conjuntos abiertos y cerrados podemos entender que existe el uso de estos conceptos para definir y establecer regiones sobre un espacio topológico. Notaremos que a partir de las definiciones de interior, cerradura y frontera podemos llegar de forma natural a la consideración de un axioma para espacios topológicos, el cual abordaremos más adelante.

Definición 3 (Interior de un conjunto). Sea X un espacio topológico y A un subconjunto de X . El **interior** de A se define como la unión de todos los conjuntos abiertos contenidos en A , es decir

$$\text{Int}A = \bigcup_{i \in \mathcal{I}} A_i \mid A_i \subset A$$

con A_i abierto para $i \in \mathcal{I}$.

Proposición 1. Sea X un espacio topológico y A un conjunto abierto. Entonces A es abierto en X si y sólo si $\text{Int}A = A$

Definición 4 (Cerradura de un conjunto). Sea X un espacio topológico y A un subconjunto de X . La **cerradura** de A se define como la intersección de todos los conjuntos cerrados que contienen a A , es decir

$$\bar{A} = \bigcap_{i \in \mathcal{I}} A_i \mid A \subset A_i$$

con A_i cerrado para $i \in \mathcal{I}$.

Proposición 2. Sea X un espacio topológico y A un conjunto cerrado. Entonces A es cerrado en X si y sólo si $\bar{A} = A$

Definición 5 (Frontera de un conjunto). Sea X un espacio topológico y A un subconjunto de X . Un punto $x \in \bar{A} \cap \overline{X - A}$ se denomina **punto frontera** de A ; al conjunto $\bar{A} \cap \overline{X - A}$ se le llama **frontera** de A .

2.1.2. Espacio de Hausdorff

Notemos que, en cierto modo, las definiciones de la sección anterior nos hablan del comportamiento de los puntos respecto a un conjunto. Con la noción de **interior** podemos describir cuándo un punto "pertenece completamente" a la región; la **cerradura** nos indica cuándo un punto está lo suficientemente "cerca" de ella; y la **frontera** señala que un punto está justo "en el límite" de dicho conjunto. A partir de estas ideas surge de forma natural la condición que define un **espacio de Hausdorff**: que dos puntos distintos "no se tocan", es decir, que existen entornos abiertos disjuntos alrededor de cada uno, cuyas cerraduras tampoco se intersecan. En consecuencia, sus interiores quedan totalmente separados. Este requisito cobra especial relevancia al pasar de ejemplos familiares, como $\mathbb{R}$ o $\mathbb{R}^2$, donde cada conjunto puntual $\{x_0\}$ es automáticamente cerrado, a topologías más generales en las que ese hecho ya no se cumple. Para evitar estudiar espacios en los que los puntos no puedan separarse nítidamente introducimos la definición de espacios de Hausdorff. En el contexto de esta tesis, este criterio nos permitirá centrar nuestro análisis exclusivamente en objetos que satisfagan dicha condición de separación.

Definición 6. (Espacio de Hausdorff)

Sea $(X, \mathcal{A})$ un espacio topológico. Se dice que X es un **Espacio de Hausdorff**, si para cada par x_1, x_2 de puntos distintos de X , existen vecindades U_1, U_2 de x_1, x_2 respectivamente, que son ajenas. Es decir $U_1 \cap U_2 = \emptyset$

Ejemplo 3. Sea $X = \mathbb{R}$ y definamos la topología dada por el conjunto $\mathcal{B} = \{(a, b) \mid 0 \in (a, b) \subset \mathbb{R}\}$. Tomemos dos puntos distintos arbitrarios, es decir $x \neq y \in X$, entonces queremos ver si existen dos vecindades U, V de x, y tales que $U \cap V = \emptyset$. Pero por la

construcción de $\mathcal{B}$, toda vecindad contiene al punto 0, por tanto $U \cap V \neq \emptyset$ y por lo tanto no es de Hausdorff.

Teorema 1. Sea X un espacio topológico de Hausdorff. Sea U un conjunto con un número finito de puntos. Entonces U es cerrado.

Demostración. Sean x, x_0 en X tales que $x \neq x_0$. Como X es de Hausdorff entonces existen vecindades U, V tales que $U \cap V = \emptyset$. Como U no interseca al conjunto x_0 entonces el punto x no puede pertenecer a la cerradura del conjunto x_0 . Como consecuencia $\overline{\{x_0\}} = x_0$, por lo tanto es cerrado. $\square$

2.1.3. Compacidad

Otra de las propiedades importantes que necesitamos para entender los objetos centrales de este escrito, es el concepto de compacidad. De manera intuitiva podemos decir que un espacio es compacto si podemos "cubrirlo" por completo por conjuntos más pequeños pero que en la suma, den el espacio total. Comencemos por definir en primera instancia a que nos referimos por "cubierta", específicamente a "cubierta abierta".

Definición 7. (Cubierta abierta)

Sea X un espacio topológico, sea $\mathcal{A} = \{\mathcal{A}_i\}_{i \in \mathcal{I}}$ una familia de subconjuntos abiertos de X . Se dice que $\mathcal{A}$ es una **cubierta abierta** de X si $X \subseteq \bigcup_{i \in \mathcal{I}} \{\mathcal{A}_i\}$

Definición 8. (Subcubierta finita) Sea X un espacio topológico, sea $\mathcal{A}$ una cubierta abierta de X .

Decimos que $\mathcal{A}'$ es una subcubierta de $\mathcal{A}$ si $\mathcal{A}' \subset \mathcal{A}$, en particular si $\mathcal{A}'$ es finito, entonces es una **subcubierta finita**.

Por último buscamos cubrir en su totalidad de manera controlada y finita por estos objetos que definimos anteriormente, de esta forma la definición de espacio compacto queda como sigue:

Definición 9. (Espacio Compacto)

Sea X un espacio topológico, se dice que X es **compacto** si de cada cubierta abierta podemos extraer una subcubierta finita de X .

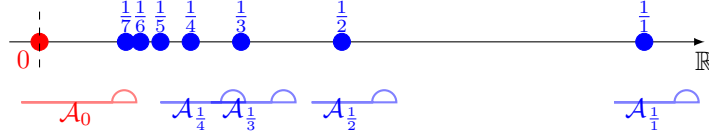
Ejemplo 4. Sea $X = \{0\} \cup \{\frac{1}{n} \mid n \in \mathbb{Z}_+\}$ un subespacio de $\mathbb{R}$.

Demostraremos a continuación que el espacio X es un espacio compacto:

Demostración. Sea $\{\mathcal{A}_\alpha\}$ una cubierta abierta de X , es decir $X \subseteq \bigcup_{\alpha} \mathcal{A}_\alpha$.

Como $0 \in X$ y $\{\mathcal{A}_\alpha\}$ es cubierta abierta, entonces existe un entorno abierto $\mathcal{A}_0$ que contiene a 0, es decir, existe $\epsilon > 0$ tal que $(-\epsilon, \epsilon) \subseteq \mathcal{A}_0$. Notamos que, en particular existe un N suficientemente grande tal que para todo $n > N$, se cumple que $\frac{1}{n} \in (-\epsilon, \epsilon)$, de ahí que los puntos de la forma $\frac{1}{N}, \frac{1}{(N+1)}, \frac{1}{(N+2)}, \dots$ están en $\mathcal{A}_0$, ahora los puntos restantes $1, 2, \dots, \frac{1}{N-1}$ notamos que son finitos y además están en algún $\mathcal{A}_\alpha$. Es decir están en una subcubierta finita.

Por último notamos que la subcubierta finita de X es de la forma $\mathcal{A}_0 \cup \mathcal{A}_1 \cup \dots \cup \mathcal{A}_{N-1}$ por tanto, X es compacto. $\square$

Figura 2.2: Ilustración de la compacidad del espacio X

Lema 1. Sea X un espacio topológico. Sea Y un subespacio de X . Entonces Y es compacto si, y sólo si, cada cubierta de Y por abiertos de X contiene una subcolección finita que cubre a Y .

Demostración. Supongamos primero que Y es compacto y que $\mathcal{A} = \{A_\alpha\}_{\alpha \in J}$ es una cubierta de Y por abiertos de X , entonces la colección:

$$\{A_\alpha \cap Y \mid \alpha \in J\}$$

es una cubierta de Y por conjuntos abiertos en Y ; como Y es compacto entonces existe una subcubierta abierta finita de la forma

$$\{A_{\alpha_1} \cap Y, \dots, A_{\alpha_n} \cap Y\}$$

□

Notamos además que, existe una relación entre los espacios compactos y los espacios de Hausdorff que definimos anteriormente:

Teorema 2. Sea X un espacio topológico de Hausdorff. Sea Y un subespacio compacto de X , entonces Y es cerrado.

Demostración. Sea Y un subespacio compacto de un espacio topológico de Hausdorff X . Queremos mostrar que $X - Y$ es abierto, y por tanto Y será cerrado.

Sea x_0 un punto en $X - Y$. Procederemos a demostrar que existe un entorno de x_0 que no interseca a Y .

Cómo X es de Hausdorff, elegimos entornos disjuntos U_y y V_y de los puntos x_0 y y respectivamente, para cada punto $y \in Y$. Ahora, notamos que la colección

$$\{V_y \mid y \in Y\}$$

es una cubierta de Y por abiertos de X , como Y es compacto, entonces existe una subcolección finita de la forma $V = \{V_{y_1}, \dots, V_{y_n}\}$ que cubre a Y . Además notamos que el conjunto

$$U = \{U_{y_1} \cap \dots \cap U_{y_n}\}$$

formado de tomar las intersecciones de los correspondientes entornos x_0 es disjunto con V . Ya que si $z \in V$ entonces $z \in V_{y_i}$ para algún i , por lo tanto $z \notin U_{y_i}$, y por tanto $z \notin U$. Así U es un entorno de x_0 que no interseca a Y , lo que implica que $X - Y$ es abierto. Por tanto Y es cerrado. □

2.1.4. Espacios numerables

Por último, definiremos otra de las propiedades que nos sirven como base para llegar a definir nuestro objetivo, la cual es la de **numerabilidad**. Esta propiedad nos permite establecer una relación entre los puntos de un espacio topológico y las vecindades que estos poseen. En particular, nos interesa saber si cada punto tiene una base de vecindades numerable, lo que nos permitirá trabajar con espacios más manejables y estructurados.

Definición 10. (*Base de vecindades*)

Sea X un espacio topológico y $x \in X$. Decimos que una familia $\mathcal{B}_x$ de vecindades de x es una **base de vecindades** de x si para cualquier vecindad U de x existe una vecindad $V \in \mathcal{B}_x$ tal que $V \subset U$.

Definición 11. (*Base numerable*) Sea un espacio topológico X . Se dice que X tiene una base numerable en x si existe una base de vecindades numerable.

Definición 12. (*Primer axioma de numerabilidad*) Sea X un espacio topológico. Se dice que X satisface el primer axioma de numerabilidad o que es 1-numerable, si cumple que, cada punto de X tiene una base de vecindades numerable.

Definición 13. (*Segundo axioma de numerabilidad*) Sea X un espacio topológico, se dice que X satisface el segundo axioma de numerabilidad, o que es 2-numerable, si la topología X admite una base numerable.

Ejemplo 5. Observamos que en la topología uniforme $\mathbb{R}^\omega$ satisface el primer axioma de numerabilidad por ser metrizable. Sin embargo no satisface el segundo. Para comprobar este hecho mostraremos que si X es un espacio que tiene base numerable $\mathcal{B}$, entonces cualquier subespacio discreto A de X debe ser numerable. Elijamos para cada $a \in A$ un elemento B_a de la base que interseque a A únicamente en el punto a , si a y b son distintos de A , los conjuntos B_a y B_b son distintos, puesto que el primero contiene a a y el segundo no. Se sigue que la aplicación $a \rightarrow B_a$ es una inyección de A en $\mathcal{B}$, por lo que A debe ser numerable. Ahora bien, el subespacio A de $\mathbb{R}^\omega$ formado por todas las sucesiones de ceros y unos no es numerable y tiene la topología discreta porque $\bar{\rho}(a, b) = 1$ no tiene una base numerable.

2.1.5. Variedades topológicas

Para poder definir una superficie de forma rigurosa, es necesario establecer un contexto adecuado que nos permita trabajar con objetos que cumplan ciertas propiedades topológicas. En este sentido, las **variedades topológicas** son espacios que cumplen con condiciones específicas, como ser de Hausdorff y 2-numerables, lo que garantiza que cada punto tiene una base de vecindades numerable. Estas condiciones son fundamentales para asegurar que los puntos de la variedad se comporten de manera controlada y predecible, lo que facilita, en nuestro caso, el estudio de superficies.

Definición 14. (*Variedad topológica*) Un espacio topológico X , de Hausdorff, 2-numerable es una **variedad topológica** de dimensión n , también llamada n -variedad, si cada punto $x \in X$ tiene una vecindad homeomorfa a un abierto U en la n -bola cerrada $\mathbb{B}^n$.

Definición 15. (*Superficie*)

Una 2-variedad S es una superficie. Si es compacta y no tiene frontera se le llama **Superficie cerrada**.

Ejemplo 6. Sea $S = \{(x, y, z) \in \mathbb{R}^3 | z = x^2 - y^2\}$. Demostraremos que S es una superficie, entonces, tenemos que demostrar que para cada punto $p \in S$ existen conjuntos, $U \subset \mathbb{R}^2$, $W \subset \mathbb{R}^3$ abiertos tal que $p \in W \cap S$ y que existe $h : S \cap W \subset \mathbb{R}^3 \rightarrow U \subset \mathbb{R}^2$ tal que h es un homeomorfismo.

Demostración. Sea $p = (x_0, y_0, z_0) \in S$ entonces por definición $z_0 = x_0^2 - y_0^2$, por otro lado definimos a $W \subset \mathbb{R}^3$ como la bola abierta centrada en p de radio r $\mathcal{B}_p(r)$ y a $U \subset \mathbb{R}^2$ como la vecindad alrededor de (x_0, y_0) , $V(x_0, y_0)$.

Definamos ahora la función $\Pi : S \cap W \rightarrow U$ dada por:

$$\Pi(x, y, x) = (x, y)$$

Es decir la función proyección restringida a $S \cap W$, ahora notemos lo siguiente:

- Π es una función continua ya que es la restricción de la función proyección que es continua en $\mathbb{R}^3$.
- Π es inyectiva pues, dado $u = (x_1, y_1, z_1), w = (x_2, y_2, z_2) \in S \cap W$ tenemos que :

$$\begin{aligned} \Pi(x_1, y_1, z_1) = \Pi(x_2, y_2, z_2) &\Rightarrow \\ (x_1, y_1) = (x_2, y_2) &\Rightarrow \\ x_1 = x_2, y_1 = y_2 & \end{aligned}$$

Por otro lado como $u, w \in S$ entonces:

$$z_1 = x_1^2 - y_1^2 = x_2^2 - y_2^2 = z_2$$

Es decir :

$$u = (x_1, y_1, z_1) = (x_2, y_2, z_2) = w$$

Por tanto Π es inyectiva.

- Sea $(x, y) \in U$ entonces notamos que existe un punto en $p \in S \cap W$ dado por $p = (x, y, x^2 - y^2)$ por tanto Π es sobreyectiva.
- Notamos que Π^{-1} está dada por $\pi^{-1}(x, y) = (x, y, x^2 - y^2)$, como las componentes x, y son continuas y $x^2 - y^2$ es una combinación lineal de componentes continuas entonces $\Pi^{-1}(x, y) = (x, y, x^2 - y^2)$ es continua

Por tanto Π es un homeomorfismo y por tanto $S \cap W$ es homeomorfo a U y por lo tanto S es una superficie. $\square$

2.1.6. Complejos simpliciales y triangulaciones

Habiendo establecido una definición rigurosa de superficie, resulta natural preguntarse cómo representar estos objetos mediante estructuras discretas que sean más adecuadas para su análisis computacional. En este sentido, los **complejos simpliciales** proporcionan una herramienta muy útil para construir triangulaciones de superficies, permitiendo describirlas

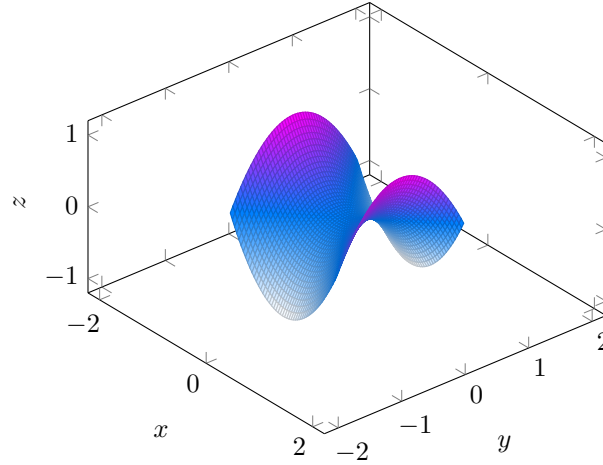


Figura 2.3: Ejemplo de superficie en $\mathbb{R}^3$ comúnmente conocida como "Silla de Montar"

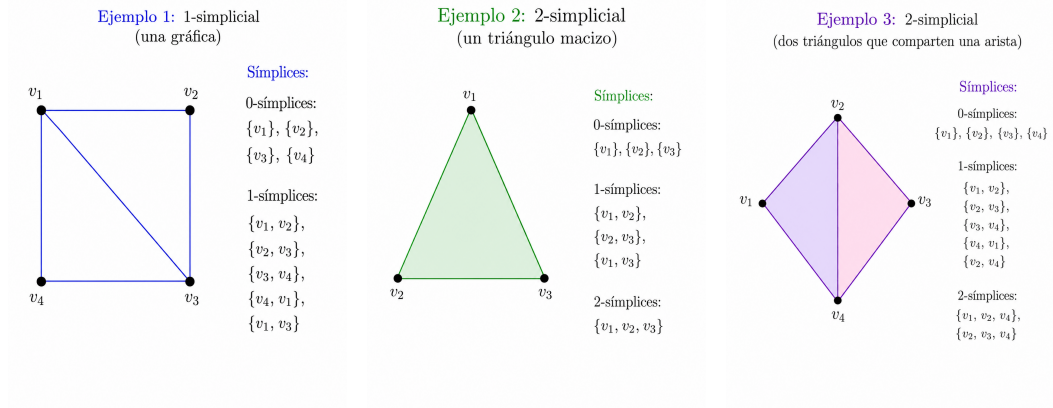
a partir de vértices y aristas. Notamos así que, los complejos simpliciales sirven como puente entre la geometría continua de las superficies y las herramientas combinatorias y algorítmicas que se utilizarán posteriormente para optimizar gráficas y estudiar el género de estos objetos.

Definición 16 (Complejo simplicial). Sea V un conjunto finito cuyos elementos llamaremos *vértices*. Un **complejo simplicial** sobre V es una colección $\mathcal{K}$ de subconjuntos no vacíos de V , llamados **símplices**, que satisface las siguientes propiedades:

1. Para todo vértice $v \in V$, el conjunto v pertenece a $\mathcal{K}$.
2. Si $\sigma \in \mathcal{K}$ y $\tau \subseteq \sigma$ con $\tau \neq \emptyset$, entonces $\tau \in \mathcal{K}$.

Es decir, si un conjunto de vértices pertenece al complejo simplicial, entonces todos sus subconjuntos no vacíos también pertenecen al complejo. A los subconjuntos de un símplice se les llama **caras**.

De esta forma los complejos simpliciales permiten representar objetos geométricos mediante piezas simples. En particular, una superficie puede describirse a partir de una triangulación formada por vértices, aristas y triángulos. Y así esta representación discreta permite asociar una gráfica a la superficie al considerar sus vértices y aristas, lo cual facilita el uso de herramientas combinatorias y algoritmos de optimización.



(a) Complejo 1-simplicial. (b) Complejo 2-simplicial. (c) Complejo 3-simplicial.

Figura 2.4: Ejemplos de complejos simpliciales.

2.2. Gráficas

A partir de las definiciones previas, es posible introducir nuevos objetos que servirán como base para la caracterización formal de las superficies de interés. De este modo, comenzaremos definiendo las **gráficas**, que son estructuras matemáticas compuestas por vértices y aristas. Estas gráficas nos permitirán representar las superficies de manera más manejable y eficiente, facilitando su estudio y análisis. En particular, al considerar la triangulación de una superficie, cada 1-símplice puede ser representado como una gráfica, donde los vértices corresponden a los vértices del triángulo y las aristas a sus lados. Dada la naturaleza de la construcción de las superficies, estas gráficas serán simples, es decir, no tendrán bucles ni aristas múltiples. A continuación, se presentarán las definiciones y propiedades fundamentales de las gráficas simples, que servirán como base para el desarrollo posterior de la teoría.

2.2.1. Gráficas simples y propiedades básicas

Definición 17 (Gráfica simple). *Una gráfica simple G está definida por un par ordenado $G = (V, E)$, donde V es un conjunto cuyos elementos son llamados vértices (o nodos) de la gráfica y E un conjunto cuyos elementos son las aristas de la gráfica.*

Regularmente denotamos a V y E como $V(G)$ y $E(G)$ que hacen referencia a la gráfica G , entonces $G = (V(G), E(G))$. Definimos a E como:

$$E(V) = \{\{u, v\} | u, v \in V, u \neq v\}$$

Usualmente denotamos al par $\{u, v\}$ simplemente como uv .

Derivado de la definición anterior, podemos establecer algunas propiedades que nos permitirán caracterizar y analizar los objetos de estudio de manera más precisa. Estas propiedades incluyen el orden y tamaño de la gráfica, así como las relaciones entre los vértices y aristas.

Definición 18 (Orden y tamaño). *Para una gráfica G , tenemos que:*

1. $\nu_G = |\mathbf{V}(\mathbf{G})|$ es llamado el orden de G .
2. $\varepsilon_G = |\mathbf{E}(\mathbf{G})|$ es llamado el tamaño de G .
3. Para cada arista $e = uv \in E$ los vértices u y v son llamados extremos de la arista.
4. Los vértices u y v son adyacentes o vecinos si $e = uv \in E$.
5. Dos aristas $e_1 = uv$ y $e_2 = uw$ tienen un extremo común, entonces son adyacentes entre sí.

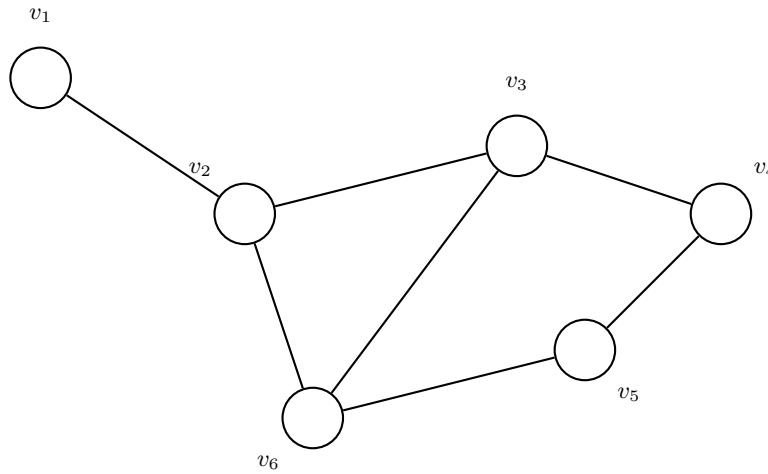


Figura 2.5: Ejemplo de gráfica simple G .

En la figura 2.5 podemos observar que $\nu_G = 6$, $\varepsilon_G = 7$, los extremos de la arista ab son los vértices a y b , también los vértices a y b son vecinos y por último las aristas ab y bf son adyacentes ya que comparten el vértice b .

Dada una gráfica $G = (V, E)$, podemos definir el grado de un vértice como el número de aristas que inciden en él. Esto nos permite entender mejor la estructura de la gráfica y las relaciones entre sus vértices. A continuación, definiremos formalmente el vecindario de un vértice y su grado, así como el grado mínimo y máximo de la gráfica.

Definición 19 (Vecindario). Sea $v \in \mathbf{G}$ un vértice de la gráfica $\mathbf{G}$. El **vecindario** de v es el conjunto dado por:

$$N_G(v) = \{u \in V(G) \mid vu \in E(G)\}$$

El vecindario de un vértice v es el conjunto de todos los vértices que son vecinos de v , es decir, aquellos que están conectados a v por una arista. Este concepto es fundamental para entender la estructura local de la gráfica y las relaciones entre sus vértices.

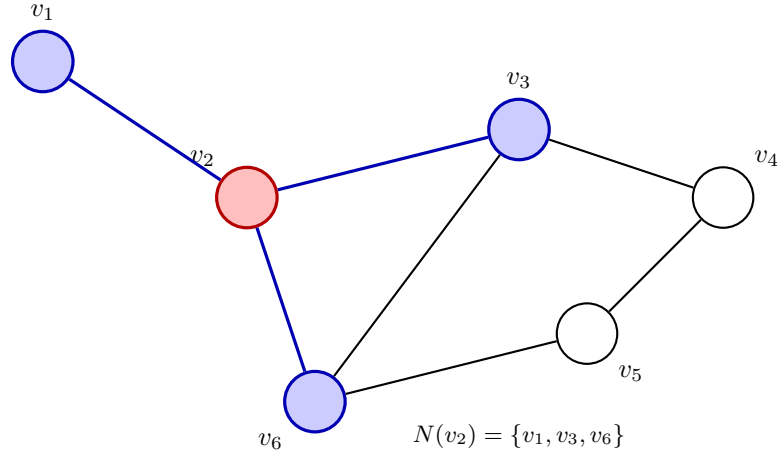


Figura 2.6: Vecindario del vértice v_2 en la gráfica G .

El vecindario resultante es $N(v_2) = \{v_1, v_3, v_6\}$

Otra de las definiciones que sirven tanto para caracterizar así como dar importancia a los nodos de una gráfica es el **grado de un vértice**. Así, esta noción nos permite cuantificar la conectividad de cada vértice dentro de la gráfica, proporcionando información sobre su estructura y las relaciones entre los vértices.

Definición 20 (Grado de un vértice). *El **grado** de v es el numero de sus vecinos.*

$$d_G(v) = |N_G(v)|.$$

De la definición anterior podemos notar que para cualquier vértice v , podría suceder que v no tenga vecinos o por el otro lado que el resto de los vértices de G sean sus vecinos, recordando que v no puede ser vecino de sí mismo. Así, se tiene que:

$$0 \leq d_G(v) \leq n - 1$$

donde n es el orden la gráfica G .

Definición 21 (Vértice aislado). *Cuando $d_G(v) = 0$ decimos que v es un vértice aislado.*

Dada una gráfica $G = (V, E)$ con $V(G) = \{v_1, \dots, v_n\}$ sabemos que el conjunto:

$$\{d_G(v_1), \dots, d_G(v_n)\}$$

Es un conjunto finito de números naturales pues el conjunto $V(G)$ lo es, entonces de esta forma podemos definir a $\delta(G)$ como el grado mínimo de G y $\Delta(G)$ como el grado máximo de G .

Definición 22 (Grado mínimo y grado máximo). *Sea $G = (V, E)$ una gráfica simple, tenemos que;*

1. $\delta(G) = \min\{d_G(v_1), \dots, d_G(v_n)\}$ es el grado mínimo de G .
2. $\Delta(G) = \max\{d_G(v_1), \dots, d_G(v_n)\}$ es el grado máximo de G .

Para la gráfica de la figura 2.5 tenemos que $\delta(G) = 1$ y que $\Delta(G) = 3$.

Proposición 3. *Dada una gráfica G , tenemos que*

$$\delta(G) \leq d(G) \leq \Delta(G).$$

Demostración. Sabemos que, por definición de grado mínimo y máximo, para todo $v \in V(G)$ se cumple que $\delta(G) \leq d_G(v) \leq \Delta(G)$. Sumando sobre todos los vértices:

$$\sum_{v \in V(G)} \delta(G) \leq \sum_{v \in V(G)} d_G(v) \leq \sum_{v \in V(G)} \Delta(G)$$

Como $\delta(G)$ y $\Delta(G)$ son constantes respecto a la suma, esto se puede escribir como:

$$\nu_G \cdot \delta(G) \leq \sum_{v \in V(G)} d_G(v) \leq \nu_G \cdot \Delta(G)$$

Dividiendo entre ν_G en toda la desigualdad, obtenemos:

$$\delta(G) \leq \frac{1}{\nu_G} \sum_{v \in V(G)} d_G(v) \leq \Delta(G)$$

Por definición, el término central es el grado promedio $d(G)$, así que finalmente:

$$\delta(G) \leq d(G) \leq \Delta(G)$$

□

2.2.2. Representación de gráficas

Existen diversas formas de representar gráficas, podemos encontrarlas en su forma visual como la de la figura 2.5, o bien podemos encontrarlas en su forma matricial conocida como **matriz de adyacencia**. La matriz de adyacencia es una representación matricial de una gráfica que permite identificar rápidamente la conexión entre los vértices. En esta matriz, las filas y columnas representan los vértices de la gráfica, y los elementos de la matriz indican si existe una arista entre los vértices correspondientes. A continuación, definiremos formalmente la matriz de adyacencia de una gráfica simple y presentaremos un ejemplo para ilustrar su uso.

Definición 23 (Matriz de Adyacencia). *Sea $V_G = \{v_1, \dots, v_n\}$ ordenado. La matriz de adyacencia $\mathbf{A}$ de $\mathbf{G}$ está definida como*

$$\mathbf{A}(\mathbf{G}) = [a_{ij}]$$

donde:

$$a_{ij} = \begin{cases} 1 & \text{si } v_i v_j \in \mathbf{E}_G \\ 0 & \text{si no} \end{cases}$$

Ejemplo 7. *La matriz de adyacencia de la figura 2.5 quedaría de la siguiente forma:*

$$A(G) = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{pmatrix} \quad (2.1)$$

Notamos que la matriz de adyacencia es una matriz simétrica, ya que si v_i y v_j son vecinos entonces v_j y v_i también lo son, por lo tanto $a_{ij} = a_{ji}$. Además, notamos que la diagonal principal de la matriz de adyacencia es cero, ya que no existe una arista que conecte un vértice consigo mismo en una gráfica simple.

Una alternativa que mejora notablemente la optimización de memoria es la representación de gráficas mediante listas de adyacencia. Este modelo asocia a cada vértice una colección de sus nodos colindantes, lo que permite un acceso más rápido a la información de la conexiones entre los vértices al prescindir de una matriz completa.

Definición 24 (Lista de adyacencia). *Sea $G = (V, E)$ una gráfica simple. La **lista de adyacencia** de G es un conjunto de listas $\{L_v\}_{v \in V}$ donde cada lista L_v contiene los vértices adyacentes a v . Es decir, para cada vértice $v \in V$, tenemos:*

$$L_v = \{u \in V \mid vu \in E\}$$

En la figura 2.5, la lista de adyacencia sería:

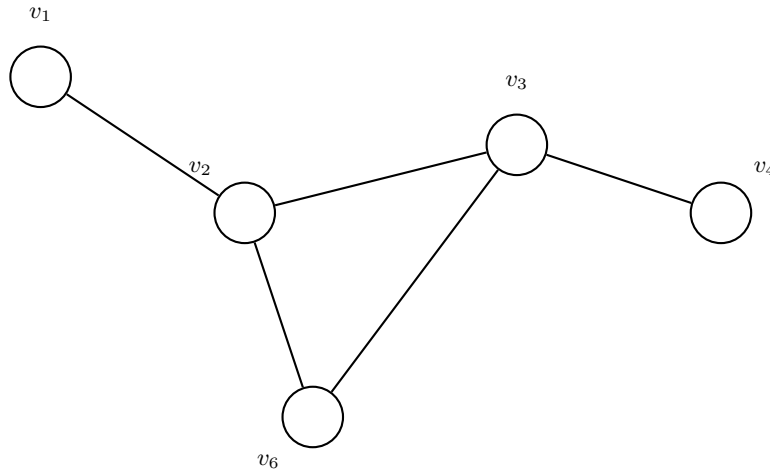
Ejemplo 8. *Sea $G = (V, E)$ la gráfica de la figura 2.5, entonces la lista de adyacencia de G sería:*

Vértice	Lista de adyacencia
v_1	$\{v_2\}$
v_2	$\{v_1, v_3, v_6\}$
v_3	$\{v_2, v_4, v_6\}$
v_4	$\{v_3, v_5\}$
v_5	$\{v_4, v_6\}$
v_6	$\{v_2, v_3, v_5\}$

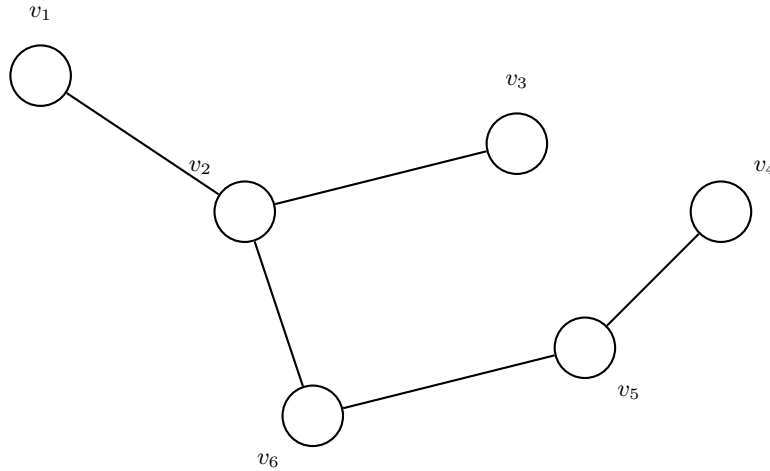
2.2.3. Subgráficas

Debido a los métodos que se utilizarán más adelante en el estudio de los objetos y debido al gran número de datos a procesar, es necesario introducir conceptos que se pueden emplear para el estudio local de las gráficas. De este modo introduciremos el concepto de subgráfica.

Definición 25 (Subgráfica). *Sea $G = (V, E)$ una gráfica simple. Una subgráfica de G es una gráfica $H = (V_H, E_H)$ tal que: $\subseteq G$, si $V(H) \subseteq V(G)$ Y $E(H) \subseteq E(G)$.*

Figura 2.7: Ejemplo de subgráfica de G .

Definición 26. Sea G una gráfica simple y H una subgráfica de G . Decimos que una subgráfica $H \subseteq G$ abarca G (y H es una subgráfica generadora de G) si cada vértice de G está en H , i.e $V(H) = V(G)$

Figura 2.8: Ejemplo de una gráfica H que abarca a G .

Definición 27. Sea G una gráfica simple. Decimos que una subgráfica $H \subseteq G$, es una subgráfica inducida, si $E(H) = E(G) \cap E(V_H)$. En este caso H es inducido por el conjunto $V(H)$ de vértices.

En otras palabras, una subgráfica inducida es aquella que contiene todas las aristas de G que conectan los vértices de H . Esto significa que si dos vértices en H están conectados por una arista en G , entonces esa arista también estará presente en H .

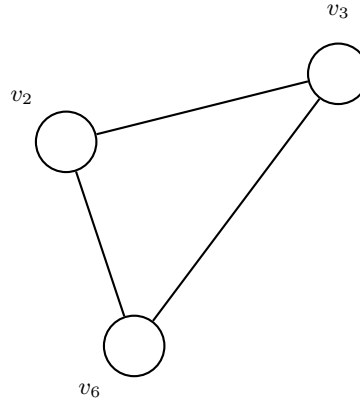


Figura 2.9: Ejemplo de una gráfica inducida.

2.2.4. Caminos y ciclos

Uno de los conceptos fundamentales que se introducirán y además se implementarán en la creación de los algoritmos de la sección final es el concepto de caminos, este concepto se utilizará como herramienta para el estudio del recorrido de la gráfica completa.

Definición 28 (Camino, ciclo y caminos cerrados). Sea $e_i = u_i u_{i+1} \in E(G)$ aristas de G para $i \in \{1, \dots, k\}$.

Un camino es una gráfica no vacía $W = (V, E)$ de la forma;

$$V = \{u_0, u_1, \dots, u_k, u_{k+1}\} \quad E = \{e_0, e_1, \dots, e_k\}$$

donde: u_i es adyacente a u_{i+1} para todo $i \in \{1, \dots, k\}$.

Además:

1. W es **cerrado**, si $u_0 = u_{k+1}$.
2. W es un **camino**, si $u_i \neq u_j$, para todo $i \neq j$.
3. W es un **ciclo**, si es cerrado y $u_i \neq u_j$ para $i \neq j$ excepto para $u_1 = u_{k+1}$

Además, el número $l(W)$ es el tamaño o longitud del camino y está dado por $l(W) = |E(W)|$

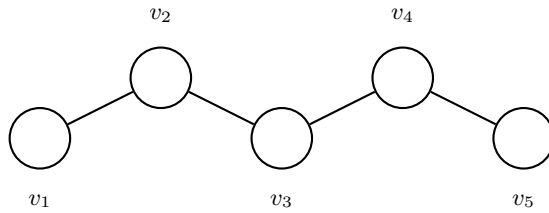


Figura 2.10: Ejemplo de un camino.

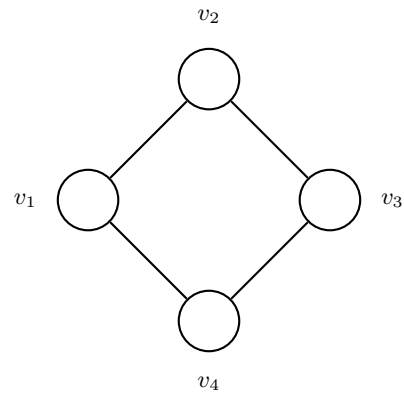


Figura 2.11: Ejemplo de un ciclo.

2.2.5. Árboles

Una forma de reducir el número de vértices sin que la gráfica pierda gran parte de su información, es mediante el estudio de los árboles, veremos su definición y los conceptos más importantes relacionados con el estudio de estas estructuras.

Definición 29 (Gráfica conexa). *Una gráfica $G = (V, G)$ es conexa si para cualquier par de vértices u y v de G si existe un camino que inicia en u y termina en v*

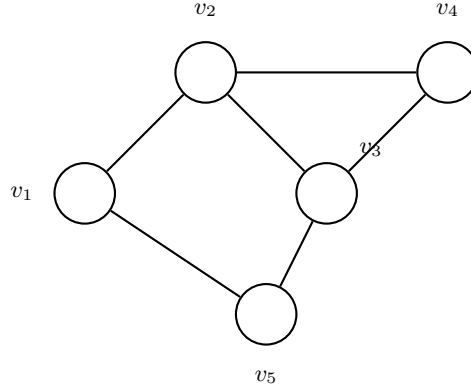


Figura 2.12: Ejemplo de una gráfica conexa.

Notamos que una gráfica G es conexa si y sólo si tiene una única componente conexa, es decir, si no se puede dividir en subgráficas conexas más pequeñas. En caso contrario, si G no es conexa, entonces se puede dividir en múltiples componentes conexas, cada una de las cuales es maximal con respecto a la propiedad de ser conexa.

Definición 30 (Punto). *Dada un gráfica G conexa y una arista $e \in E_G$. Decimos que e es un puente de la gráfica G , si $G - e$ es desconexa.*

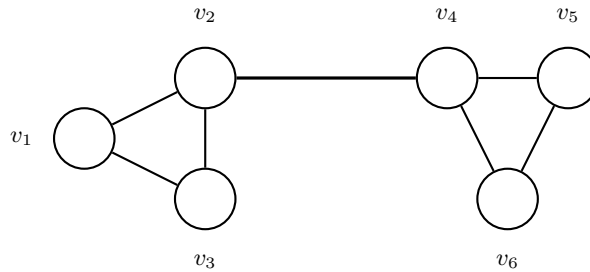


Figura 2.13: Ejemplo de una gráfica con un puente (arista de corte).

En la figura 2.13 notamos que la arista de es un puente, ya que si la eliminamos, la gráfica resultante se vuelve desconexa, separando los vértices a , b y c de los vértices d , e y f .

Proposición 4. *Sea una gráfica G . Un vértice $e \in E(G)$ es un puente de G si y sólo si e no está en ningún ciclo de G .*

Demostración. Primero notemos que $e = uv$ es un puente sí y sólo sí u, v pertenecen a diferentes componentes conexas de $G - e$, pues de lo contrario implicaría que existe un camino que conecta a u con v lo cual sería una contradicción ya que $G - e$ es desconexo.

($\Rightarrow$) Por contrapositiva, Supongamos que existe un ciclo de G que contiene a e , entonces existe un camino tal que $C = \{u_0 = u, v, \dots, u_{k+1} = u\}$. De aquí, notamos que en $G - e$ existe un camino $C' = \{v_0 = v, \dots, v_{k+1} = u\}$, entonces existe un camino alternativo que conecta a u con v lo cual implica que $G - e$ es conexas. Por tanto $e = uv$ no es puente.

($\Leftarrow$) Supongamos que $e = uv$ no es puente, por la observación hecha al inicio esto implica que u, v pertenecen a la misma componente conexas de $G - e$. Entonces existe un camino que $C = \{v_0 = v, \dots, v_{k+1} = u\}$ conecta a v con u , de ahí notamos que eC es un ciclo pues $eC = \{u, v_0 = v, \dots, v_{k+1} = u\}$. $\square$

Definición 31 (Gráfica acíclica). *Una gráfica es llamada acíclica si no contiene ciclos.*

Definición 32 (Árbol). *Un árbol es una gráfica acíclica y conexas.*

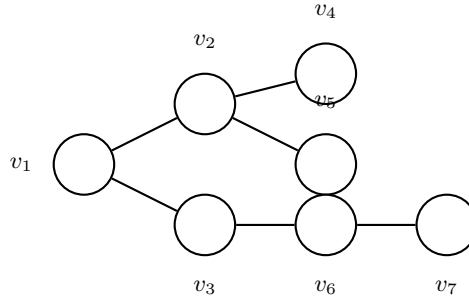


Figura 2.14: Ejemplo de una gráfica acíclica (un árbol).

De la **Proposición 2.2** y la **Definición 22** se deriva el siguiente corolario.

Corolario 1. *Una gráfica conexas es un árbol sí y sólo sí todas sus aristas son puentes.*

Notamos que un árbol es una gráfica conexas y acíclica, además de que todas sus aristas son puentes. Esto implica que si eliminamos cualquier arista de un árbol, la gráfica resultante se vuelve desconexas, lo cual es una propiedad clave de los árboles.

Teorema 3. *Los siguientes enunciados son equivalentes:*

1. T es un árbol
2. Cualesquiera dos vértices están conectados en T por un único camino.
3. T es acíclico y $\varepsilon_T = \nu_T - 1$

Demostración. Sea $\nu_T = n$, Si $n=1$ el teorema es trivial, así que supongamos para $n \geq 2$.

(1) $\Rightarrow$ (2) Supongamos que T es un árbol. Sean u y v dos vértices de T y por contradicción supongamos que existen dos caminos P y Q tales que conectan a u y v . Sin pérdida de generalidad supongamos que $l(P) > l(Q)$. Entonces existe al menos una arista e que está en P pero no está en Q . Por **Corolario 2** sabemos que cada arista de T es

un puente. Y por lo visto anteriormente entonces $\mathbf{u}$ y $\mathbf{v}$ están en diferentes componentes conexas de $\mathbf{T} - \mathbf{e}$. Pero notamos que por el camino $\mathbf{Q}$ siguen estando conectados. Lo cual es una contradicción ya que implicaría que $\mathbf{u}$ y $\mathbf{v}$ no estén en diferentes componentes conexas.

(2) $\Rightarrow$ (3) Esta implicación la probaremos por inducción sobre $\mathbf{n}$.

Para $\mathbf{n} = 2$. Sabemos que no existen componentes disconexas pues cualesquiera dos vértices están conectados por un camino, por lo cual $\varepsilon_{\mathbf{T}} = 1 = 2 - 1 = \nu_{\mathbf{T}} - 1$.

Supongamos que la afirmación se cumple para los valores menores a $\mathbf{n}$ y probemos para $\mathbf{n}$. Sea $\mathbf{P}$ el máximo camino en $\mathbf{T}$ entre $\mathbf{u}$ y $\mathbf{v}$, es decir que no existen aristas $\mathbf{e}$ tales que $\mathbf{eP}$ o $\mathbf{Pe}$ sean caminos. De ahí que $d_{\mathbf{T}}(\mathbf{v}) = 1$. Si $\mathbf{uw} \in \mathbf{E}(\mathbf{T})$ entonces $\mathbf{w} \in \mathbf{P}$. Ahora notamos que la subgráfica $\mathbf{T} - \mathbf{v}$ está conectada por un único camino $\mathbf{P}'$, además notamos que $l(\mathbf{P})$ es a lo más $\mathbf{n}$ y cómo $l(\mathbf{P}) > l(\mathbf{P}')$ entonces $l(\mathbf{P}') \leq \mathbf{n} - 1$ dado que la subgráfica $\mathbf{T} - \mathbf{v}$ está conectada por un único camino (Pues $\mathbf{T}$ lo está) y por hipótesis de inducción tenemos que:

$$\varepsilon_{\mathbf{T}-\mathbf{v}} = \nu_{\mathbf{T}-\mathbf{v}} - 1 = (\mathbf{n} - 1) - 1 = \mathbf{n} - 2$$

Y de ahí que:

$$\varepsilon_{\mathbf{T}} = \varepsilon_{\mathbf{T}-\mathbf{v}} + 1 = \mathbf{n} - 1$$

(3) $\Rightarrow$ (1) Supongamos (3). Basta con probar que $\mathbf{T}$ es conexo. Sean $\mathbf{T}_i = (\mathbf{V}_i, \mathbf{E}_i)$ componentes conexas de $\mathbf{T}$ para $i \in \{1, \dots, k\}$. Sabemos que $\mathbf{T}$ es acíclico, por lo tanto sus componentes conexas $\mathbf{T}_i$ también lo son por tanto $|\mathbf{E}_i| = |\mathbf{V}_i| - 1$. Notemos que:

$$\nu_{\mathbf{T}} = \sum_{i=1}^k |\mathbf{V}_i|, \varepsilon_{\mathbf{T}} = \sum_{i=1}^k |\mathbf{E}_i|$$

Entonces:

$$\begin{aligned} \mathbf{n} - 1 = \varepsilon_{\mathbf{T}} &= \sum_{i=1}^k (|\mathbf{V}_i| - 1) = \sum_{i=1}^k |\mathbf{V}_i| - k = \mathbf{n} - k \\ \therefore k &= 1 \end{aligned}$$

Lo cual quiere decir que $\mathbf{T}$ es un sólo componente conexo, por lo tanto $\mathbf{T}$ es un árbol. $\square$

Definición 33 (Árbol generador). *Dada una gráfica conexas G una subgráfica T de G es un subárbol generado de G si satisface que;*

1. T es una subgráfica generadora de G y
2. T es un árbol.

Teorema 4. *Cada gráfica conexas contiene un árbol generador.*

Demostración. Sea $H \subset G$ la mínima gráfica generadora conexas de G , es decir $H - e$ es disconexa para todo $e \in E(H)$. Esta gráfica es obtenida de G removiendo todas las aristas que no son puentes:

1. Hacemos $H_0 = G$.

2. Para $i \geq 0$ sea $H_{i+1} = H_i - e_i$ donde e no es un puente de H_i . Como e_i no es un puente H_{i+1} es una subgráfica generadora y conexa de H_i y por lo tanto de G .
3. De este forma hacemos $H = H_k$ cuando sólo quedan puentes en la subgráfica. Y por el **Corolario 2** H es un árbol.

□

Definición 34 (Árbol generador mínimo). *Un árbol generador mínimo es un árbol generador tal que la suma de los pesos de sus aristas es la mínima posible, es decir.*

$$w(T) = \sum_{(u,v) \in T} \min\{w(u,v)\}$$

Donde $w(u,v)$ es el peso que hay entre el vértice u y la arista v .

Con lo definido hemos establecido una base sólida para el estudio de nuestras estructuras de interés. Al abordar formalmente las superficies y su discretización a través de complejos simpliciales y así al traducirlos nuevamente a gráficas podemos garantizar que los algoritmos que desarrollaremos posteriormente operen de manera eficiente y precisa. De la mano de estos conceptos, podemos dar el siguiente paso e introducir en el siguiente capítulo el paradigma de las redes neuronales, que nos permitirá abordar problemas complejos de manera más efectiva y con un enfoque basado en el aprendizaje automático. Detallaremos como esta intersección nos proporciona las herramientas algorítmicas necesarias para abordar problemas de optimización y análisis de datos en el contexto de las gráficas y sus aplicaciones en diversas áreas.

Capítulo 3

Redes Neuronales Gráficas

En las últimas décadas, las redes neuronales han emergido como una herramienta fundamental en el campo del aprendizaje automático y la inteligencia artificial. Su capacidad para aprender de grandes volúmenes de datos, reconocer patrones complejos y adaptarse a distintos entornos ha sido clave para su popularidad y adopción en una amplia gama de tareas. Estas aplicaciones abarcan desde el procesamiento del lenguaje natural y la visión por computadora hasta la predicción de series temporales y la toma de decisiones, consolidando a las redes neuronales como una tecnología esencial en desarrollos tan diversos como los sistemas de recomendación y la conducción autónoma. Aunque la literatura actual ha abordado ampliamente las redes neuronales convencionales, la creciente demanda de resolver problemas más complejos ha impulsado la evolución hacia arquitecturas más especializadas. Entre estas, las Redes Neuronales Gráficas (Graph Neural Networks o GNN) se han destacado por su capacidad para trabajar con datos estructurados en forma de gráficas, lo que las hace especialmente útiles para el objetivo de este trabajo. A través de este capítulo, se busca proporcionar una comprensión sólida de cómo las GNNs pueden abordar problemas complejos y ofrecer soluciones efectivas en contextos donde los datos presentan relaciones intrincadas y no lineales. La primera parte de este capítulo, que comprenden las definiciones de conceptos principales, se basa en la obra [6], la segunda parte que abarca conceptos que profundiza en el tema de Redes Neuronales Gráficas se basa en el texto [7].

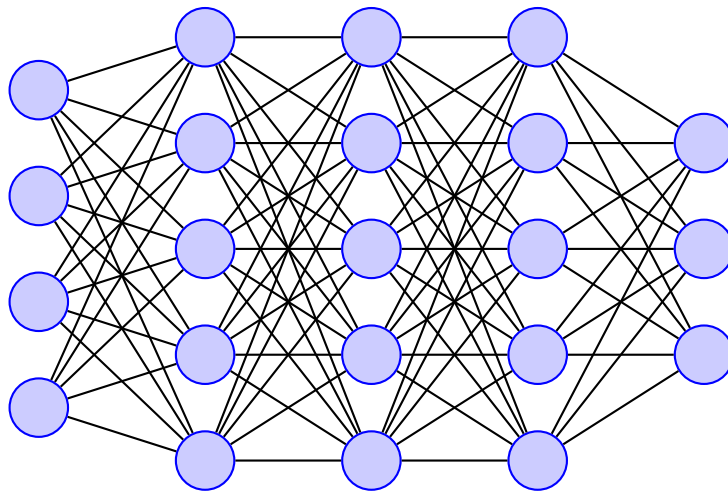


Figura 3.1: Ejemplo de red neuronal con múltiples capas

3.1. Redes Neuronales

3.1.1. Conceptos básicos

Una red neuronal puede entenderse cómo la composición de múltiples funciones no lineales, que en conjunto forman un modelo capaz de aprender representaciones complejas de los datos. Estas redes están inspiradas en la estructura y funcionamiento del cerebro humano, donde las neuronas se conectan entre sí para procesar información y tomar decisiones.

A continuación se daremos la definición formal del modelo de red neuronal partiendo de la unidad más básica denominada como perceptrón.

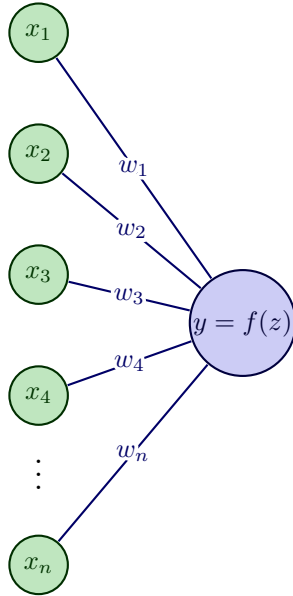
Definición 35 (Perceptrón). Sea $\mathbf{w} \in \mathbb{R}^d$ un vector de pesos, $\mathbf{x} \in \mathbb{R}^d$ un vector de entrada y $b \in \mathbb{R}$ un sesgo (umbral). Definimos la función

$$f : \mathbb{R}^d \rightarrow \{0, 1\}, \quad f(\mathbf{x}) = \chi_{\{\mathbf{y} \in \mathbb{R}^d : \mathbf{w}^T \mathbf{y} + b > 0\}}(\mathbf{x}),$$

donde χ_A denota la función característica del conjunto $A \subseteq \mathbb{R}^d$.

En forma explícita,

$$f(\mathbf{x}) = \begin{cases} 1, & \text{si } \mathbf{w}^T \mathbf{x} + b > 0, \\ 0, & \text{si } \mathbf{w}^T \mathbf{x} + b \leq 0. \end{cases}$$



$$z = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^n x_i w_i + b,$$

$$y = f(z) = \begin{cases} 1, & \text{si } z > 0, \\ 0, & \text{si } z \leq 0. \end{cases}$$

Figura 3.2: Ejemplo de perceptrón con suma ponderada, sesgo y activación escalón.

De la anterior definición, podemos observar que el perceptrón es un clasificador lineal que toma una decisión binaria basada en una combinación lineal de las entradas ponderadas por los pesos y ajustada por un sesgo. La función característica χ_A actúa como una función de activación que determina si la salida es 1 o 0, dependiendo de si la combinación lineal supera el umbral definido por el sesgo.

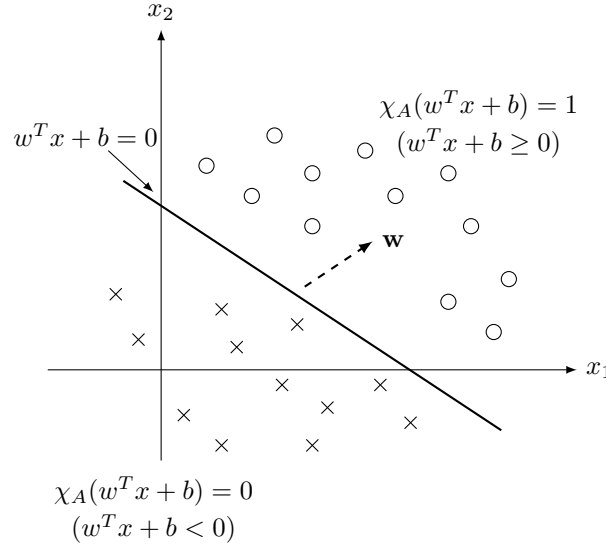


Figura 3.3: Ejemplo de clasificador lineal. Los círculos representan la clase positiva y las cruces la clase negativa. La línea representa la frontera de decisión definida por el perceptrón.

Con el perceptrón definido, podemos construir redes neuronales más complejas al combinar múltiples perceptrones en capas. Una capa, cómo lo veremos a continuación, es la combinación de funciones entre un perceptrón y una función de activación.

Definición 36 (Capa densa de una red neuronal). Sea $L \in \mathbb{N}$ el índice de una capa en una red neuronal. Sean: $\mathbf{W}^{(L)} \in \mathbb{R}^{n_L \times n_{L-1}}$ la matriz de pesos que conecta la capa $(L-1)$ con la capa L , $\mathbf{b}^{(L)} \in \mathbb{R}^{n_L}$ el vector de sesgos (bias), $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ una función de activación (aplicada componente a componente).

Definimos la **capa L -ésima** como la aplicación

$$f^{(L)} : \mathbb{R}^{n_{L-1}} \longrightarrow \mathbb{R}^{n_L},$$

dada por

$$f^{(L)}(\mathbf{x}) = \sigma\left(\mathbf{W}^{(L)}\mathbf{x} + \mathbf{b}^{(L)}\right),$$

donde la función de activación σ se aplica de manera componente a componente sobre el vector $\mathbf{W}^{(L)}\mathbf{x} + \mathbf{b}^{(L)}$.

En particular:

$$f^{(0)}(\mathbf{x}) = \mathbf{x}, \quad \mathbf{x} \in \mathbb{R}^{n_0}.$$

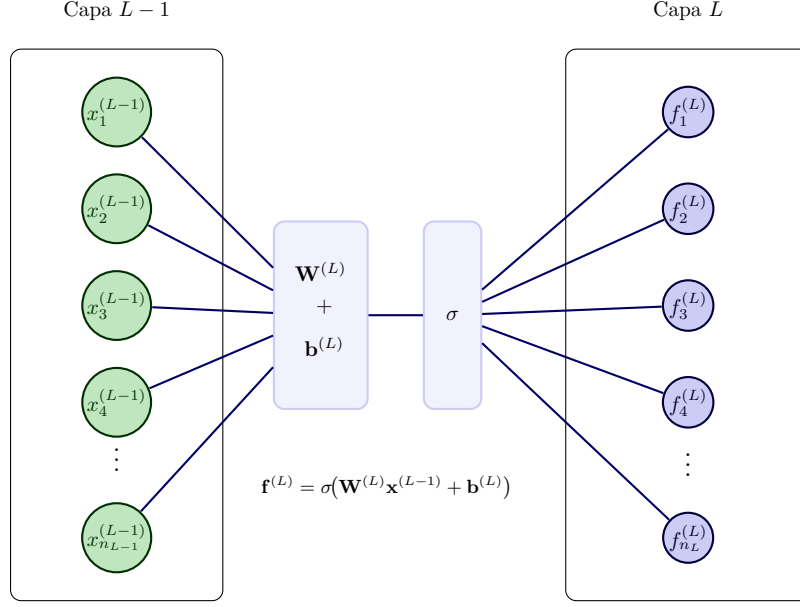


Figura 3.4: Capa de una red neuronal con múltiples neuronas en la capa L .

Observación 1. La función de activación σ introduce no linealidad en la red neuronal, lo cual es de importancia para que una red pueda aprender representaciones complejas y no lineales de los datos. Algunas funciones de activación comunes son:

- **Función sigmoide:** $\sigma(x) = \frac{1}{1+e^{-x}}$
- **Función ReLU (Rectified Linear Unit):** $\sigma(x) = \max(0, x)$
- **Función tanh:** $\sigma(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

Con la definición anterior podemos ahora entender y definir formalmente cómo se compone una red neuronal a partir de múltiples capas.

Definición 37 (Red neuronal). Sea $L \in \mathbb{N}$ el número de capas. Una **red neuronal** con L capas es una función

$$f(\mathbf{x}; \boldsymbol{\theta}) : \mathbb{R}^{n_0} \longrightarrow \mathbb{R}^{n_L}$$

definida como la composición

$$f(\mathbf{x}; \boldsymbol{\theta}) = f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(2)} \circ f^{(1)}(\mathbf{x}),$$

donde: $\mathbf{x} \in \mathbb{R}^{n_0}$ es el vector de entrada, para cada $i \in \{1, \dots, L\}$, $f^{(i)} : \mathbb{R}^{n_{i-1}} \rightarrow \mathbb{R}^{n_i}$ es la transformación asociada a la i -ésima capa, $\boldsymbol{\theta} = \{\mathbf{W}^{(i)}, \mathbf{b}^{(i)} \mid i = 1, \dots, L\}$ es el conjunto de parámetros de la red (matrices de pesos y vectores de sesgo), $\mathbf{y} = f(\mathbf{x}; \boldsymbol{\theta}) \in \mathbb{R}^{n_L}$ es la salida de la red.

En este caso a la función $f^{(1)}$ se le conoce como la primer capa de la red, y a la capa $f^{(L)}$ se le conoce como la capa de salida. A las capas restantes se les conoce como capas ocultas

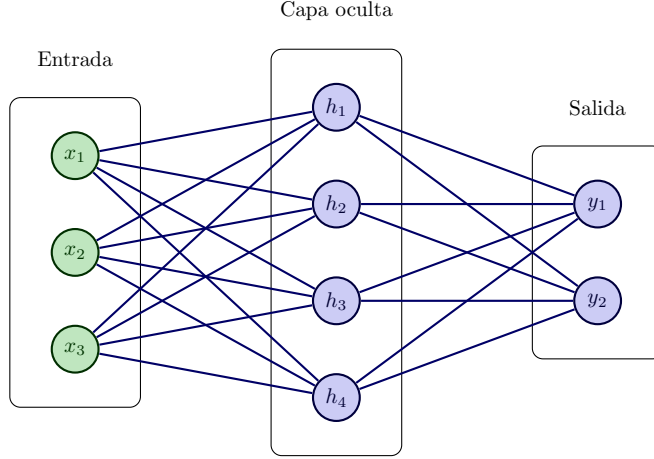


Figura 3.5: Ejemplo de red neuronal con una capa oculta.

3.1.2. Entrenamiento de redes neuronales

El entrenamiento de una red neuronal implica ajustar sus parámetros θ para minimizar la función de pérdida $L(y, \hat{y})$. Este proceso se lleva a cabo mediante un conjunto de datos de entrenamiento, que consiste en pares de entrada-salida (x_i, y_i) . Esta función cuantifica la discrepancia entre las predicciones de la red $\hat{y}_i = f(x_i; \theta)$ y los valores reales y_i . El objetivo del entrenamiento es encontrar los parámetros θ tales que minimicen esta función en promedio sobre todo el conjunto de datos.

A continuación, presentamos las definiciones formales de los conceptos involucrados en el entrenamiento de redes neuronales.

Definición 38 (Conjunto de entrenamiento). Sea $\mathcal{X}$ el espacio de entrada (o espacio de características) y sea $\mathcal{Y}$ el espacio de salida (o espacio de etiquetas).

Un **conjunto de datos de entrenamiento** es una colección finita de N observaciones empíricas, denotada por

$$\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\} = \{(x_i, y_i)\}_{i=1}^N,$$

donde cada par $(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}$ representa un ejemplo individual. En este contexto, x_i corresponde a las características observadas del i -ésimo ejemplo y y_i es su valor verdadero o etiqueta asociada.

Definición 39 (Función de pérdida). Sea $\mathcal{Y}$ el espacio de valores verdaderos y sea $\hat{\mathcal{Y}}$ el espacio de predicciones de un modelo.

Una **función de pérdida** es una función

$$\ell : \mathcal{Y} \times \hat{\mathcal{Y}} \longrightarrow \mathbb{R}_{\geq 0},$$

donde $\ell(y, \hat{y})$ cuantifica la discrepancia entre el valor verdadero $y \in \mathcal{Y}$ y una predicción $\hat{y} \in \hat{\mathcal{Y}}$.

En particular, si

$$f(\cdot; \theta) : \mathcal{X} \longrightarrow \hat{\mathcal{Y}}$$

es una red neuronal parametrizada por $\theta \in \Theta$, entonces la pérdida asociada a un dato de entrenamiento (x_i, y_i) está dada por

$$\ell(y_i, f(x_i; \theta)).$$

Como ya se ha mencionado, el proceso de entrenamiento de una red neuronal tiene como propósito ajustar los parámetros del modelo $\theta \in \Theta$ para minimizar la discrepancia entre las predicciones $\hat{y}_i = f(x_i; \theta)$ y los valores reales y_i .

De manera teórica, este objetivo puede obtenerse a través de la **función de riesgo**.

Definición 40 (Función de riesgo). Sea $\mathcal{D}$ una distribución de probabilidad sobre el espacio producto $\mathcal{X} \times \mathcal{Y}$, donde $\mathcal{X}$ denota el espacio de entradas y $\mathcal{Y}$ el espacio de etiquetas verdaderas.

Dada una función de pérdida $L : \mathcal{Y} \times \hat{\mathcal{Y}} \rightarrow \mathbb{R}_{\geq 0}$ y una función de predicción $f(\cdot; \theta) : \mathcal{X} \rightarrow \hat{\mathcal{Y}}$ parametrizada por $\theta \in \Theta$, la **función de riesgo** (o riesgo esperado) es un mapeo $\mathcal{R} : \Theta \rightarrow \mathbb{R}_{\geq 0}$ definido por la esperanza matemática del error:

$$\mathcal{R}(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}} [L(y, f(x; \theta))] = \int_{\mathcal{X} \times \mathcal{Y}} L(y, f(x; \theta)) d\mathcal{D}(x, y).$$

Esta función cuantifica la pérdida esperada que el modelo parametrizado por θ comete sobre toda la población de datos gobernada por la distribución subyacente $\mathcal{D}$.

Hasta aquí hemos definido el riesgo esperado (o teórico). Sin embargo, en la práctica la distribución de probabilidad subyacente $\mathcal{D}$ es desconocida. Por ello, el riesgo teórico se aproxima mediante su **riesgo empírico**, calculado sobre una muestra finita $\{(x_i, y_i)\}_{i=1}^N$ proveniente de $\mathcal{D}$:

$$\hat{\mathcal{R}}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(y_i, f(x_i; \theta)). \quad (3.1)$$

De esta forma, el problema de aprendizaje se plantea como una tarea de *minimización del riesgo empírico* (Empirical Risk Minimization, ERM):

$$\min_{\theta \in \Theta} \hat{\mathcal{R}}(\theta) = \min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^N \ell(y_i, f(x_i; \theta)). \quad (3.2)$$

En este contexto, la función de pérdida ℓ cuantifica el error a nivel puntual, mientras que la función de riesgo $\mathcal{R}(\theta)$ expresa dicho error en promedio —ya sea teóricamente (riesgo esperado) o empíricamente (riesgo observado).

El proceso de entrenamiento de la red busca, por tanto, minimizar una estimación del riesgo esperado mediante técnicas de optimización numérica como el *descenso del gradiente estocástico* (Stochastic Gradient Descent, SGD) y sus variantes.

Así definimos a continuación formalmente la función de riesgo empírico.

Definición 41 (Función de riesgo empírico). Sea $S = \{(x_i, y_i)\}_{i=1}^N \subset \mathcal{X} \times \mathcal{Y}$ un conjunto de datos de entrenamiento con N observaciones.

Dada una función de pérdida $\ell : \mathcal{Y} \times \hat{\mathcal{Y}} \rightarrow \mathbb{R}_{\geq 0}$ y una función de predicción $f(\cdot; \theta) : \mathcal{X} \rightarrow \hat{\mathcal{Y}}$ parametrizada por $\theta \in \Theta$, la **función de riesgo empírico** es un mapeo $\hat{\mathcal{R}} :$

$\Theta \rightarrow \mathbb{R}_{\geq 0}$ definido como:

$$\hat{\mathcal{R}}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(y_i, f(x_i; \theta)).$$

Esta función calcula la pérdida promedio que comete el modelo con parámetros θ exclusivamente sobre el conjunto de datos observado S .

Como ya se ha mencionado, una manera de minimizar las funciones de pérdida, es utilizando el algoritmo de Gradiente Descendente.

Para ello recordemos la definición de gradiente, así como visualizaremos el algoritmo de gradiente descendente.

Definición 42 (Gradiente). Sea $f : \Omega \subset \mathbb{R}^n \rightarrow \mathbb{R}$, una función diferenciable en x_0 . Entonces el vector cuyas componentes son las derivadas parciales de f en x_0 es el vector gradiente, denotado por ∇f , donde:

$$\nabla f(x_0) = \left(\frac{\partial f(x_0)}{\partial x_1}, \dots, \frac{\partial f(x_0)}{\partial x_n} \right)$$

Este vector contiene la información de cuanto crece o decrece la función en un punto en específico.

Definición 43 (Descenso por gradiente). Sea $J : \Theta \rightarrow \mathbb{R}$ una función diferenciable, denominada función objetivo, donde $\Theta \subseteq \mathbb{R}^n$ representa el espacio de parámetros. Dado un valor inicial $\theta_0 \in \Theta$ y una tasa de aprendizaje $\eta > 0$, el método de descenso por gradiente genera una sucesión $\{\theta_k\}_{k \in \mathbb{N}}$ definida recursivamente por

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} J(\theta_k),$$

donde $\nabla_{\theta} J(\theta_k)$ denota el gradiente de J evaluado en θ_k .

Notamos que el descenso por gradiente es un método iterativo que busca minimizar una función objetivo $J(\theta)$ ajustando los parámetros θ en la dirección opuesta al gradiente de la función. La tasa de aprendizaje η controla el tamaño de los pasos que se dan en cada iteración, lo que afecta la velocidad y estabilidad de la convergencia hacia el mínimo.

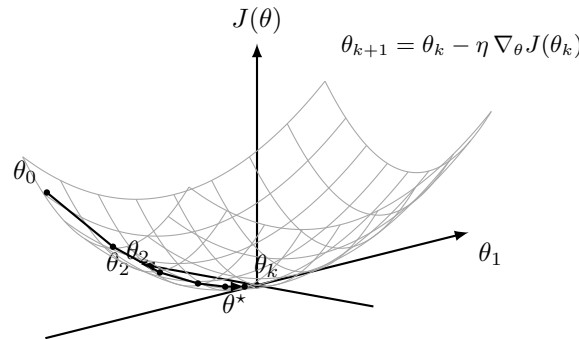


Figura 3.6: Ilustración geométrica del método de descenso por gradiente aplicado a una función objetivo $J : \Theta \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$.

De esta forma podemos describir formalmente el algoritmo de actualización de parámetros por medio del descenso de gradiente.

Algorithm 1 Algoritmo de actualización GD**Inputs:**

$J : \Theta \rightarrow \mathbb{R}$, $\mathbf{T} > 0$ número de iteraciones, η rango de aprendizaje.

Initialize:

Seleccionar $\theta^{(0)} \in \Theta$ aleatoriamente.

for $t \in \{1, \dots, \mathbf{T}\}$ **do**

Calcular $R(\theta^{(t)})$, $\nabla_{\theta^{(t)}} R(\theta^{(t)})$

if $\nabla_{\theta^{(t)}} R(\theta^{(t)}) \neq 0$ **then**

$\theta^{(t+1)} \leftarrow \theta^{(t)} - \eta \nabla_{\theta^{(t)}} R(\theta^{(t)})$

else

Fin del algoritmo

end if

end for

A medida que va aumentando el número de capas en la red neuronal, también aumenta la complejidad de determinar los pesos de todas las conexiones de manera correcta para minimizar los errores, es por eso que nace la necesidad de implementar algoritmos más complejos para la optimización de redes multicapa, de ahí que nace el **BackPropagation**.

3.1.3. Backpropagation

El algoritmo de **backpropagation** es un método eficiente para calcular los gradientes de la función de pérdida con respecto a los parámetros de una red neuronal. Este algoritmo se basa en la regla de la cadena del cálculo diferencial y permite propagar el error desde la capa de salida hacia las capas anteriores, ajustando los pesos y sesgos de manera iterativa para minimizar la función de pérdida.

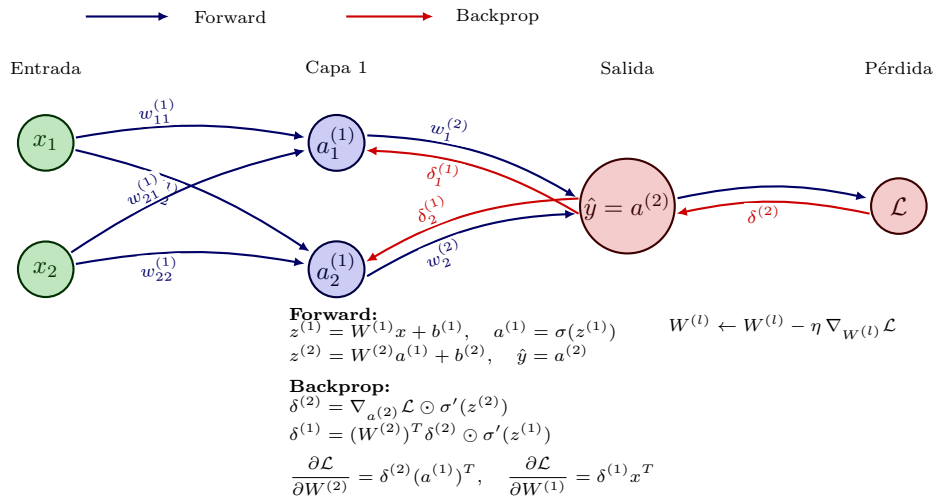


Figura 3.7: Backpropagation: forward (azul) y backward (rojo) con trayectorias separadas mediante desplazamiento de anclajes.

Definición 44 (Gradiente de la función de riesgo empírico). Sea $f(\cdot; \theta) : \mathcal{X} \rightarrow \hat{\mathcal{Y}}$ una red neuronal de L capas parametrizada por $\theta \in \Theta$, dada por la composición $f = f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(1)}$.

El **gradiente de la función de riesgo empírico** $\widehat{\mathcal{R}}(\theta)$ respecto al vector de parámetros θ se define como:

$$\nabla_{\theta} \widehat{\mathcal{R}}(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \ell(y_i, f(x_i; \theta)),$$

donde el gradiente de la pérdida puntual ℓ para cada ejemplo (x_i, y_i) se calcula mediante la regla de la cadena a través de las L capas de la red:

$$\nabla_{\theta} \ell(y_i, f(x_i; \theta)) = \frac{\partial \ell}{\partial f^{(L)}} \cdot \frac{\partial f^{(L)}}{\partial f^{(L-1)}} \cdots \frac{\partial f^{(2)}}{\partial f^{(1)}} \cdot \frac{\partial f^{(1)}}{\partial \theta}.$$

De esta forma a partir de la definición anterior, podemos comenzar con la definición formal del algoritmo dividiéndolo en dos partes principales: la propagación hacia adelante y la propagación hacia atrás.

Propagación hacia adelante

En esta fase, los datos de entrada se pasan a través de la red capa por capa, calculando las activaciones en cada capa utilizando los pesos y sesgos actuales. La salida final de la red se compara con la etiqueta verdadera para calcular el error utilizando una función de pérdida.

Así, sabemos que para cada capa (L) de una red neuronal, la salida $\mathbf{h}^{(L)}$ se calcula de manera recursiva, dado un vector de entrada $\mathbf{x}$ la red realiza las siguientes dos operaciones:

Preactivación

Dada la matriz de pesos $\mathbf{W}^{(L)}$, el vector de sesgos $\mathbf{b}^{(L)}$ en la capa L y $\mathbf{h}^{(L-1)}$ la salida de la capa anterior, entonces la preactivación está dada por:

$$\mathbf{z}^{(L)} = \mathbf{W}^{(L)} \mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}$$

Activación

Dada σ una función de activación (sigmoide, ReLu, etc), y $\mathbf{z}^{(L)}$ la preactivación de la capa L , entonces la activación en la capa L está dada por:

$$\mathbf{h}^{(L)} = \sigma(\mathbf{z}^{(L)})$$

Así, la salida de la última capa se compara con la etiqueta verdadera $\mathbf{y}$ utilizando alguna función de pérdida $\mathbf{J}(\mathbf{h}^{(L)}, \mathbf{y})$ para calcular el error de la red.

Propagación hacia atrás

Para minimizar el error en la función de pérdida utilizamos la definición 45, por una serie de pasos dados de la siguiente manera:

Error en la capa de salida

Sea $\frac{\partial \mathbf{J}}{\partial \mathbf{h}^{(L)}}$ el gradiente de la función de pérdida con respecto a la salida de la red, $\sigma'(\mathbf{z}^{(L)})$, la derivada de la función de activación de la última capa, entonces el error en la capa de salida está dado por:

$$\delta^{(L)} = \frac{\partial \mathbf{J}}{\partial \mathbf{z}^{(L)}} = \frac{\partial \mathbf{J}}{\partial \mathbf{h}^{(L)}} \cdot \sigma'(\mathbf{z}^{(L)})$$

Propagación del error hacia atrás

Sea $\delta^{(L+1)}$ el error de la capa siguiente y $\sigma'(z^{(L)})$ la derivada de la función de activación de la capa L, entonces la propagación del error se da mediante la siguiente expresión:

$$\delta^{(L)} = \left(W^{(L+1)}\right)^T \delta^{(L+1)} \cdot \sigma'(z^{(L)})$$

Gradiente de los parámetros

Para calcular los gradientes de los parametros, vemos que se realiza de la siguiente forma:

- Sea $h^{(L-1)}$ la salida de la capa anterior, entonces el gradiente de los pesos está dado por:

$$\frac{\partial \mathbf{J}}{\partial \mathbf{W}^{(L)}} = \delta^{(L)} \cdot \left(a^{(L-1)}\right)^T$$

- Y para los sesgos se calcula de la siguiente forma:

$$\frac{\partial \mathbf{J}}{\partial b^{(L)}} = \delta^{(L)}$$

Actualización de los parámetros

Una vez que se tienen los pesos y sesgos calculados de la red, se actualizan utilizando descenso del gradiente, de esta forma la actualización está dada por:

Pesos:

$$W^{(L)} = W^{(L)} - \eta \frac{\partial \mathbf{J}}{\partial W^{(L)}}$$

Bias:

$$b^{(L)} = b^{(L)} - \eta \frac{\partial \mathbf{J}}{\partial b^{(L)}}$$

Con lo observado anteriormente, podemos definir formalmente el algoritmo **backpropagation**.

Algorithm 2 Algoritmo Backpropagation

```

1: Initialize Inicializamos pesos  $W^{[l]}$  y bias  $b^{[l]}$  de manera aleatoria para cada capa  $l = 1, \dots, L$ 
2: for cada época do
3:   for cada ejemplo de entrenamiento  $(x, y)$  do
4:     Propagación hacia adelante:
5:     Asignamos  $a^{[0]} = x$ 
6:     for each layer  $l = 1, \dots, L$  do
7:        $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$ 
8:        $a^{[l]} = g(z^{[l]})$ 
9:     end for
10:    Calculamos la pérdida  $J(a^{[L]}, y)$ 
11:    Backward Pass:
12:    Calculamos el error para cada capa de salida:
13:     $\delta^{[L]} = \frac{\partial J}{\partial a^{[L]}} \cdot g'(z^{[L]})$ 
14:    for cada capa  $l = L - 1, \dots, 1$  do
15:      Propagación del error hacia atrás:
16:       $\delta^{[l]} = (W^{[l+1]})^\top \delta^{[l+1]} \cdot g'(z^{[l]})$ 
17:      Calculamos los gradientes:
18:       $\frac{\partial J}{\partial W^{[l]}} = \delta^{[l]} \cdot (a^{[l-1]})^\top$ 
19:       $\frac{\partial J}{\partial b^{[l]}} = \delta^{[l]}$ 
20:    end for
21:    Actualización de parámetros (Descenso del gradiente):
22:    for each layer  $l = 1, \dots, L$  do
23:       $W^{[l]} = W^{[l]} - \eta \cdot \frac{\partial J}{\partial W^{[l]}}$ 
24:       $b^{[l]} = b^{[l]} - \eta \cdot \frac{\partial J}{\partial b^{[l]}}$ 
25:    end for
26:  end for
27: end for

```

De esta forma hemos definido los componentes principales de una red neuronal, así como los algoritmos necesarios para su entrenamiento. A continuación, nos enfocaremos en un tipo específico de redes neuronales conocidas como **redes convolucionales (CNN)**, que son especialmente efectivas para procesar datos con estructura espacial, como imágenes y series temporales.

3.2. Redes convolucionales CNN

Las **redes neuronales convolucionales (CNN)** son una clase especializada de redes neuronales diseñadas para procesar datos que poseen una estructura de malla, como imágenes y series de tiempo. Estas redes se emplean comúnmente en tareas de procesamiento de imágenes, donde los datos se organizan en una malla bidimensional, o en el caso de las series temporales, que pueden representarse como una malla unidimensional con muestras tomadas a lo largo de intervalos de tiempo.

A diferencia de las redes neuronales tradicionales, donde se utiliza la multiplicación

general de matrices, las CNN aplican la operación de convolución en al menos una de sus capas. Esto les permite extraer automáticamente características locales de los datos, como bordes, texturas o patrones, lo que mejora significativamente su capacidad para procesar datos con una estructura espacial clara.

Estudiar las CNN es fundamental como antecedente para entender las redes gráficas convolucionales (GCN), ya que comparten la idea central de la convolución, pero aplicadas a estructuras de datos más complejas, como las gráficas. En esta sección, exploraremos la importancia de las CNN como base teórica, profundizando en su arquitectura y en las definiciones que subyacen a este tipo particular de redes. La mayoría de las definiciones fueron obtenidas del libro [6].

3.2.1. Operación de convolución

Para entender las redes convolucionales, es esencial comprender primero la operación matemática de la convolución. A continuación, se presentan las definiciones clave relacionadas con la convolución en diferentes contextos.

Definición 45 (Convolución). Sean $x, w : \mathbb{R} \rightarrow \mathbb{R}$ dos funciones integrables. La **convolución** de x y w , denotada por $x * w$, es la función

$$(x * w) : \mathbb{R} \rightarrow \mathbb{R}$$

definida, para todo $t \in \mathbb{R}$, por

$$(x * w)(t) = \int_{-\infty}^{\infty} x(a) w(t - a) da,$$

siempre que dicha integral exista.

En el contexto de las redes neuronales convolucionales, el primer término (la función x) se refiere a la entrada, y el segundo término (la función w) es el kernel o filtro.

Dado que en muchos escenarios, el tipo de datos es discreto, se maneja otro tipo de convolución, llamada convolución discreta.

Definición 46 (Convolución discreta). Sean $x, w : \mathbb{Z} \rightarrow \mathbb{R}$ dos sucesiones absolutamente sumables. La **convolución discreta** de x y w , denotada por $x * w$, es la sucesión

$$(x * w) : \mathbb{Z} \rightarrow \mathbb{R}$$

definida, para todo $n \in \mathbb{Z}$, por

$$(x * w)(n) = \sum_{a=-\infty}^{\infty} x(a) w(n - a),$$

siempre que dicha serie sea convergente.

Finalmente, para el uso particular del procesamiento de imágenes en dos dimensiones, podemos utilizar la convolución discreta en dos dimensiones.

Definición 47 (Convolución discreta en dos dimensiones). Sean $\mathbf{I}, \mathbf{K} : \mathbb{Z}^2 \rightarrow \mathbb{R}$ dos funciones (o matrices discretas) absolutamente sumables. La **convolución discreta bidimensional** de $\mathbf{I}$ y $\mathbf{K}$, denotada por $\mathbf{I} * \mathbf{K}$, es la función

$$(\mathbf{I} * \mathbf{K}) : \mathbb{Z}^2 \rightarrow \mathbb{R}$$

definida, para todo $(i, j) \in \mathbb{Z}^2$, por

$$(\mathbf{I} * \mathbf{K})(i, j) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} \mathbf{I}(m, n) \mathbf{K}(i - m, j - n),$$

siempre que la doble suma sea convergente.

Observación 2. La operación de convolución es conmutativa, es decir,

$$\mathbf{I} * \mathbf{K} = \mathbf{K} * \mathbf{I}.$$

Demostración. Sean $(i, j) \in \mathbb{Z}^2$. Por definición de convolución discreta en dos dimensiones, tenemos:

$$(\mathbf{I} * \mathbf{K})(i, j) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} \mathbf{I}(m, n) \mathbf{K}(i - m, j - n).$$

Observamos que cada suma converge, por lo que podemos intercambiar el orden de las sumas y reordenar los términos. Haciendo el cambio de variables $u = i - m$ y $v = j - n$, obtenemos:

$$\begin{aligned} (\mathbf{I} * \mathbf{K})(i, j) &= \sum_{u=-\infty}^{\infty} \sum_{v=-\infty}^{\infty} \mathbf{I}(i - u, j - v) \mathbf{K}(u, v) \\ &= \sum_{u=-\infty}^{\infty} \sum_{v=-\infty}^{\infty} \mathbf{K}(u, v) \mathbf{I}(i - u, j - v) \\ &= (\mathbf{K} * \mathbf{I})(i, j) \end{aligned}$$

Además, observamos que la segunda igualdad es posible, debido a la conmutatividad del producto en cada término de la suma. Cómo esto es válido para todo $(i, j) \in \mathbb{Z}^2$, concluimos que:

$$\mathbf{I} * \mathbf{K} = \mathbf{K} * \mathbf{I}.$$

□

En consecuencia, la expresión anterior puede reescribirse de manera equivalente como

$$(\mathbf{K} * \mathbf{I})(i, j) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} \mathbf{K}(m, n) \mathbf{I}(i - m, j - n).$$

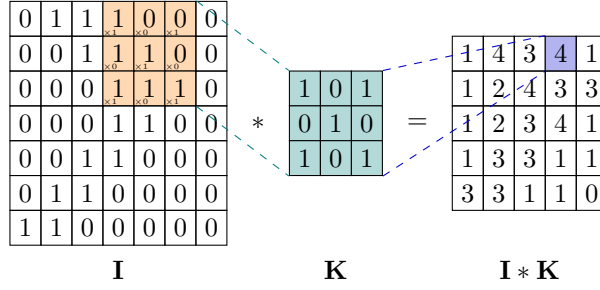


Figura 3.8: Ilustración de la operación de convolución **I** y un kernel **K**.

3.2.2. Propiedades de convolución

Una de las propiedades más importantes de la operación de convolución es su comportamiento bajo ciertas transformaciones, específicamente la invarianza y la equivarianza. Estas propiedades son fundamentales para entender cómo las redes convolucionales pueden aprender características relevantes de los datos de entrada. A continuación, se presentan las definiciones formales de invarianza y equivarianza.

Definición 48 (Invarianza). Sea $f : \mathbf{X} \rightarrow \mathbf{Y}$ una función, $\mathbf{T}$ una transformación en $\mathbf{X}$. Se dice que una función f es invariante bajo $\mathbf{T}$ si:

$$f(\mathbf{T}(x)) = f(x), \quad \forall x \in \mathbf{X}$$

Definición 49 (Equivarianza). Sean f, g dos funciones tales que $f : \mathbf{X} \rightarrow \mathbf{Y}$, $g : \mathbf{Y} \rightarrow \mathbf{X}$. Se dice que f es equivariante bajo g si:

$$f(g(x)) = g(f(x)) \quad \forall x \in \mathbf{X}$$

De lo anterior podemos observar que la invarianza implica que la salida de la función no cambia cuando se aplica una transformación a la entrada, mientras que la equivarianza implica que la transformación de la entrada se refleja de manera consistente en la salida. A continuación, demostraremos que la operación de convolución es equivariante bajo traslaciones.

Teorema 5 (Invarianza de la convolución bajo traslaciones). Sea $f, k \in L^1(\mathbb{R})$ y sea $\mathbf{T}_a$ el operador de traslación definido por

$$(\mathbf{T}_a f)(x) = f(x - a), \quad a \in \mathbb{R}.$$

Entonces, para todo $x \in \mathbb{R}$, se cumple que

$$(\mathbf{T}_a f * k)(x) = (f * k)(x - a).$$

Demostración. Por definición de la convolución en $\mathbb{R}$, tenemos que

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} (\mathbf{T}_a f)(t) k(x - t) dt.$$

Sustituyendo la definición de la traslación, se obtiene

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} f(t - a) k(x - t) dt.$$

Haciendo el cambio de variable $u = t - a$, de modo que $du = dt$ y $t = u + a$, se sigue que

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} f(u) k(x - (u + a)) du.$$

Reordenando los términos en el argumento de k , obtenemos

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} f(u) k((x - a) - u) du.$$

Por la definición de la convolución, esto equivale a

$$(f * k)(x - a) = \int_{\mathbb{R}} f(u) k((x - a) - u) du.$$

Por lo tanto,

$$(\mathbf{T}_a f * k)(x) = (f * k)(x - a), \quad \forall x \in \mathbb{R}.$$

□

De esta forma, demostramos que la operación convolución para funciones definidas en $\mathbb{R}$ son equivariantes con respecto a las traslaciones. Es decir, que en el contexto de las redes neuronales, si trasladamos la entrada de la red, significa que la salida estará trasladada en la misma medida, lo cual es útil cuando se aplica a múltiples ubicaciones en la entrada.

3.2.3. Filtros y mapas de características

En el contexto de las redes neuronales convolucionales, los **filtros** (o kernels) son matrices de pesos que se utilizan para extraer características locales de los datos de entrada mediante la operación de convolución. A continuación, se presentan las definiciones clave relacionadas con los filtros y los mapas de características.

Definición 50 (Kernel (Filtro)). *Sea $\mathbf{X} \in \mathbb{R}^{m \times n}$ una matriz que representa los datos de entrada. Un **kernel** (o **filtro**) es una matriz de pesos*

$$\mathbf{K} \in \mathbb{R}^{h \times w}, \quad \text{con } 1 \leq h \leq m, \ 1 \leq w \leq n.$$

*tal que la acción del kernel sobre la entrada $\mathbf{X}$ se define mediante la **convolución discreta bidimensional**, dada por*

$$(\mathbf{K} * \mathbf{X})(i, j) = \sum_{u=1}^h \sum_{v=1}^w \mathbf{X}(i + u - 1, j + v - 1) \mathbf{K}(u, v),$$

para todo (i, j) tal que los índices sean válidos en la matriz resultante.

El desplazamiento en los índices por -1 se introduce para asegurar la alineación del kernel respecto al elemento central de la región de entrada sobre la cual actúa.

A continuación, enunciaremos algunas propiedades clave de los filtros.

Definición 51 (Propiedades del kernel). *Sea $\mathbf{K} \in \mathbb{R}^{h \times w}$ un filtro (kernel) discreto de dimensiones $h \times w$. Entonces, se definen las siguientes propiedades:*

- **Dimensión.** *La dimensión del kernel viene dada por el producto de sus componentes espaciales:*

$$\dim(\mathbf{K}) = h \times w.$$

- **Simetría.** *El kernel $\mathbf{K}$ se dice simétrico si sus valores son invariantes bajo reflexión respecto al centro del filtro, es decir:*

$$\mathbf{K}(i, j) = \mathbf{K}(h - i + 1, w - j + 1), \quad \forall i \in \{1, \dots, h\}, j \in \{1, \dots, w\}.$$

- **Normalización.** *El kernel $\mathbf{K}$ está normalizado si la suma total de sus coeficientes es igual a la unidad:*

$$\sum_{i=1}^h \sum_{j=1}^w \mathbf{K}(i, j) = 1.$$

Con las definiciones anteriores, podemos definir el concepto de mapa de características, que es el resultado de aplicar un filtro a una entrada mediante la operación de convolución.

Definición 52 (Mapa de características). *Sea $\mathbf{I} \in \mathbb{R}^{m \times n}$ un tensor (o matriz) de entrada y sea $\mathbf{K} \in \mathbb{R}^{h \times w}$ un kernel (o filtro) de convolución. El **mapa de características** asociado a $\mathbf{I}$ y $\mathbf{K}$ se define como la matriz $\mathbf{F} \in \mathbb{R}^{(m-h+1) \times (n-w+1)}$ cuyos elementos están dados por:*

$$\mathbf{F}(i, j) = (\mathbf{I} * \mathbf{K})(i, j) = \sum_{u=1}^h \sum_{v=1}^w \mathbf{I}(i + u - 1, j + v - 1) \mathbf{K}(u, v),$$

para todo $1 \leq i \leq m - h + 1$ y $1 \leq j \leq n - w + 1$.

Adicionalmente, existen dos conceptos importantes en el contexto de las redes convolucionales: el **stride** y el **padding**. El **stride** se refiere al paso con el que se mueve el filtro sobre la entrada durante la operación de convolución, mientras que el **padding** implica agregar bordes adicionales a la entrada para controlar las dimensiones del mapa de características resultante.

Definición 53 (Stride). *Sea $\mathbf{I} \in \mathbb{R}^{m \times n}$ una matriz de entrada y $\mathbf{K} \in \mathbb{R}^{h \times w}$ un kernel de convolución. Definimos el **stride** (o paso de desplazamiento) como un entero positivo $s \in \mathbb{N}$ que determina el número de posiciones que se desplaza el kernel $\mathbf{K}$ sobre la entrada $\mathbf{I}$ en cada dirección.*

De este modo, los índices (i, j) del mapa de características $\mathbf{F}$ corresponden a las posiciones:

$$(i, j) = (p \cdot s, q \cdot s), \quad \text{para } p, q \in \mathbb{N}.$$

Por tanto, los elementos del mapa de características se definen como:

$$\mathbf{F}(p, q) = (\mathbf{I} *_s \mathbf{K})(p, q) = \sum_{u=1}^h \sum_{v=1}^w \mathbf{I}(p \cdot s + u - 1, q \cdot s + v - 1) \mathbf{K}(u, v),$$

donde $*_s$ denota la convolución con stride s .

Definición 54 (Padding). Sea $\mathbf{I} \in \mathbb{R}^{m \times n}$ una entrada y sea $\rho \in \mathbb{N}$ el tamaño del **padding**. Definimos la función de padding

$$\mathbf{p}_\rho : \mathbb{R}^{m \times n} \longrightarrow \mathbb{R}^{(m+2\rho) \times (n+2\rho)}$$

como

$$\mathbf{p}_\rho(\mathbf{I})(i, j) = \begin{cases} \alpha, & \text{si } i \leq \rho \text{ o } i > m + \rho \text{ o } j \leq \rho \text{ o } j > n + \rho, \\ \mathbf{I}(i - \rho, j - \rho), & \text{en otro caso.} \end{cases}$$

donde $\alpha \in \mathbb{R}$ es el valor con el que se rellenan los bordes (por ejemplo, $\alpha = 0$ en el caso de zero-padding).

Otra operación común en las redes convolucionales es el **pooling**, que se utiliza para reducir las dimensiones espaciales de los mapas de características, ayudando a disminuir la cantidad de parámetros y computación en la red, además de controlar el sobreajuste.

Definición 55 (Max Pooling). Sea $\mathbf{X} \in \mathbb{R}^{m \times n}$ una matriz de entrada, y sean $h, w \in \mathbb{N}$ las dimensiones de la ventana pooling, y $s \in \mathbb{N}$ el parámetro de stride. Definimos la operación de **max pooling** como una función

$$\mathbf{M} : \mathbb{R}^{m \times n} \longrightarrow \mathbb{R}^{p \times q},$$

dada por

$$\mathbf{M}(\mathbf{X})(i, j) = \max_{0 \leq u < h, 0 \leq v < w} \mathbf{X}(i \cdot s + u, j \cdot s + v),$$

para todo $i = 0, \dots, p - 1$ y $j = 0, \dots, q - 1$, donde $p = \lfloor \frac{m-h}{s} \rfloor + 1$ y $q = \lfloor \frac{n-w}{s} \rfloor + 1$.

Definición 56 (Average Pooling). Sea $\mathbf{X} \in \mathbb{R}^{m \times n}$ una matriz de entrada, y sean $h, w \in \mathbb{N}$ las dimensiones de la ventana de pooling, y $s \in \mathbb{N}$ el parámetro de stride. Definimos la operación de **average pooling** como una función

$$\mathbf{A} : \mathbb{R}^{m \times n} \longrightarrow \mathbb{R}^{p \times q},$$

dada por

$$\mathbf{A}(\mathbf{X})(i, j) = \frac{1}{h \cdot w} \sum_{u=0}^{h-1} \sum_{v=0}^{w-1} \mathbf{X}(i \cdot s + u, j \cdot s + v),$$

para todo $i = 0, \dots, p - 1$ y $j = 0, \dots, q - 1$, donde $p = \lfloor \frac{m-h}{s} \rfloor + 1$ y $q = \lfloor \frac{n-w}{s} \rfloor + 1$.

Con lo anterior hemos definido los conceptos fundamentales relacionados con las redes neuronales convolucionales, incluyendo la operación de convolución, los filtros, los mapas de características, el stride, el padding y las operaciones de pooling. A continuación, exploraremos la arquitectura típica de una CNN y cómo estos componentes se integran para formar una red completa.

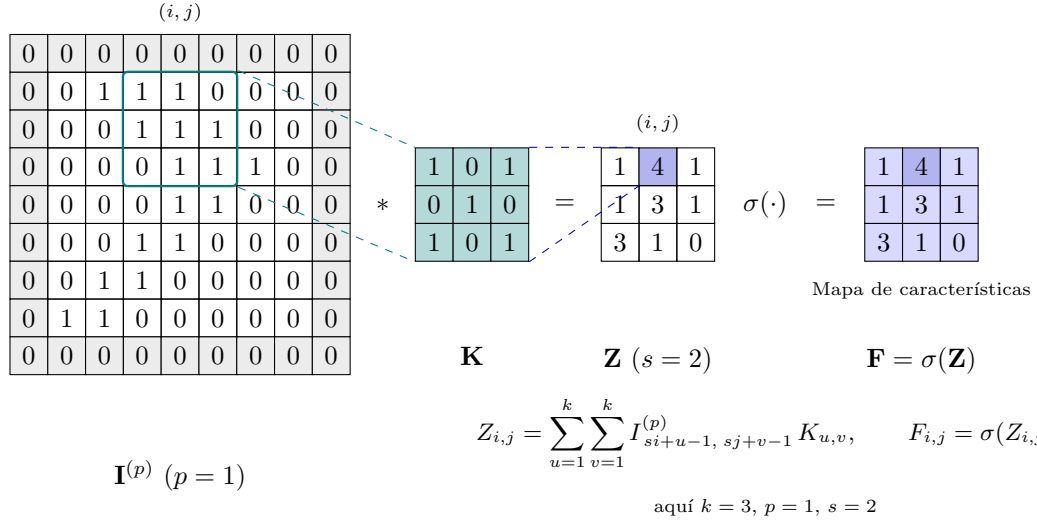


Figura 3.9: Convolución con padding p y stride s produciendo un mapa de características. La salida del filtro es la pre-activación $\mathbf{Z}$ y, tras aplicar la no linealidad σ , se obtiene el mapa de características $\mathbf{F}$.

3.2.4. Arquitectura de una CNN

A continuación, presentamos la definición formal de la arquitectura de una red neuronal convolucional (CNN).

Definición 57 (Arquitectura de una red neuronal convolucional (CNN)). Sea $f : \mathbb{R}^{m \times n \times d} \rightarrow \mathbb{R}^k$ una función que representa el modelo completo de una red neuronal convolucional. La arquitectura de una CNN se define como la composición de tres funciones principales:

$$\mathbf{y} = (\mathbf{g} \circ \mathbf{P} \circ \mathbf{C})(\mathbf{X}),$$

donde $\mathbf{C}$ denota la capa convolucional, $\mathbf{P}$ la capa de pooling y $\mathbf{g}$ la capa completamente conectada, definidas como sigue.

La **capa convolucional** se define por

$$\mathbf{C}(\mathbf{X}) = (\mathbf{K} * \mathbf{X}) + \mathbf{b},$$

donde $\mathbf{K}$ representa el conjunto de filtros convolucionales y $\mathbf{b}$ el sesgo asociado.

La **capa de pooling**, en este caso correspondiente al max pooling, está dada por

$$\mathbf{P}(\mathbf{C}(\mathbf{X}))(i, j, c) = \max_{0 \leq u < h, 0 \leq v < w} \mathbf{C}(\mathbf{X})(i \cdot s + u, j \cdot s + v, c),$$

donde (h, w) son las dimensiones de la ventana y s el desplazamiento o stride.

Finalmente, la **capa completamente conectada** se expresa como

$$\mathbf{y} = \mathbf{g}(\mathbf{z}) = \sigma(\mathbf{W}\mathbf{z} + \mathbf{b}),$$

donde $\mathbf{z}$ es la versión aplanada de $\mathbf{P}(\mathbf{C}(\mathbf{X}))$, $\mathbf{W}$ la matriz de pesos y σ una función de activación no lineal.

Con lo definido anteriormente, se ha establecido una base sólida para comprender las redes neuronales convolucionales (CNN). A continuación, nos adentraremos en el estudio de las redes neuronales gráficas (GNN), que extienden los conceptos de las CNN para trabajar con datos estructurados en forma de gráficas.

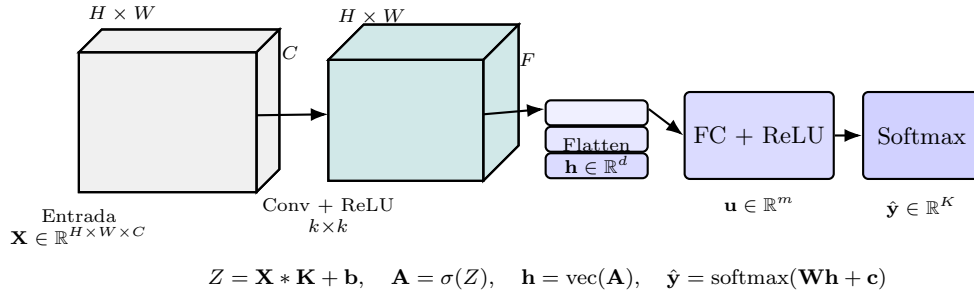
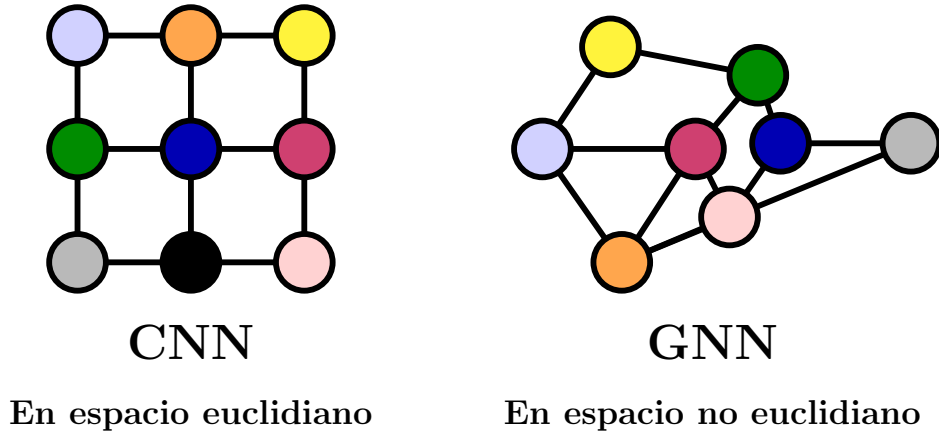


Figura 3.10: Arquitectura CNN simple ilustrada con volúmenes 3D. La red consta de una capa convolucional seguida de una capa completamente conectada y una capa de salida Softmax.

3.3. Redes Neuronales Gráficas

Las **Redes Neuronales Gráficas** (GNN, por sus siglas en inglés) son una clase de modelos de aprendizaje automático diseñados para trabajar con datos estructurados en forma de gráficas. A diferencia de las redes neuronales tradicionales que operan sobre datos tabulares o secuenciales, las GNN están diseñadas para capturar las relaciones y dependencias entre los nodos y las aristas de una gráfica.

Cómo paso inicial, estableceremos algunas definiciones, que nos ayudarán a entender y formalizar el concepto de GNN. Puesto que hemos definido y estudiado las definiciones básicas de gráficas en la sección 2, no nos detendremos en ellas. Las definiciones que se presentan a continuación están basadas en el libro [7].



3.3.1. Encoders y Decoders

Para el objeto de estudio de las Redes Neuronales Gráficas (GNN), los conceptos de **encoder** y **decoder** son de vital importancia para comprender cómo se procesan y representan los datos estructurados en forma de gráficas. A continuación, se presentan las definiciones formales de estos conceptos.

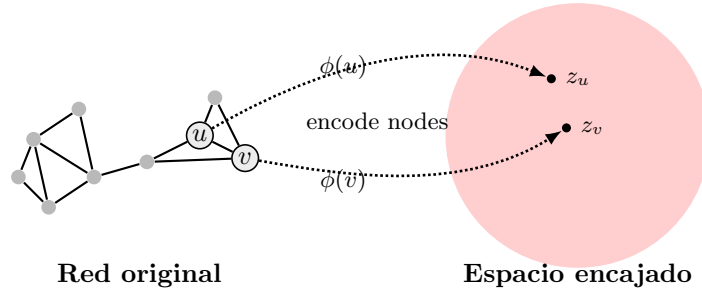
Definición 58 (Encoder). Sea $G = (V, E)$ una gráfica, donde V denota el conjunto de vértices y E el conjunto de aristas. Un encoder es una función

$$\phi : V \longrightarrow \mathbb{R}^d$$

que asigna a cada vértice $v \in V$ un encaje $z_v \in \mathbb{R}^d$, el cual se obtiene a partir de la información local de v , sus vecinos $\mathcal{N}(v)$ y las aristas de la gráfica $E(G)$, esto es:

$$\phi(v, \mathcal{N}(v), E(G)) = z_v, \quad \text{con } z_v \in \mathbb{R}^d.$$

Por simplicidad denotaremos $\phi(v, \mathcal{N}(v), E(G)) := \phi(v)$.



Notamos que a partir de la codificación de los vértices de una gráfica se obtiene un vector que contiene la información estructural de la gráfica, lo que permite que cada vértice sea representado en conjunto como una matriz, por lo cual de forma similar a la matriz de adyacencia de una gráfica y de la definición de encoder, podemos definir la matriz de encajes de una gráfica.

Definición 59 (Matriz de encajes). Sea $G = (V, E)$ una gráfica, con $|V| = \nu_G$. La matriz de encajes es una matriz

$$\mathbf{Z} \in \mathbb{R}^{\nu_G \times d}$$

cuyas filas corresponden a los encajes de los vértices. Para cada $v \in V$, el vector de encaje asociado se denota por

$$\phi(v) = \mathbf{Z}[v],$$

donde $\mathbf{Z}[v]$ representa la fila de $\mathbf{Z}$ correspondiente al vértice v , y $\phi(v) = z_v \in \mathbb{R}^d$ es el vector de encaje del nodo v .

Dicha matriz de encajes nos puede ser útil para representar la información estructural de la gráfica en un espacio vectorial, lo que facilita su uso y manipulación en distintas tareas de aprendizaje automático.

Por otro lado, podemos definir el concepto de decoder como la función que toma los encajes producidos por el encoder y genera una predicción en el espacio de salida.

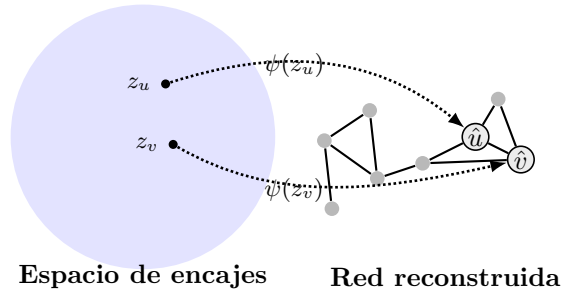
Recordemos que en el contexto de autoencoders sobre gráficas, el decoder busca reconstruir la estructura original de la gráfica (por ejemplo, su matriz de adyacencia) o inferir propiedades derivadas de ella, utilizando la información contenida en los vectores de encaje.

Definición 60 (Decoder). Sea $z_v \in \mathbb{R}^d$ el vector de encaje asociado a un vértice $v \in V$, obtenido mediante un encoder. Se denomina decoder a una función

$$\psi : \mathbb{R}^d \longrightarrow \mathcal{Y},$$

que asigna a cada representación latente z_v una predicción en el espacio de salida $\mathcal{Y}$, de modo que

$$\psi(z_v) = \hat{y}.$$



Los conceptos previamente definidos son esenciales en el contexto de las GNN, ya que permiten transformar la información estructural de las gráficas en representaciones vectoriales más fácilmente manejables para distintas tareas de aprendizaje automático, como clasificación de nodos, predicción de enlaces y generación de gráficas.

En secciones posteriores, explicaremos el uso de estos conceptos en las arquitecturas de redes utilizadas para los objetivos de este trabajo, mostrando cómo los encoders y decoders se integran en el proceso de aprendizaje de las GNN para capturar y utilizar la información estructural de las gráficas de manera efectiva.

3.3.2. Propagación de Mensajes Neuronales

El concepto de **Propagación de Mensajes Neuronales** se refiere al mecanismo mediante el cual las representaciones de los vértices de una gráfica, se actualizan mediante el intercambio de información de su vecindario. En cada paso del mecanismo, los nodos intercambian mensajes que contienen información sobre el estado en que se encuentran para luego combinarse y actualizar su información interna.

Esto viene motivado por la necesidad de capturar relaciones y dependencias entre los vértices de una gráfica para estudiar su comportamiento estructural.

El concepto de **Propagación de Mensajes Neuronales**, permite estudiar a la gráfica, no sólo a nivel de característica de vértice, sino también a nivel estructural de gráfica.

Cómo se mencionó anteriormente, durante cada iteración del mecanismo en una GNN (Red Neuronal Gráfica), los encajes ocultos $h_u^{(k)}$ correspondientes a cada nodo $u \in V$, son actualizadas de acuerdo a la información recopilada del vecindario de u . El mecanismo puede ser definido de la siguiente forma:

Definición 61 (Propagación de mensajes neuronales). Sea $G = (V, E)$ una gráfica, $N_G(v)$ el vecindario del nodo $v \in V$, $h_u^{(t)}$ los encajes ocultos de cada nodo u , y $\mathcal{A}$ una función de agregación que combina los encajes de los vecinos de v . Entonces, el mecanismo de propagación de mensajes se define como:

$$m_v^{(t)} = \mathcal{A}(\{h_u^{(t)} : u \in N_G(v)\}).$$

Una vez que el vértice ha recibido el mensaje, la actualización de su información es el resultado de la combinación del mensaje recibido, con la información actual que tenía y se define de la siguiente forma:

Definición 62 (Actualización de estados neuronales). Sea $h_v^{(t-1)}$ el encaje oculto del nodo v en la capa anterior $(t-1)$, y $m_v^{(t)}$ el mensaje agregado en la capa t . Entonces, la actualización del estado del nodo está dada por:

$$h_v^{(t)} = \mathcal{U}(h_v^{(t-1)}, m_v^{(t)}),$$

donde $\mathcal{U}$ es una función de actualización, típicamente implementada mediante una red neuronal, por ejemplo, una capa lineal seguida de una función de activación (como ReLU).

Después de K iteraciones del mecanismo, podemos utilizar la salida del final de la capa para definir los encajes de cada vértice, es decir:

$$z_u = h_u^{(K)}.$$

Con lo anterior, se ha establecido el marco general para la propagación de mensajes en las GNN, que permite capturar la información estructural de las gráficas a través del intercambio iterativo de mensajes entre los nodos.

Iteración t : Propagación local

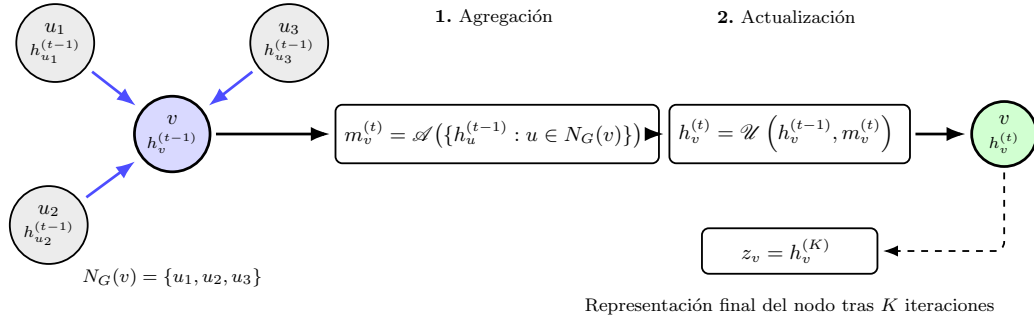


Figura 3.11: Esquema del mecanismo de propagación de información en una red neuronal sobre grafos. En la iteración t , cada nodo v agrega las representaciones de sus vecinos mediante una función invariante al orden $\mathcal{A}$, obteniendo $m_v^{(t)}$. Posteriormente, su representación se actualiza mediante $\mathcal{U}$, combinando el estado previo con la información agregada. Tras K iteraciones, se obtiene la representación final $z_u = h_u^{(K)}$, que captura información estructural del grafo.

3.3.3. La GNN fundamental

Hasta el momento, se ha estudiado el marco de las GNN de una manera un poco abstracta utilizando las funciones que definimos en la sección anterior e iterando K veces. Para introducir una manera de abordarlas de forma más general, introducimos el modelo GNN más básico. Podemos definir el modelo GNN fundamental mediante la siguiente ecuación:

Definición 63 (Modelo GNN fundamental). *Sea $G = (V, E)$ una gráfica, x_u la característica del nodo $u \in V$, y $\mathcal{N}(u)$ el conjunto de vecinos de u . La actualización de la representación del nodo u en una capa de la GNN se define como:*

$$h_u = \phi \left(x_u, \bigoplus_{v \in \mathcal{N}(u)} \psi(x_u, x_v) \right).$$

En esta expresión, h_u representa la nueva representación del nodo u después de aplicar una capa de la red.

La variable x_u corresponde a las características originales del nodo antes de la actualización.

La función $\psi(x_u, x_v)$ transforma las características del nodo u y de su vecino v , generando un mensaje o interacción entre ambos.

La operación $\bigoplus_{v \in \mathcal{N}(u)}$ denota una función de agregación —como la suma, el promedio o el max pooling— que combina los mensajes provenientes de los vecinos de u .

Finalmente, $\phi(\cdot)$ es la función de actualización que integra las características propias del nodo con la información agregada de su vecindario para producir la nueva representación h_u .

En términos generales, el comportamiento del modelo GNN fundamental basado en el mecanismo descrito anteriormente puede estructurarse en los siguientes pasos:

Mensajes entre nodos vecinos ($\psi(x_u, x_v)$). La función ψ calcula la interacción entre las características del nodo u y las de cada uno de sus vecinos v . Esta operación permite capturar las relaciones entre nodos de manera dependiente de sus características, es decir, la forma en que un nodo percibe a su entorno en función de su propio estado y el de sus vecinos.

Agregación ($\bigoplus$). Una vez obtenidas las interacciones entre u y sus vecinos, estas se combinan mediante una operación de agregación $\bigoplus$, que suele ser una suma, un promedio o una operación de *max pooling*. El propósito de esta etapa es sintetizar la información proveniente de todos los vecinos de u en un único vector representativo.

Actualización (ϕ). Finalmente, la función ϕ toma la característica original del nodo u , representada por x_u , junto con el resultado de la agregación, para producir la nueva representación h_u . La función ϕ suele implementarse como una red neuronal *feedforward* o alguna otra transformación no lineal capaz de integrar la información propia del nodo con la de su vecindario.

En síntesis, el modelo GNN fundamental expresa el proceso de **message passing** en una GNN, donde los mensajes de los vecinos de un nodo se combinan para actualizar la representación del nodo central, aprovechando las simetrías presentes en las gráficas.

3.4. Redes Gráficas Convolucionales (GCNs)

A lo largo de este capítulo se han revisado los conceptos fundamentales que abarcan desde las redes neuronales artificiales (RNA) hasta las redes neuronales gráficas (GNN). Las definiciones y formulaciones previamente presentadas han permitido establecer una base teórica sólida para comprender las arquitecturas más avanzadas que se estudiarán a continuación. En particular, el siguiente apartado se enfoca en una familia específica de modelos que representan una extensión natural de las redes neuronales convolucionales (CNN) al dominio de las estructuras de gráficas.

Las **Redes Convolucionales Gráficas** (*Graph Convolutional Networks*, GCNs) constituyen una clase particular dentro del marco general de las Redes Neuronales Gráficas. Su objetivo principal es generalizar el operador de convolución, ampliamente utilizado en el procesamiento de datos estructurados en forma de cuadrículas —como imágenes o señales espaciales—, hacia el contexto más general de las gráficas, donde la información está distribuida de manera irregular y las relaciones entre elementos se describen mediante aristas.

En esencia, las GCNs trasladan los principios de las CNNs al dominio no euclidiano de las gráficas, permitiendo el aprendizaje de representaciones de nodos, aristas o incluso gráficas completas a partir de la topología de las conexiones y las características asociadas a cada entidad. Este enfoque posibilita capturar dependencias estructurales complejas, manteniendo la eficiencia y la capacidad de generalización propias de los modelos convolucionales tradicionales.

Definición 64 (Arquitectura de una GNN). *Sea $G = (V, E)$ una gráfica, $\mathbf{A}$ su matriz de adyacencia, $\mathbf{F}$ una función de propagación neuronal (que incluye la activación no lineal), y $\Phi = \{\Phi_0, \Phi_1, \dots, \Phi_{K-1}\}$ el conjunto de parámetros entrenables de la red. Entonces, la arquitectura general de una **Red Neuronal Gráfica** (GNN) de K capas se define de manera recursiva como:*

$$\begin{aligned}\mathbf{H}_1 &= \mathbf{F}(\mathbf{X}, \mathbf{A}, \Phi_0), \\ \mathbf{H}_2 &= \mathbf{F}(\mathbf{H}_1, \mathbf{A}, \Phi_1), \\ &\vdots \\ \mathbf{H}_K &= \mathbf{F}(\mathbf{H}_{K-1}, \mathbf{A}, \Phi_{K-1}),\end{aligned}$$

donde $\mathbf{X}$ representa la matriz de características iniciales de los nodos y $\mathbf{H}_K$ contiene las representaciones (o encajes) finales resultantes después de K pasos de propagación de información. En términos generales, cada capa aplica una transformación y agregación de información basada en la estructura de la gráfica y las características de sus nodos.

Un aspecto fundamental de las GNNs es su capacidad para manejar la invariancia estructural de las gráficas, es decir, que el orden en que se indexan los nodos no debe afectar el resultado final de la red. Esta propiedad se conoce como **equivarianza a permutaciones** y se define formalmente a continuación:

Definición 65 (Equivarianza). *Sea $\mathbf{F}$ una función de propagación en una capa de la GNN, $\mathbf{H}$ la matriz de representaciones ocultas de los nodos, y $\mathbf{P}$ una matriz de permutación que reordena los nodos de la gráfica. Decimos que una capa es **equivariante a permutaciones***

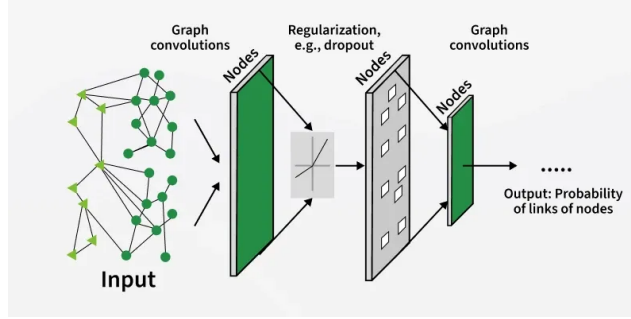


Figura 3.12: Ilustración de convoluciones en gráficos

si se cumple la siguiente relación:

$$\mathbf{H}_{K+1}\mathbf{P} = \mathbf{F}(\mathbf{H}_K\mathbf{P}, \mathbf{P}^\top \mathbf{A}\mathbf{P}, \Phi_K),$$

donde $\mathbf{A}$ denota la matriz de adyacencia de la gráfica G . Esta propiedad garantiza que el resultado de la red no depende del orden en que los nodos son indexados, lo cual es fundamental para preservar la invariancia estructural del grafo.

En conclusión, en este capítulo se han establecido los fundamentos teóricos necesarios para comprender el funcionamiento de las GNN's y, en particular, de las redes Convolucionales Gráficas (GCNs). Se han descrito sus componentes clave, como la propagación de mensajes y la actualización de representaciones, así como las propiedades estructurales que garantizan su correcta operación sobre datos en forma de gráficas. Este conocimiento servirá como base para explorar arquitecturas específicas de GCNs y sus aplicaciones en diversas tareas de aprendizaje automático sobre gráficas en los capítulos siguientes. En capítulos posteriores se introducirá el modelo **PointNet**, que es una arquitectura basada en GNNs diseñada específicamente para procesar nubes de puntos 3D, aprovechando las propiedades de las GCNs para capturar la estructura espacial y las relaciones entre los puntos en el espacio tridimensional. Aunque dicho modelo no emplea explícitamente convoluciones gráficas, su diseño se inspira en los principios fundamentales de las GNNs para manejar datos no estructurados de manera eficiente. Su análisis permitirá explorar cómo estos fundamentos pueden aplicarse a dominios no estructurados, ampliando así la comprensión y el alcance de las arquitecturas neuronales basadas en gráficas.

Capítulo 4

Construcción de algoritmo de muestreo

En los últimos años, el Aprendizaje Profundo (*Deep Learning*) se ha consolidado sobre bases matemáticas sólidas procedentes del álgebra lineal, el análisis matemático y la estadística [6]. Sin embargo, la interacción entre ambas disciplinas no se limita a la fundamentación teórica de los modelos; por el contrario, las redes neuronales se emplean cada vez más como herramientas exploratorias para analizar la estructura de objetos matemáticos abstractos y complejos [1].

En este contexto, el presente capítulo aborda el estudio y modelado de superficies tridimensionales construidas a partir de nudos de Lissajous, definidos mediante funciones armónicas periódicas que dan lugar a variedades con diversas configuraciones topológicas y grados de autointersección. Si bien las Redes Neuronales de Gráficas (*Graph Neural Networks*, GNNs) [13] han demostrado ser efectivas en el dominio de datos no euclidianos, las implementaciones convencionales presentan limitaciones para reconocer invariantes topológicos debido a que el muestreo euclidiano tradicional pierde la conectividad y adyacencia intrínsecas de la variedad [5].

Con el fin de superar esta limitación, el objetivo principal de este capítulo es desarrollar la metodología para la generación de estas superficies e implementar un algoritmo de muestreo adaptativo no euclidiano. Tal como proponen [5], este esquema de muestreo preserva la estructura topológica y la información de adyacencia, optimizando el entrenamiento y la capacidad predictiva de las redes de aprendizaje profundo. A lo largo del capítulo se detallará la metodología empleada, la cual integra el procesamiento geométrico, la representación gráfica, el algoritmo de muestreo diseñado y la evaluación de las GNNs en tareas de clasificación topológica, resaltando el potencial de estas herramientas para abordar problemas fundamentales en la teoría de nudos y la geometría computacional.

4.1. Construcción de superficies a partir de símlices

La metodología seguida para estudiar nudos de Lissajous desde un enfoque basado en gráficas comienza con la generación de superficies en $\mathbb{R}^3$ a partir de curvas de Lissajous “engordadas”, con el objetivo de obtener objetos tridimensionales con volumen. Estas superficies se discretizan mediante una triangulación utilizando la librería **trimesh** de Python, lo cual permite representar la geometría del nudo como una malla finita compuesta por vértices y caras triangulares. Un archivo en formato STL describe este tipo de superficies poligonales en $\mathbb{R}^3$ mediante una colección finita de triángulos. A partir de esta representación, se construye una estructura combinatoria asociada asignando un nodo a cada vértice geométrico y añadiendo, por cada triángulo, las tres aristas que unen sus vértices. Este procedimiento,

implementado mediante la librería **networkX**, produce una gráfica que corresponde al 1-esqueleto de la triangulación de la superficie. En este contexto, la malla triangulada induce de manera natural un complejo simplicial bidimensional $K = (V, E, F)$, donde V es el conjunto de vértices, E el conjunto de aristas y F el conjunto de caras triangulares (véase la definición formal en el Capítulo 1). La gráfica construida mediante **networkX** representa entonces el 1-esqueleto de dicho complejo. No obstante, para estudiar rigurosamente el objeto como una superficie, no basta considerar únicamente la gráfica; es necesario incorporar también la información de las caras triangulares. La realización geométrica del complejo simplicial K proporciona una aproximación discreta de la superficie original y permite analizar propiedades topológicas globales como la conectividad, la presencia de frontera y la característica de Euler,

$$\chi = |V| - |E| + |F|.$$

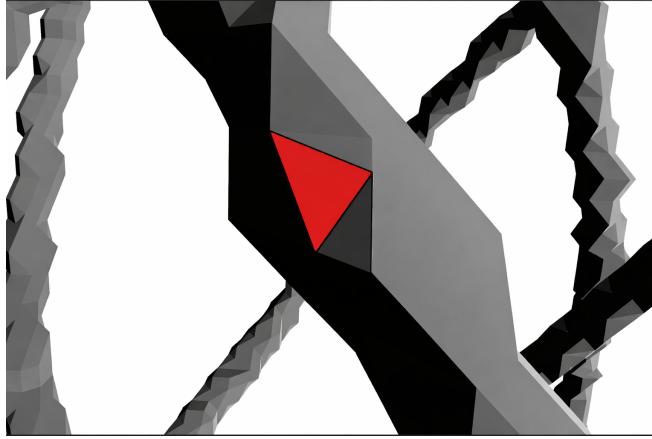


Figura 4.1: Identificación de una cara triangular en la malla asociada a una superficie triangulada. La región resaltada en rojo representa un 2-símplice del complejo simplicial bidimensional inducido por la triangulación, mientras que sus lados corresponden a 1-símplices y sus vértices a 0-símplices. La colección de estas caras, junto con sus aristas y vértices, determina la estructura combinatoria de la superficie discretizada.

Desde un punto de vista topológico, esta construcción permite verificar si la malla define una 2-variedad. En efecto, si cada arista pertenece exactamente a dos triángulos y el entorno de cada vértice es homeomorfo a un disco, entonces la realización geométrica del complejo es una superficie. Dado que la triangulación es finita, esta superficie es compacta; y si además no existen aristas de borde, se trata de una superficie cerrada. En términos computacionales, aunque la construcción de la gráfica a partir del archivo STL captura correctamente la estructura de adyacencia entre vértices, es fundamental almacenar explícitamente las caras triangulares para preservar la naturaleza bidimensional del objeto. Esto se logra manteniendo un registro de las ternas de vértices que definen cada triángulo, lo cual permite reconstruir el complejo simplicial completo. Finalmente, esta representación discreta resulta fundamental para el análisis mediante técnicas modernas de aprendizaje profundo sobre gráficas, como las Graph Neural Networks (GNNs), ya que facilita tanto la visualización —apoyada en herramientas como **matplotlib**— como el procesamiento estructural de los nudos desde una perspectiva combinatoria y geométrica.

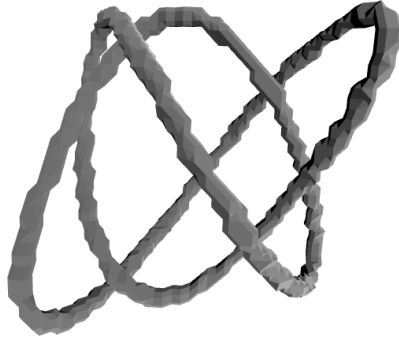


Figura 4.2: Superficie triangulada generada a partir de una curva de Lissajous engrosada en $\mathbb{R}^3$. La discretización induce un complejo simplicial bidimensional $K = (V, E, F)$, en el cual cada cara corresponde a un 2-símplice. La superficie es compacta y sin frontera, es decir, una superficie cerrada, y posee género 1, por lo que es topológicamente equivalente a un toro. La representación presenta una estructura entrelazada con auto-intersecciones aparentes en su proyección, producto de la parametrización periódica subyacente. Esta modelación discreta permite el análisis de propiedades topológicas mediante su estructura combinatoria, incluyendo el cálculo de invariantes como la característica de Euler.

4.2. Transformación superficie a gráfica

A partir de la triangulación de la superficie, el objeto geométrico puede describirse mediante un complejo simplicial bidimensional $K = (V, E, F)$, donde V es el conjunto de vértices, E el conjunto de aristas y F el conjunto de caras triangulares. Esta representación conserva la información bidimensional de la superficie y permite estudiar propiedades topológicas globales, como la conectividad, la presencia de frontera y la característica de Euler.

Sin embargo, para ciertos procedimientos computacionales y de visualización, resulta conveniente considerar únicamente la estructura subyacente formada por los vértices y las aristas del complejo. En este caso, se induce de manera natural una gráfica no dirigida $G = (V, E)$, obtenida al omitir el conjunto de caras F y conservar únicamente la relación de adyacencia entre vértices. Desde este punto de vista, cada vértice del complejo simplicial se interpreta como un nodo de la gráfica, mientras que cada arista simplicial se interpreta como una arista de la gráfica.

Esta reducción del complejo simplicial a su 1-esqueleto es especialmente útil cuando el interés se centra en el análisis estructural del objeto, ya que permite trabajar con una representación combinatoria más ligera, sin perder la información de conectividad local. En términos formales, la gráfica G codifica la incidencia entre los vértices de la malla y proporciona una estructura adecuada para estudiar nociones propias de teoría de gráficas, tales como caminos, grados, vecindades, componentes conexas y recorridos sobre la superficie discretizada.

Además, dado que cada nodo conserva su posición geométrica en $\mathbb{R}^3$, es posible dotar a la gráfica de una realización espacial. Es decir, a cada nodo $v \in V$ se le asocia un atributo de posición

$$\text{pos}(v) = (x_v, y_v, z_v) \in \mathbb{R}^3,$$

de modo que la gráfica no sólo mantiene su estructura combinatoria, sino también su inserción geométrica en el espacio tridimensional. Esto permite visualizar la gráfica respetando la forma de la superficie original.

Con base en esta representación, la visualización se lleva a cabo construyendo, por un lado, el conjunto de nodos a partir de sus coordenadas espaciales y, por otro, el conjunto de aristas a partir de las parejas de vértices adyacentes. De esta manera, la gráfica $G = (V, E)$ puede representarse en tres dimensiones como una nube de nodos enlazados por segmentos, lo cual permite inspeccionar visualmente la conectividad de la estructura triangulada y analizar la distribución espacial de sus vértices.

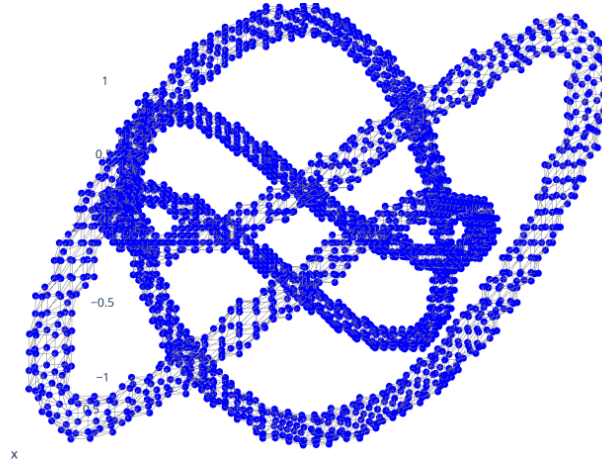


Figura 4.3: Representación tridimensional del 1-esqueleto de la superficie triangulada asociada a una curva de Lissajous engrosada. Los nodos, mostrados en azul, corresponden a los vértices de la malla, mientras que las aristas representan la conectividad entre vértices adyacentes en el complejo simplicial. La estructura resultante define una gráfica no dirigida embebida en $\mathbb{R}^3$, que preserva la geometría global de la superficie original. Esta visualización permite analizar la distribución espacial de los nodos, así como las relaciones de vecindad y conectividad inherentes a la discretización.

4.3. Muestreo por árbol generador mínimo

Durante el análisis de la estructura de la gráfica, se identificó un problema asociado al procesamiento de los datos, derivado de las limitaciones del entorno local. Para abordar este desafío, se desarrolló un conjunto de métodos y técnicas orientadas a realizar un muestreo eficiente sobre la superficie y los nodos del grafo, preservando al mismo tiempo su estructura original. Uno de los principales retos en la obtención de muestras de una gráfica consiste en definir un esquema de recorrido que evite imponer un orden arbitrario y garantice que cada nodo sea visitado a lo más una vez. Esta propiedad es fundamental para prevenir sesgos en los resultados, los cuales pueden surgir debido al orden en que los nodos son procesados. Con el objetivo de realizar un muestreo aleatorio sin un orden predefinido, se proponen diversas técnicas que surgen de la integración de dos conceptos introducidos previamente en el Capítulo 2.

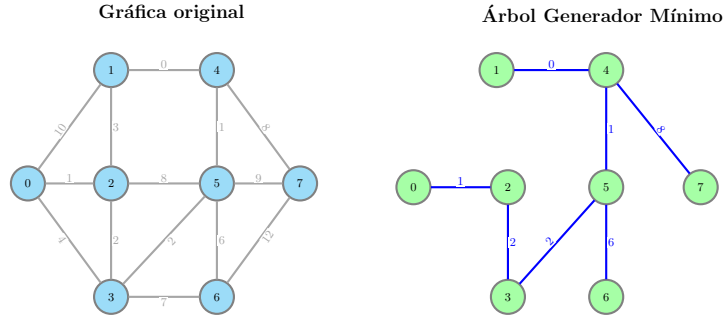


Figura 4.4: Comparación entre la gráfica original y el árbol generador mínimo obtenido.

En primer lugar, se construye un árbol generador mínimo a partir de la matriz de adyacencia A de la gráfica G , utilizando el algoritmo de Kruskal, el cual se describe a continuación:

Algorithm 3 Algoritmo de Kruskal para el Árbol de Expansión Mínima (MST)

Require: Grafo no dirigido $G = (V, E)$ con pesos en las aristas

Ensure: Árbol de expansión mínima (MST)

```

1: Inicializar  $MST \leftarrow \emptyset$                                 ▷ Conjunto vacío para el MST
2: Ordenar las aristas  $E$  en orden creciente de pesos
3: Inicializar estructura Union-Find para los vértices  $V$ 
4: for cada arista  $(u, v) \in E$  en orden creciente de peso do
5:   if  $\text{FIND}(u) \neq \text{FIND}(v)$  then                            ▷ Si u y v no están en el mismo componente
6:      $MST \leftarrow MST \cup \{(u, v)\}$                         ▷ Añadir arista al MST
7:      $\text{UNION}(u, v)$                                            ▷ Unir los componentes de u y v
8:   end if
9:   if Número de aristas en MST  $= |V| - 1$  then                ▷ MST completo
10:    break
11:   end if
12: end for
13: return  $MST$                                                 ▷ Devolver el árbol de expansión mínima

```

Posteriormente a utilizar el algoritmo de Kruskal para generar el árbol generador mínimo, convertimos el árbol resultante a una matriz simétrica de la siguiente forma : $A_s = A_{mst} + A_{mst}^T$.

El objetivo de generar el árbol generador mínimo es recorrer la estructura de la gráfica completa sin recorrer todas las aristas, sino sólo las más importantes que en este caso, son las que dan la estructura a la gráfica original. Una vez obtenido el MST, procedemos a recorrerlo mediante un orden impuesto por el algoritmo "DFS", el cual es un algoritmo que sirve para recorrer una gráfica en profundidad, es decir a lo "largo" asegurando así que visitaremos todos los nodos de dicha gráfica para preservar lo que nos interesa que es la estructura de la gráfica.

A continuación enunciaremos el algoritmo DFS, y su implementación en nuestro método:

Algorithm 4 DFS Order

Require: Nodo actual *nodo*, lista de nodos visitados *visitados*, lista del orden *orden***Ensure:** Lista *orden* con el orden en que los nodos son visitados

```

1: function DFS_ORDER(nodo, visitados, orden)
2:   visitados[nodo]  $\leftarrow$  True ▷ Marcar el nodo como visitado
3:   Agregar nodo a la lista orden ▷ Registrar el nodo en el orden de visita
4:   for all vecino en posiciones donde mst[nodo] = 1 do
5:     if visitados[vecino] = False then
6:       DFS_ORDER(vecino, visitados, orden) ▷ Llamada recursiva para el vecino
       no visitado
7:     end if
8:   end for
9: end function

```

Posterior a establecer el orden en el que se recorrerá la gráfica, generamos un orden entre nodos para visitar los nodos más lejanos del nodo actual, asegurando de esta forma ir lo más lejos posible en el menor número de pasos.

4.4. Muestreo por Esferas

Una de las formas de mantener la estructura original de la gráfica es enfocándonos en su estructura local, es decir, podemos mirar a partir de un nodo v su vecindario, así como también tomar una pequeña esfera $S_v(r)$ con radio r y centro en v que interseque su vecindario $N_G(v)$, para así mantener las propiedades locales y poder asegurar que se preserve la estructura recibida.

Para identificar la muestra resultado de la intersección entre $N_G(V)$ y la esfera $S_v(r)$ se ha construido un método denominado Método polar para desviaciones normales [9, p. 122-123]. A continuación describiremos un algoritmo que sirve como base a la implementación realizada para muestrear uniformemente puntos en el volumen de una esfera, se conoce cómo *Método polar para desviaciones normales* o *Método polar de Marsaglia*.

Algorithm 5 Método polar para desviaciones normales

-
- 1: **P1** [Obtener variables uniformes]
 - 2: Generar dos variables aleatorias independientes $U_1, U_2 \sim U(0, 1)$.
 - 3: Asignar $V_1 \leftarrow 2U_1 - 1$, $V_2 \leftarrow 2U_2 - 1$, ahora $V_1, V_2 \sim U(-1, 1)$.
 - 4: **P2** [Calcular S]
 - 5: Asignar $S \leftarrow V_1^2 + V_2^2$.
 - 6: **P3** [*¿Es $S \geq 1$?*]
 - 7: **if** $S \geq 1$ **then**
 - 8: Regresar al paso **P1**.
 - 9: **end if**
 - 10: **P4** [Calcular X_1, X_2]
 - 11: Asignar:
-

$$X_1 \leftarrow V_1 \sqrt{\frac{-2 \ln S}{S}}, \quad X_2 \leftarrow V_2 \sqrt{\frac{-2 \ln S}{S}}$$

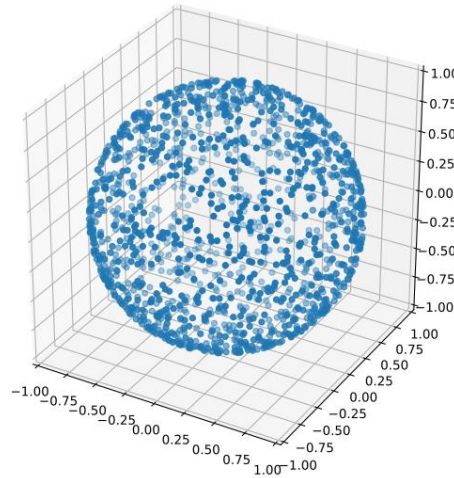


Figura 4.5: La figura muestra la distribución de un conjunto de puntos generados mediante un proceso de muestreo aleatorio uniforme sobre la superficie de una esfera unitaria centrada en el origen. Cada punto corresponde a una realización de variables aleatorias transformadas a coordenadas tridimensionales (x, y, z) , garantizando una cobertura homogénea sin sesgos de concentración en regiones específicas. Esta representación permite visualizar la correcta dispersión espacial de los puntos, lo cual es fundamental para aplicaciones posteriores como la construcción de grafos geométricos, análisis de vecindades locales o procesos de aprendizaje sobre estructuras tridimensionales.

Para probar la validez del método anterior utilizaremos geometría analítica y un poco de cálculo.

Demostración. Notamos que si $S < 1$ en el paso *P3*, el punto en el plano cartesiano con las coordenadas (V_1, V_2) es un punto aleatorio uniformemente distribuido dentro del círculo

unitario.

Transformando a coordenadas polares tenemos que: $V_1 = R\cos\theta$, $V_2 = R\sin\theta$ y sustituyendo tenemos que:

$$S = R^2, \quad X_1 = (R\cos\theta)\sqrt{\frac{-2\ln S}{R^2}}, \quad X_2 = (R\sin\theta)\sqrt{\frac{-2\ln S}{R^2}}$$

De aquí observamos que:

$$\begin{aligned} X_1 &= (R\cos\theta)\sqrt{\frac{-2\ln S}{R^2}} \\ &= R\cos\theta \frac{\sqrt{-2\ln S}}{\sqrt{R^2}} \\ &= R\cos\theta \frac{\sqrt{-2\ln S}}{|R|} \\ &= \cos\theta\sqrt{-2\ln S} \end{aligned}$$

Pues $R > 0$ De igual forma $X_2 = \sin\theta\sqrt{-2\ln S}$. Utilizando coordenadas polares y haciendo:

$$R' = \sqrt{-2\ln S}, \quad \theta' = \theta$$

Tenemos que $X_1 = R'\cos\theta'$, $X_2 = R'\sin\theta'$ Así, observamos que R' y θ' son independientes pues R y θ lo son. También $\theta' \sim U(0, 2\pi)$ pues es el rango de las funciones $\sin$ y $\cos$.

A su vez notamos que:

$$\begin{aligned} P(R' \leq r) &= P(-2\ln S \leq r^2) \\ &= P(S \geq e^{-\frac{r^2}{2}}) \end{aligned}$$

Por otro lado , como $S \sim U(0, 1)$ entonces su función de distribución acumulada es :

$$F(s) = P(S \leq s) = s, \quad s \in [0, 1]$$

De esta forma la probabilidad complementaria es:

$$\begin{aligned} P(S < s) &= 1 - P(S \geq s) \\ P(S \geq s) &= 1 - P(S < s) \end{aligned}$$

Por tanto:

$$\begin{aligned} P(S \geq e^{-\frac{r^2}{2}}) &= 1 - P(S < e^{-\frac{r^2}{2}}) \\ &= 1 - e^{-\frac{r^2}{2}} \end{aligned}$$

Ahora, la probabilidad que R' esté entre r y $r + dr$ es la derivada de $1 - e^{-\frac{r^2}{2}}$ que es igual a $re^{-\frac{r^2}{2}}dr$. Similarmente la probabilidad de que θ' esté entre θ y $\theta + d\theta$ es $\frac{1}{2\pi}d\theta$ De esta forma, la probabilidad conjunta de $X_1 \leq x_1$ y $X_2 \leq x_2$ es:

$$\begin{aligned} \int_{\{(r,\theta)|r\cos\theta\leq x_1, r\sin\theta\leq x_2\}} \frac{1}{2\pi} e^{-\frac{r^2}{2}} r dr d\theta &= \frac{1}{2\pi} \int_{\{(x,y)|x\leq x_1, y\leq x_2\}} e^{-\frac{(x^2+y^2)}{2}} dx dy \\ &= \left(\sqrt{\frac{1}{2\pi}} \int_{-\infty}^{x_1} e^{-\frac{x^2}{2}} dx \right) \left(\sqrt{\frac{1}{2\pi}} \int_{-\infty}^{x_2} e^{-\frac{y^2}{2}} dy \right) \end{aligned}$$

Lo cual prueba que X_1 y X_2 son variables aleatorias independientes con distribución normal, tal como se requería. □

Basado en el algortimo 3, podemos derivar un método que permita generar puntos aleatorios dentro de una esfera en 3 dimensiones. Sean $U_1, U_2 \sim \mathcal{U}(0, 1)$ entonces, hacemos V_1, V_2 y S como:

$$V_1 = 2U_1 - 1, \quad V_2 = 2U_2 - 1, \quad S = V_1^2 + V_2^2$$

Notamos entonces que $V_1, V_2 \sim \mathcal{U}(-1, 1)$

Por otro lado, sabemos que la esfera $\mathbb{S}^2$ tiene las siguientes coordenadas:

$$X = \sin(\theta)\cos(\phi), \quad Y = \sin(\theta)\sin(\phi), \quad Z = \cos(\theta)$$

Notamos que $Z = \cos(\theta)$ tiene un rango de valores de $[-1, 1]$, entonces utilizando la transformación $Z = 2S - 1$ para $S = V_1^2 + V_2^2 \leq 1$ esto implica que. $Z = \cos(\theta) = 2S - 1 \in [-1, 1]$ Por otro lado;

$$\begin{aligned} \cos^2(\theta) + \sin^2(\theta) &= 1 \\ \sin^2(\theta) &= 1 - \cos^2(\theta) \\ \sin^2(\theta) &= 1 - Z^2 \\ \sin(\theta) &= \sqrt{1 - Z^2} \end{aligned}$$

Por tanto:

$$X = \sqrt{1 - Z^2}\cos(\phi), \quad Y = \sqrt{1 - Z^2}\sin(\phi) \tag{1}$$

Por otro lado, notamos que por construcción

$$\begin{aligned} S &= V_1^2 + V_2^2 \\ \sqrt{S} &= \sqrt{V_1^2 + V_2^2} \end{aligned}$$

Y además, en el plano XY

$$\begin{aligned}\cos(\phi) &= \frac{V_1}{\sqrt{V_1^2 + V_2^2}} \\ &= \frac{V_1}{\sqrt{S}}\end{aligned}$$

Análogamente: $\sin(\phi) = \frac{V_2}{\sqrt{S}}$

Sustituyendo en 1

$$\begin{aligned}X &= \sqrt{1 - Z^2} \frac{V_1}{\sqrt{S}} \\ &= \frac{\sqrt{1 - Z^2}}{\sqrt{S}} V_1 \\ &= \sqrt{\frac{1 - Z^2}{S}} V_1\end{aligned}$$

Desarrollando $\sqrt{\frac{1 - Z^2}{S}}$:

$$\begin{aligned}\sqrt{\frac{1 - Z^2}{S}} &= \sqrt{\frac{1 - (2S - 1)^2}{S}} \\ &= \sqrt{\frac{1 - ((2S)^2 + (4S)(-1) + 1)}{S}} \\ &= \sqrt{\frac{1 - (4S^2 - 4S + 1)}{S}} \\ &= \sqrt{\frac{-4S^2 + 4S}{S}} \\ &= \sqrt{\frac{4S(-S + 1)}{S}} \\ &= \sqrt{4(1 - S)} \\ &= \sqrt{4}\sqrt{1 - S} \\ &= 2\sqrt{1 - S}\end{aligned}$$

Es decir se toma la parte positiva de la raíz cuadrada ya que se están normalizando coordenadas, por otro lado notamos que $1 - S \geq 0$ pues $S \leq 1$. Así, por convención hacemos $\alpha = 2\sqrt{1 - S}$, por tanto $X = \alpha V_1$ y análogamente, $Y = \alpha V_2$. De esta forma la generación de puntos aleatorios en la superficie de la esfera es:

$$(\alpha V_1, \alpha V_2, 2S - 1)$$

Donde:

$$\begin{aligned}
 X^2 + Y^2 + Z^2 &= (\alpha V_1)^2 + (\alpha V_2)^2 + (2S - 1)^2 \\
 &= \alpha^2(V_1^2 + V_2^2) + (2S - 1)^2 \\
 &= S(2\sqrt{1 - S})^2 + (2S - 1)^2 \\
 &= 4S(1 - S) + 4S^2 - 4S + 1 \\
 &= -4S^2 + 4S + 4S^2 - 4S + 1 \\
 &= 1
 \end{aligned}$$

Por último, para generar puntos aleatorios, dentro del volumen de la esfera, basta con generar un radio aleatorio de la siguiente forma: Dado $R = \mathcal{U}(0, 1)^{\frac{1}{3}}$, los puntos distribuidos dentro de la esfera están dados por:

$$R(\alpha V_1, \alpha V_2, 2S - 1)$$

De esta forma mostraremos el algoritmo resultante en la siguiente forma;

Algorithm 6 Generar Puntos Uniformes en una Esfera

Require: Coordenadas del centro h, k, l , número de puntos num_puntos

Ensure: Matriz de puntos uniformemente distribuidos en la esfera

```

1: function GENERARPUNTOSUNIFORMES( $h, k, l, num\_puntos$ )
2:   Inicializar una lista vacía  $puntos$ 
3:   for  $i \leftarrow 1$  to  $num\_puntos$  do
4:     repeat
5:       Generar  $V_1, V_2 \leftarrow \text{UNIFORM}(-1, 1)$ 
6:       Calcular  $S \leftarrow V_1^2 + V_2^2$ 
7:     until  $S < 1$ 
8:     Calcular  $a \leftarrow \sqrt{1 - S}$ 
9:     Calcular  $x \leftarrow a \cdot V_1 + h$ 
10:    Calcular  $y \leftarrow a \cdot V_2 + k$ 
11:    Calcular  $z \leftarrow 2 \cdot S - 1 + l$ 
12:    Generar  $r \leftarrow \text{UNIFORM}(0, 1)^{\frac{1}{3}}$ 
13:    Agregar  $[r \cdot x, r \cdot y, r \cdot z]$  a la lista  $puntos$ 
14:  end for
15:  return Matriz convertida a partir de  $puntos$ 
16: end function

```

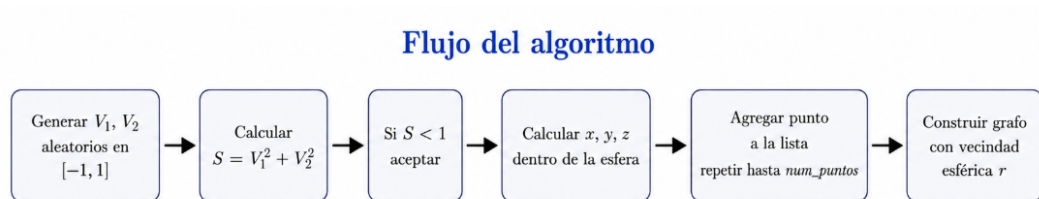
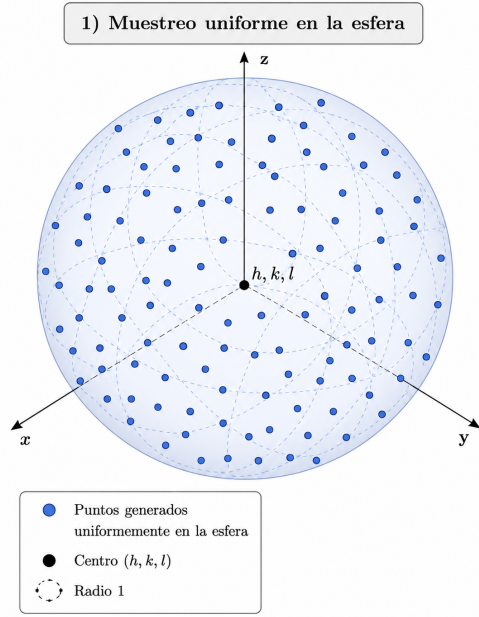
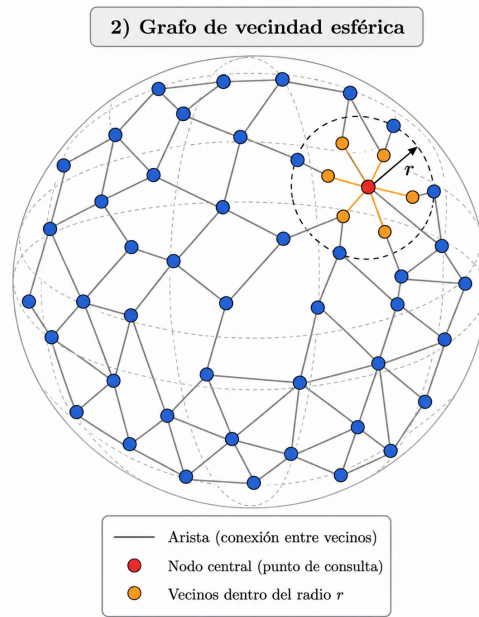


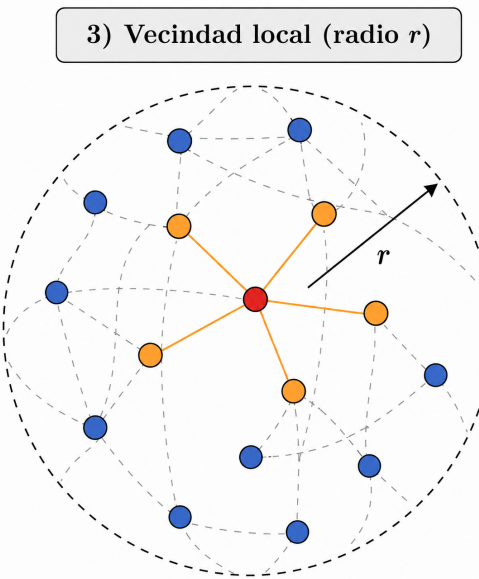
Figura 4.6: Flujo del algoritmo de muestro por esferas.



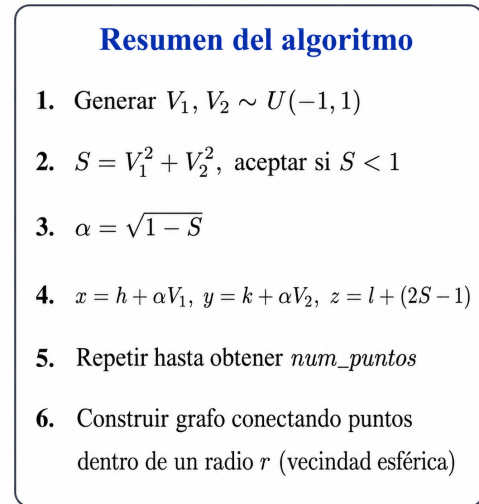
(a) Muestreo uniforme en la esfera.



(b) Grafo de vecindad esférica.



(c) Vecindad local.



(d) Resumen del algoritmo.

Figura 4.7: Proceso de muestreo uniforme y construcción de la gráfica de vecindad esférica.

4.5. Reasignación de adyacencias

En el punto anterior notamos que cuando se elimina un nodo contenido dentro de la esfera de muestreo, los vecinos de este nodo pierden cierta información estructural contenida en el nodo eliminado, para evitar esto y preservar la estructura lo más parecida posible a la inicial hacemos una reasignación de adyacencias.

Describiremos a continuación el proceso para eliminar un nodo v de G y que sus vecinos mantengan la adyacencia entre ellos, es decir si dos nodos u, w eran vecinos de v entonces después de eliminar v , u y w pasaran a ser vecinos entre ellos.

Definición 66. Sea G una gráfica y A su matriz de adyacencia, entonces definimos al elemento $(A)_{ij}$ como el elemento ij -ésimo de la matriz A , donde $i, j \in \{0, \dots, \nu_G - 1\}$

Ejemplo 9. Sea la matriz A dada por :

$$A(G) = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{pmatrix} \quad (4.1)$$

Entonces los elementos $(A)_{02}, (A)_{23}, (A)_{54}$ son iguales a :

$$\begin{aligned} (A)_{02} &= 0 \\ (A)_{23} &= 1 \\ (A)_{54} &= 1 \end{aligned}$$

Dada esta definición podemos ahora establecer una operación que sirve como punto de partida para implementar un método de eliminación controlada para los nodos de la gráfica. La siguiente definición implementará una operación entre los elementos de la matriz de adyacencia A_n donde n representa el paso antes de realizar la operación y por ende la matriz A_{n+1} denota a la matriz de adyacencia A después de la operación.

Definición 67 (Eliminación del nodo k). Sea G una gráfica A_n la matriz de adyacencia en el paso n , A_{n+1} la matriz de adyacencia en el paso $n + 1$, k el nodo a eliminar y $N_G(k)$ el vecindario del nodo k .

Entonces definimos la función,

$$\psi : (A_n)_{ij} \times (A_n)_{kj} \rightarrow (A_{n+1})_{ij}$$

Dada por:

$$\psi_{ij} := (A_{n+1})_{ij} = \begin{cases} (A_n)_{ij} +_{or} (A_n)_{kj} & , \quad i \neq j, j \neq k \\ 0 & , \quad k = j, i = j \end{cases} \quad (4.2)$$

Con $i \in N_G(k) \cup \{k\}$, además $\psi_{kj} = 0, \forall j \in \{0, \dots, \nu_G\}$

Ejemplo 10. Dada la matriz $A_0 = A(G)$ del ejemplo (2), $k = 5$, $\nu_G = 6$, $j \in \{0, \dots, 5\}$ entonces calcularemos los valores de ψ . Observamos que $N_G(k) = \{1, 2, 4\}$, entonces:

$$\begin{aligned}
 \psi_{10} &:= (A_1)_{10} = (A_0)_{10} +_{or} (A_0)_{50} \\
 &= 1 +_{or} 0 \\
 &= 1 \\
 \psi_{11} &:= (A_1)_{11} = 0 \\
 \psi_{12} &:= (A_1)_{12} = (A_0)_{12} +_{or} (A_0)_{52} \\
 &= 1 +_{or} 1 \\
 &= 1 \\
 \psi_{5j} &:= (A_1)_{5j} = 0 \quad \forall j \in \{0, \dots, 5\}
 \end{aligned}$$

Y notamos que análogamente para $i = \{2, 4\}$. De esta forma la matriz A_1 quedaría de la siguiente forma:

$$A_1(G) = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \quad (4.3)$$

Así, hemos definido una operación para las matrices de adyacencia donde, dado un nodo k podemos eliminarlo de dicha matriz, es decir, $(A)_{kj}, (A)_{ik} = 0$, donde $i, j \in \{0, \dots, \nu_G\}$ asegurando que sus vecinos preserven la adyacencia entre ellos, sin modificar la estructura original de la gráfica. De esta forma podemos definimos el algoritmo de la siguiente manera:

Algorithm 7 Eliminar Nodo con Redistribución de Adyacencias

Require: Nodo a eliminar *nodo*, matriz de adyacencia *matrizAdj*

Ensure: Matriz de adyacencia actualizada sin el nodo y con conexiones redistribuidas

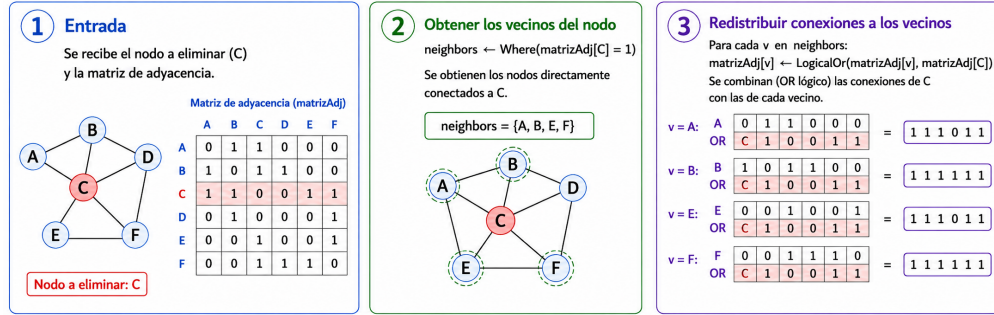
```

1: function ELIMINANODOCONADYACENCIA(nodo, matrizAdj)
2:   Obtener los vecinos del nodo: neighbors  $\leftarrow$  WHERE(matrizAdj[nodo] = 1)  $\triangleright$  Lista
   de nodos directamente conectados a nodo.
3:   for all v en neighbors do
4:     matrizAdj[v]  $\leftarrow$  LOGICALOR(matrizAdj[v], matrizAdj[nodo])  $\triangleright$  Redistribuir
     conexiones del nodo eliminado a sus vecinos.
5:   end for
6:   matrizAdj[nodo, :]  $\leftarrow$  0  $\triangleright$  Eliminar todas las conexiones salientes del nodo.
7:   matrizAdj[:, nodo]  $\leftarrow$  0  $\triangleright$  Eliminar todas las conexiones entrantes al nodo.
8:   return matrizAdj
9: end function

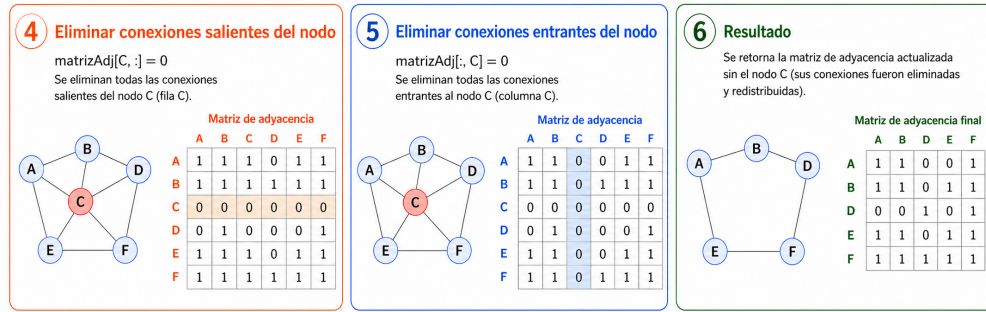
```



Figura 4.8: Flujo del algoritmo de reducción de nodos.



(a) Etapas 1 a 3 del algoritmo.



(b) Etapas 4 a 6 del algoritmo.

Figura 4.9: Proceso de eliminación de un nodo y redistribución de sus conexiones.

4.6. Método general para la reducción de nodos

Una vez establecidos los métodos descritos anteriormente, es posible combinar sus principios para desarrollar un enfoque generalizado de reducción de nodos.

En esta sección, exploraremos en detalle dicho enfoque, examinando su complejidad computacional, describiendo su implementación paso a paso y destacando las ventajas clave que se derivan de su aplicación.

El proceso de muestreo eficiente en una gráfica se basa en reducir el número de nodos mientras se conservan sus propiedades clave. Primero, se genera un orden de recorrido utilizando un árbol generador mínimo y el algoritmo DFS, lo que permite recorrer toda la gráfica y establecer un orden basado en DFS.

En el ciclo principal, se iteran los nodos hasta alcanzar el tamaño deseado de la muestra. Se calculan las distancias promedio entre los nodos ordenados y el centroide de la muestra,

priorizando los más lejanos para evitar sesgos. Con el nuevo orden, se generan puntos alrededor de cada nodo utilizando una esfera, identificando y eliminando vecinos dentro de un radio definido, manteniendo la adyacencia. Este proceso se repite hasta obtener la muestra requerida. El algoritmo se divide en fases para facilitar su comprensión.

Algorithm 8 Método General

Require: Lista de nodos con coordenadas x , Matriz de adyacencia $adjacency$, Tamaño de muestra $size$, Radio $alcance$, Tamaño de la muestra de esfera $esfera_size$

Ensure: Nueva lista de nodos new_x , Matriz de adyacencia Actualizada new_adj

```

1: function SAMPLING( $x, adjacency, size, alcance, esfera\_size$ )
2:   Obtener el árbol generador mínimo (MST) desde  $adjacency$  ▷ Algoritmo 3
3:   Inicializar  $visited[] \leftarrow FALSE$ ;  $order[] \leftarrow empty\_list$ 
4:   Obtener orden mediante DFS  $dfs\_order(0, visited, order)$  ▷ Algoritmo 4
5:   Inicializar  $removed\_nodes[] \leftarrow empty\_list$ ;  $queue[] \leftarrow copy(order)$ 
6:   while  $number\_nodes - removed\_nodes \geq size$  do
7:     if  $removed\_nodes$  no es vacío then
8:        $dist \leftarrow dist(currency\_node, centroid)$ 
9:       Ordena  $queue$  por distancias en orden descendente
10:    end if
11:    Obtener coordenadas del primer elemento de  $queue$ .
12:     $idx \leftarrow queue.pop(); h, k, l \leftarrow x[idx]$ 
13:    Obtenemos esfera uniforme GENERAPUNTOSUNIFORMES( $idx$ ) ▷ Algoritmo 6
14:    Hacemos  $neighbors \leftarrow neighbors \cap GeneraPuntosUniformes(idx)$ 
15:    for all  $vecinos \in vecinos - eliminados$  do
16:      ELIMINANODOCONADYACENCIA( $n, adjacency$ ) ▷ Algoritmo 7
17:    end for
18:  end while
19:  return  $new\_x, new\_adj$ 
20: end function

```

4.7. Análisis de Complejidad

El análisis de complejidad de nuestro algoritmo se realiza descomponiendo sus principales componentes y justificando cada cálculo. La construcción del Árbol Generador Mínimo (MST) utiliza el algoritmo de Kruskal, que tiene una complejidad de $\mathcal{O}(E \log(V))$ [2], donde E es el número de aristas y V el número de vértices de la gráfica. Este paso es esencial para establecer una estructura inicial que guía el muestreo de nodos. El siguiente paso es ordenar los nodos del grafo mediante un recorrido en profundidad (DFS), cuya complejidad es $\mathcal{O}(V + E)$ [2]. Esto se debe a que el DFS visita cada vértice y cada arista exactamente una vez. Este orden nos permite recorrer sistemáticamente los nodos y controlar las operaciones de muestreo.

El bucle principal del algoritmo, que se ejecuta hasta alcanzar el tamaño deseado de la muestra, domina la mayor parte del costo computacional, en el peor de los casos este bucle se ejecuta V veces, considerando que el tamaño de la muestra puede representar una cota

ya que en cada iteración se elimina un nodo por tanto su costo podría acotarse por $\mathcal{O}(V)$.

Dentro de este bucle, el cálculo de distancias entre los nodos y el centroide de la muestra se calcula mediante la norma euclídeana, es decir se calcula la distancia a lo más V veces y se repite por el número d de dimensión que en este caso $d = 3$ por tanto la complejidad total es: $\mathcal{O}(V)$.

Luego, la complejidad del algoritmo para generar la muestra uniforme está determinado por el número de veces que se ejecuta su bucle principal, ya que, las operaciones realizadas no dependen del número de puntos a generar, por lo tanto la complejidad es constante, de esta forma el número de pasos que realiza el método es en el orde de *esfera.size* que, para términos de la implementación de dicho algoritmo, siempre es constante.

La intersección de los vecinos generados en la esfera con los vecinos del nodo en la matriz de adyacencia tiene un costo lineal de $\mathcal{O}(V)$. Este cálculo depende del tamaño de los conjuntos comparados, donde el conjunto mayor está determinado por el número de vértices de la gráfica. Una vez identificados los nodos dentro de la esfera, el algoritmo actualiza la matriz de adyacencia. Dado que esta matriz tiene un tamaño de $V \times V$, las operaciones de reajuste tienen un costo de $\mathcal{O}(V^2)$.

Finalmente, los nodos eliminados deben ser descartados de la matriz de adyacencia, lo que implica un costo adicional de $\mathcal{O}(V)$ por el número de pasos k , donde k es el número de nodos eliminados que para terminos de la implementación lo podemos dejar como una constante, por lo tanto la complejidad final sería $\mathcal{O}(V)$. Este costo se suma al procesamiento necesario para mantener la consistencia de la estructura de datos. Resumiendo, notamos que las operaciones más costosas que otorgan la complejidad final al algoritmo es la obtención del MST $\mathcal{O}(E \log V)$ y las iteraciones del bucle principal, que resulta ser $\mathcal{O}(V * V^2) = \mathcal{O}(V^3)$ pues, las operaciones se ejecutan V veces multiplicado por el reordenamiento de la matriz de adyacencia, entonces dado las dos operaciones mas costosas en diferentes fases del algoritmo la complejidad final está dada por: $\mathcal{O}(E \log V + V^3)$.

Como observación final, en el peor de los casos sabemos que $E = V^2$, pero como estamos trabajando con un tipo de gráficas especiales, para este caso $E \neq V^2$, en específico $E < V^2$ captura equivarianzas sobre la superficie por eso los resultados parecen ser mejores.

Capítulo 5

Resultados

Uno de los principales problemas que motivaron el método de optimización propuesto en este escrito es la dificultad de diversas arquitecturas de aprendizaje profundo para capturar propiedades topológicas globales en estructuras tridimensionales, especialmente en superficies de género mayor que uno. Aunque modelos basados en operadores neuronales, mecanismos de atención o nubes de puntos han mostrado buenos resultados en tareas geométricas, su desempeño puede verse limitado cuando la clasificación depende de invariantes topológicos como la característica de Euler o el género, ya que suelen dar prioridad a información local o relaciones geométricas extrínsecas sin incorporar explícitamente la conectividad de los puntos que conforman la malla. En este contexto surge EuLearn [5], una base de datos diseñada para evaluar modelos de aprendizaje profundo en tareas de clasificación topológica. A diferencia de otros conjuntos de datos tridimensionales enfocados en reconocimiento de objetos, EuLearn contiene superficies con géneros controlados, lo que permite analizar si una arquitectura es capaz de distinguir entre diferentes clases topológicas a partir de campos escalares, nubes de puntos, mallas trianguladas o gráficas asociadas. Uno de los principales retos al trabajar con estas superficies es el tamaño y la complejidad de las mallas, compuestas por una gran cantidad de vértices, aristas y caras. Esto incrementa el costo computacional y dificulta el entrenamiento eficiente de los modelos. Además, una reducción aleatoria de puntos puede eliminar relaciones de adyacencia relevantes para la clasificación topológica. Por ello, en esta tesis se analiza un método de optimización sobre gráficas que reduce la representación original de la malla preservando relaciones estructurales esenciales. El método propuesto puede interpretarse como una estrategia de muestreo estructurado sobre la gráfica asociada a la malla. A diferencia de enfoques basados únicamente en distancia euclidiana, como ocurre en ciertas etapas de arquitecturas tipo PointNet++, esta aproximación considera la conectividad intrínseca de la superficie. De esta manera, se reduce la dimensionalidad de los datos de entrada sin descartar información local relevante, facilitando el entrenamiento y favoreciendo una mejor capacidad de generalización. A lo largo de este capítulo se presentan las arquitecturas utilizadas para evaluar el método propuesto, incluyendo Fourier Neural Operator, Dynamic Graph CNN, PointNet, PointNet++, modelos basados en atención, Graph Sampled PointNet y Graph Sampled Attention. Asimismo, se comparan los resultados obtenidos sobre EuLearn y se discute cómo el uso de información de adyacencia mejora el aprendizaje de propiedades topológicas globales asociadas al género y a la característica de Euler de superficies tridimensionales.

5.1. Arquitecturas utilizadas

A lo largo de la evaluación de las superficies contenidas en la base de datos [5], se utilizan diversas arquitecturas de aprendizaje profundo, cada una con características particulares que las hacen adecuadas para procesar datos tridimensionales. A continuación, se describen

brevemente las principales arquitecturas empleadas.

5.1.1. Fourier Neural Operator

El modelo Fourier Neural Operator (FNO) es una arquitectura diseñada para aprender operadores entre espacios de funciones. En el contexto de EuLearn, esta arquitectura se utiliza sobre la representación continua de la superficie, en lugar de operar directamente sobre la nube de puntos o la malla triangular. Su idea principal consiste primero en transformar información geométrica en un conjunto de frecuencias mediante la transformada de Fourier, posteriormente la red aprende qué patrones de esas frecuencias son relevantes y, finalmente transforma nuevamente la información al espacio original mediante la transformada inversa de Fourier. De esta forma esta estrategia permite capturar relaciones globales en la superficie lo cual es de suma importancia para la clasificación topológica.

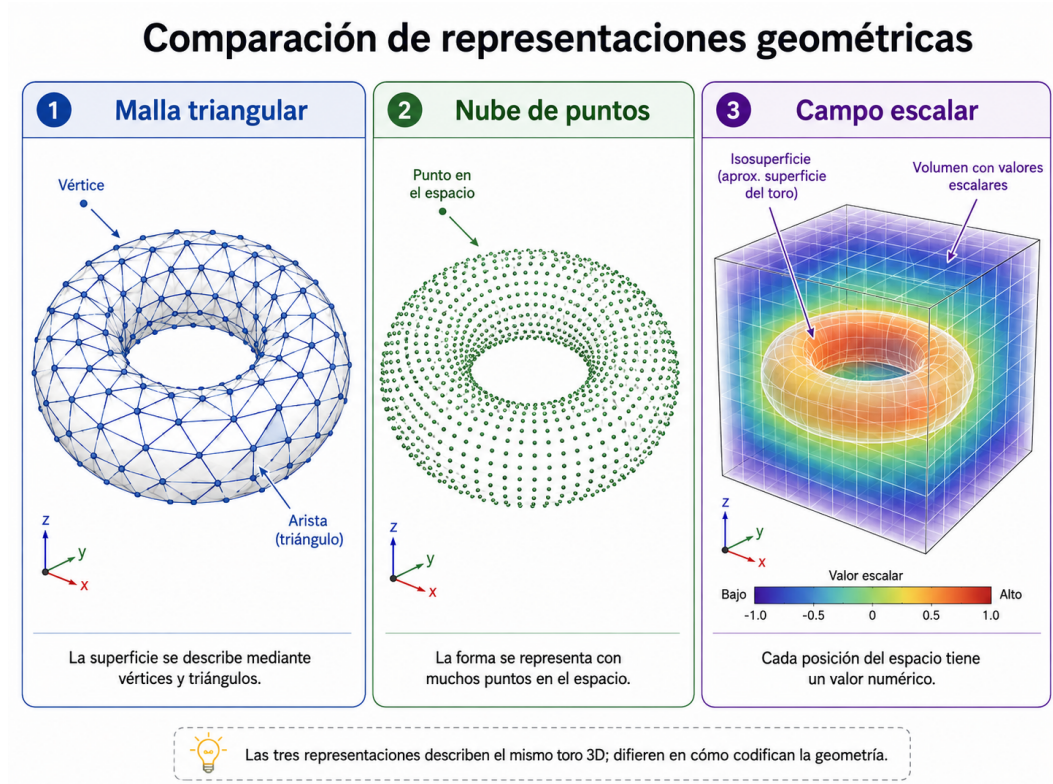


Figura 5.1: Comparación entre malla triangulada, nube de puntos y campo escalar.

De manera general, un bloque FNO puede escribirse como

$$\sigma \left(W v_t(x) + \mathcal{F}^{-1} (R_\theta \cdot \mathcal{F}(v_t)) (x) \right),$$

donde (v_t) es la representación de entrada en la capa (t) , (W) es una transformación lineal local, $(\mathcal{F})$ y $(\mathcal{F}^{-1})$ denotan la transformada de Fourier y su inversa, respectivamente, (R_θ) es un tensor de pesos aprendibles en el dominio frecuencial y (σ) es una función de activación.

5.1.2. Dynamic Graph CNN

La arquitectura Dynamic Graph CNN (DGCNN) es una arquitectura diseñada para procesar nubes de puntos mediante convoluciones definidas sobre gráficas dinámicas. Para cada punto, identifica los más cercanos, usualmente mediante (k)-nearest neighbors, y los conecta formando una gráfica, de esta forma, la red analiza la relación entre cada punto y sus vecinos para aprender características locales de la forma de la superficie. Además, estas conexiones pueden volver a calcularse durante las diferentes capas de la red, por lo que la gráfica cambia dinámicamente conforme el modelo aprende nuevas características.

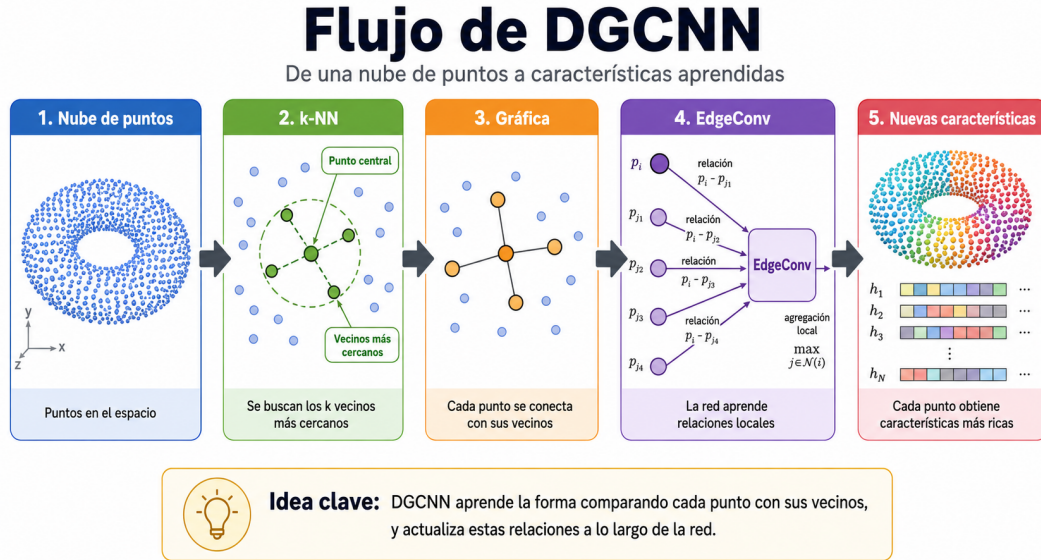


Figura 5.2: Flujo de la operación EdgeConv en DGCNN.

La operación central de DGCNN es EdgeConv, definida como

$$\max_{j \in \mathcal{N}k(i)} \phi\theta \left(h_i^{(l)}, h_j^{(l)} - h_i^{(l)} \right),$$

donde $(h_i^{(l)})$ representa la característica del punto (i) en la capa (l) , $(\mathcal{N}k(i))$ es el conjunto de sus (k) vecinos más cercanos, $(\phi\theta)$ es una función aprendible, usualmente implementada mediante una red MLP, y la operación (máx) agrega la información de los vecinos.

5.1.3. PointNet

PointNet es una arquitectura diseñada para procesar directamente nubes de puntos. Cada punto contiene normalmente coordenadas tridimensionales (x, y, z) , por lo que no es necesario transformar previamente el objeto en imágenes o vóxeles. Su principal propiedad es la invariancia ante permutaciones, es decir, el resultado no depende del orden en que se presenten los puntos de entrada, por lo que PointNet está diseñada para producir el mismo resultado aunque los puntos se presenten en un orden diferente. Esto se logra mediante el uso de operaciones simétricas, como la función (máx), que agregan información de todos los puntos sin importar su orden. Así, PointNet puede obtener una representación global de la forma que puede utilizarse para tareas de clasificación o segmentación.

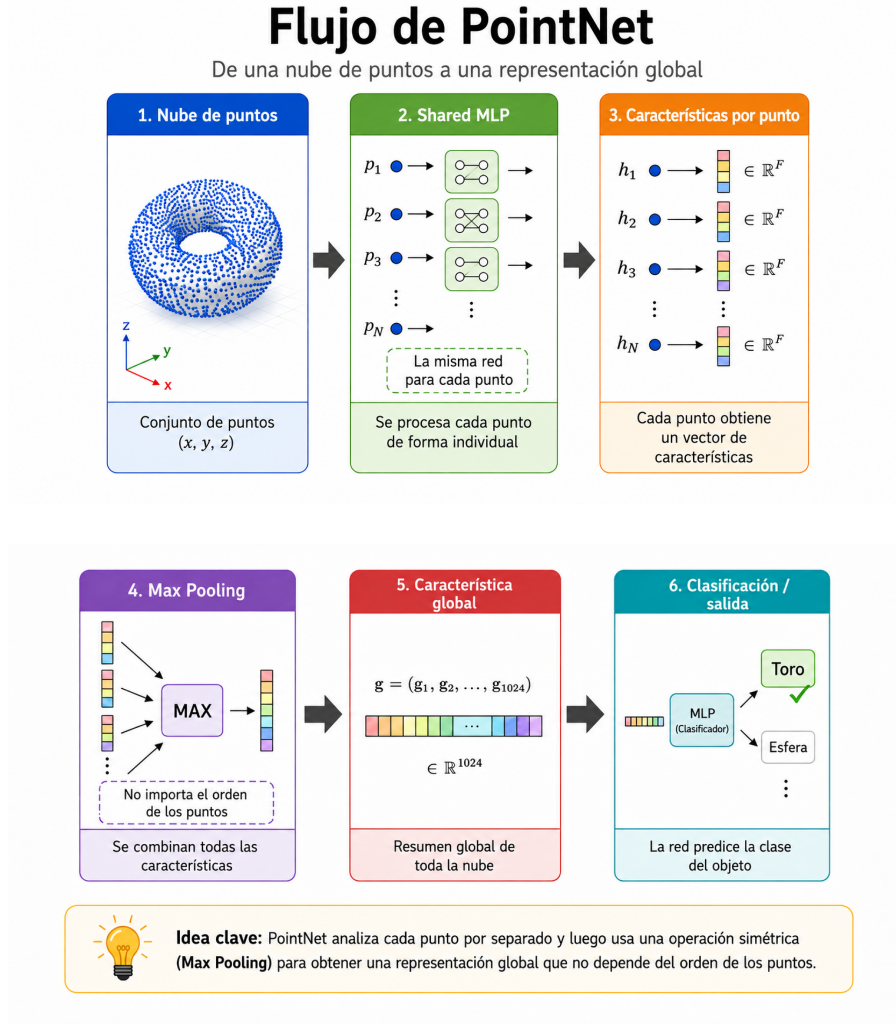


Figura 5.3: Flujo de la arquitectura PointNet. La primera imagen muestra la transformación de cada punto mediante una red MLP, mientras que la segunda ilustra la agregación de características mediante la operación simétrica máx y la posterior clasificación mediante otra red MLP.

Dada una nube de puntos

$$X = x_1, x_2, \dots, x_n, \quad x_i \in \mathbb{R}^3,$$

PointNet aproxima una función sobre conjuntos mediante

$$f(X) \approx \gamma \left(\max_{i=1, \dots, n} \phi(x_i) \right),$$

donde (ϕ) transforma cada punto de manera independiente, (máx) es una operación simétrica que garantiza invariancia al orden, y (γ) es una red que produce la representación global para la clasificación.

5.1.4. PointNet++

PointNet++ extiende PointNet incorporando una estructura jerárquica de agrupamiento local. En lugar de procesar todos los puntos de manera completamente independiente, PointNet++ selecciona algunos puntos representativos y construye al rededor de ellos vecindades locales. Cada uno de estas vecindades representa una pequeña región de la superficie y es procesado mediante una versión local de PointNet para extraer características de esa zona. Después, estas regiones pueden volver a agruparse para formar regiones más grandes, así, PointNet++ aprende características de la superficie a diferentes escalas, lo que permite capturar tanto información local como global.

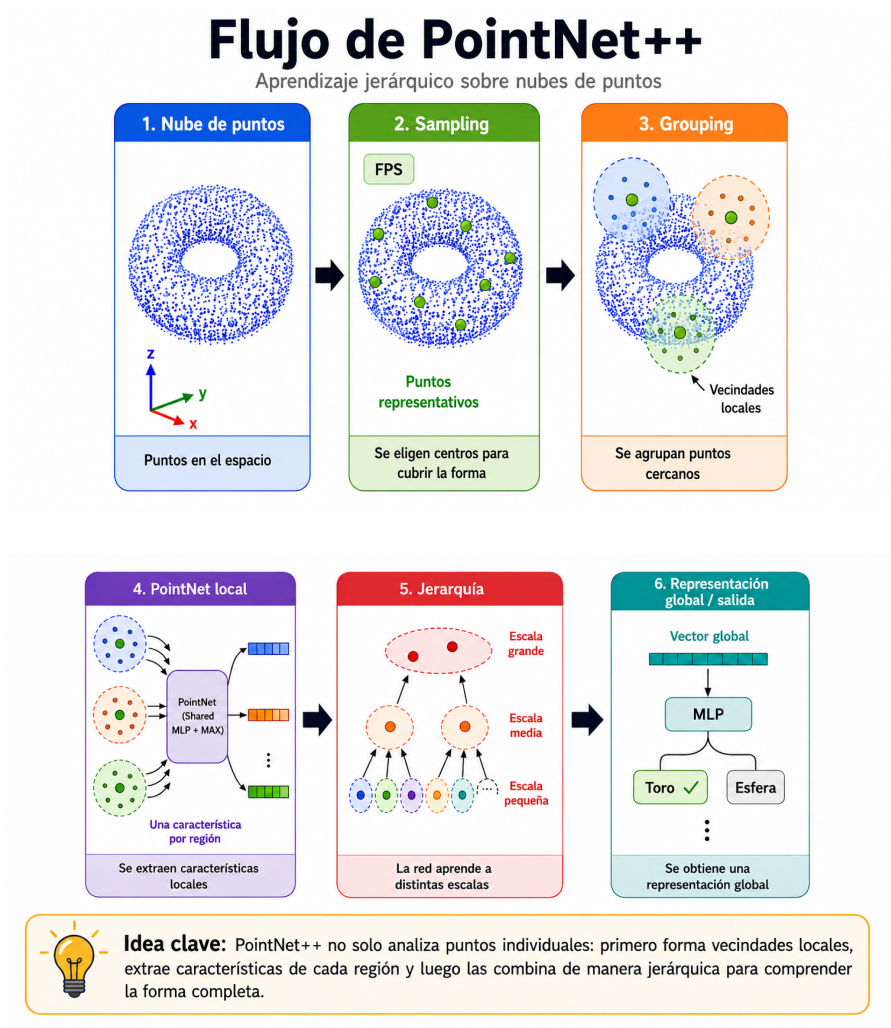


Figura 5.4: Flujo de la arquitectura PointNet++.

De forma esquemática, para un punto central (x_i), se considera una vecindad local

$$x_j \in X : |x_j - x_i| \leq r,$$

o bien una vecindad definida por (k) vecinos más cercanos. La representación local puede

expresarse como

$$\gamma \left(\max_{x_j \in \mathcal{N}(x_i)} \phi(x_j - x_i) \right),$$

donde $(x_j - x_i)$ representa la posición relativa de los puntos vecinos respecto al centroide (x_i) . Posteriormente, estas representaciones locales se combinan jerárquicamente para obtener una representación global de la nube de puntos.

5.2. Análisis de resultados

Con el objetivo de evaluar la efectividad del método de optimización propuesto en esta tesis, se realizaron diversos experimentos sobre el conjunto de datos EuLearn, el cual contiene superficies tridimensionales con distintas características topológicas. Se comparó el desempeño de diferentes arquitecturas de aprendizaje profundo utilizando las métricas de Precisión (Precision), Exhaustividad (Recall), Medida F_1 y Exactitud (Accuracy).

La Tabla 5.1 presenta los resultados obtenidos. En particular, se comparan arquitecturas tradicionales basadas en atención, operadores neuronales y procesamiento de nubes de puntos, contra las versiones optimizadas mediante el procedimiento desarrollado en este trabajo.

Cuadro 5.1: Comparación de resultados obtenidos con distintos modelos. Las variantes GS corresponden a arquitecturas que incorporan el método de optimización y preservación estructural propuesto en esta tesis.

Modelo	Precisión	Recall	F_1	Accuracy
Attention (clásico)	0.01	0.10	0.02	0.10
FNO	0.02	0.11	0.04	0.20
DGCNN	0.07	0.14	0.07	0.16
PointNet (clásico)	0.50	0.49	0.43	0.49
PointNet++	0.50	0.54	0.52	0.63
GS PointNet (propuesto)	0.82	0.79	0.78	0.79
GS Attention (propuesto)	0.81	0.81	0.79	0.81

Los resultados muestran que las arquitecturas tradicionales presentan dificultades para capturar propiedades topológicas globales de las superficies. Esto se observa particularmente en los modelos basados en atención y en DGCNN, cuyos valores de precisión y exactitud permanecen por debajo del 20 %. Por otra parte, las arquitecturas diseñadas específicamente para el procesamiento de nubes de puntos, como PointNet y PointNet++, muestran una mejora considerable, alcanzando exactitudes de 49 % y 63 %, respectivamente. Estos resultados sugieren que la incorporación de información geométrica local permite una mejor representación de las superficies, aunque todavía existen limitaciones para capturar adecuadamente características topológicas globales. La principal contribución de este trabajo se refleja en las variantes GS PointNet y GS Attention, las cuales incorporan el método de optimización desarrollado en esta tesis. Dicho procedimiento realiza una reducción estructurada de la representación original preservando relaciones de adyacencia y propiedades topológicas relevantes. Como consecuencia, ambas arquitecturas obtienen los mejores resultados experimentales. En particular, GS Attention alcanzó una exactitud del 81 % y una medida F_1 de

0.79, mientras que GS PointNet obtuvo una precisión de 0.82 y una exactitud del 79 %. Estos valores representan mejoras significativas respecto a sus contrapartes tradicionales. Comparando directamente PointNet++ con GS Attention, se observa un incremento absoluto de 18 puntos porcentuales en exactitud (63 % frente a 81 %), lo que equivale a una mejora relativa aproximada del 28.6 %. Asimismo, la medida F_1 aumenta de 0.52 a 0.79, evidenciando una mejor capacidad para identificar correctamente las distintas clases topológicas presentes en el conjunto de datos. Estos resultados permiten concluir que el método de optimización propuesto contribuye de manera significativa a la preservación de información estructural relevante para el aprendizaje. La mejora observada en todas las métricas evaluadas sugiere que la estrategia desarrollada facilita la extracción de características topológicas discriminativas, permitiendo a las redes neuronales capturar propiedades globales de las superficies con mayor efectividad.

5.3. Conclusiones

Bibliografía

- [1] Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3rd edition, 2009.
- [3] Carlos Prieto de Castro. *Topología básica*. Ciencia y Tecnología. Fondo de Cultura Económica, México, 2003.
- [4] Reinhard Diestel. *Graph Theory*. Springer, Graduate Texts in Mathematics, 2017.
- [5] Rodrigo Fritz, Pablo Suárez-Serrato, Victor Mijangos, Anayanzi D. Martinez-Hernandez, and Eduardo Ivan Velazquez Richards. Eulearn: A 3d database for learning euler characteristics. *arXiv preprint*, 2025.
- [6] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [7] William L. Hamilton. *Graph Representation Learning*. McGill University Press, 2020.
- [8] Tero Harju. *Graph Theory*. Department of Mathematics University of Turku, 2007.
- [9] D Knuth. *The Art of Computer Programming*. ADDISON-WESLEY An Imprint of Addison Wesley Longman, Inc., 1998.
- [10] E. G. Minian. *Complejos Simpliciales y Poliedros (Introducción): Notas – Ejercicios Guiados*. Departamento de Matemática, Facultad de Ciencias Exactas y Naturales, Universidad de Buenos Aires.
- [11] James R. Munkres. *Topology*. Prentice Hall, Upper Saddle River, NJ, 2 edition, 2000.
- [12] Andrew Pressley. *Elementary Differential Geometry*. Springer Undergraduate Mathematics Series, 2010.
- [13] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019.

Capítulo A

Definiciones básicas

Definición 68. (Espacio métrico) Un espacio métrico consta de un conjunto X y una función $d : X \times X \rightarrow \mathbb{R}^+$, llamada métrica, o distancia que satisface los siguientes axiomas:

1. $d(x, y) = 0$ si y sólo si $x = y$.
2. $d(x, y) = d(y, x)$ para todo $x, y \in X$.
3. $d(x, y) \leq d(x, z) + d(z, y)$ para todo $x, y, z \in X$

Definición 69. (Bola abierta) Sea $x \in X$ con X espacio métrico, $\epsilon \in \mathbb{R}^+$. Definimos la bola abierta con centro en x y radio ϵ como:

$$\mathcal{B}_\epsilon(x) = \{y \in X \mid d(x, y) < \epsilon\}$$

Definición 70. (Vecindad) Sea X un espacio métrico y U un subconjunto de este, decimos que $U \subseteq X$ es una **vecindad** de x si existe $\epsilon > 0$ tal que $\mathcal{B}_\epsilon(x) \subseteq U$

Definición 71. (Conjunto abierto) Sea X un espacio métrico. Decimos que un subconjunto A de X es **abierto** si A es vecindad de x para toda $x \in A$. Es decir $A \subseteq X$ es abierto si y sólo si para todo $x \in A$ existe $\epsilon > 0$ tal que $\mathcal{B}_\epsilon(x) \subseteq A$

Teorema 6. (Unión e intersección de abiertos) Sea $\mathcal{A}$ el conjunto de todos los abiertos en un espacio métrico X . Entonces se cumplen las siguientes afirmaciones:

1. Sea $\mathcal{I}$ un conjunto arbitrario de índices. Si $\{A_i\}_{i \in \mathcal{I}}$ es una familia de elementos en $\mathcal{A}$, entonces $\bigcup_{i \in \mathcal{I}} A_i$ es elemento de $\mathcal{A}$
2. Sea $\mathcal{I}$ un conjunto finito de índices. Si $\{A_i\}_{i \in \mathcal{I}}$ es una familia de elementos en $\mathcal{A}$, entonces $\bigcap_{i \in \mathcal{I}} A_i$ es elemento de $\mathcal{A}$

Capítulo B

Código implementado en Python

Listing B.1: Algoritmo principal de muestreo con preservación de adyacencias

```
1 import numpy as np
2 from copy import deepcopy
3 from scipy.sparse.csgraph import minimum_spanning_tree
4
5
6 def generaPuntosUniformes(h, k, l, num_puntos):
7     """
8     Genera puntos uniformemente distribuidos
9     dentro de una esfera unitaria centrada
10    en (h, k, l).
11    """
12
13    puntos = []
14
15    for _ in range(num_puntos):
16
17        # Metodo de Marsaglia para generar
18        # una direccion uniforme en la esfera
19        while True:
20            V1, V2 = np.random.uniform(-1, 1, 2)
21            S = V1**2 + V2**2
22
23            if 0 < S < 1:
24                break
25
26            a = np.sqrt(1 - S)
27
28            # Direccion sobre la esfera unitaria
29            dx = 2 * V1 * a
30            dy = 2 * V2 * a
31            dz = 2 * S - 1
32
33            # Radio con distribucion proporcional
34            # al volumen de la esfera
35            r = np.random.uniform(0, 1)**(1 / 3)
36
37            # Trasladar el punto al centro (h, k, l)
38            x = h + r * dx
39            y = k + r * dy
```

```

40     z = 1 + r * dz
41
42     puntos.append([x, y, z])
43
44     return np.array(puntos)
45
46
47 def sampling(x, adjacency, size=1000, alcance=1.0, esfera_size=50):
48     """
49     Reduce el numero de nodos de una grafica
50     conservando informacion de adyacencia.
51
52     Args:
53         x:
54             Coordenadas de los nodos.
55
56         adjacency:
57             Matriz de adyacencia original.
58
59         size:
60             Numero objetivo de nodos.
61
62         alcance:
63             Distancia maxima utilizada para
64             identificar vecinos espaciales.
65
66         esfera_size:
67             Numero de puntos utilizados para
68             construir la region de busqueda.
69
70     Returns:
71         new_x:
72             Coordenadas de los nodos conservados.
73
74         new_adj:
75             Matriz de adyacencia reducida.
76     """
77
78     # -----
79     # 1. Construccion del arbol generador minimo (MST)
80     # -----
81
82     mst = minimum_spanning_tree(adjacency).toarray().astype(int)
83
84     # Convertir el MST en una estructura simetrica
85     mst = mst + mst.T
86
87     # -----
88     # 2. Recorrido DFS sobre el MST

```

```

89  # -----
90
91  def dfs_order(node, visited, order):
92      visited[node] = True
93      order.append(node)
94
95      neighbors = np.where(mst[node] == 1)[0]
96
97      for neighbor in neighbors:
98          if not visited[neighbor]:
99              dfs_order(neighbor, visited, order)
100
101  visited = np.zeros(adjacency.shape[0], dtype=bool)
102  order = []
103
104  # Comenzar el recorrido desde el nodo 0
105  dfs_order(0, visited, order)
106
107  # -----
108  # 3. Inicializacion del muestreo
109  # -----
110
111  new_adj = deepcopy(adjacency)
112
113  # Orden inicial inducido por DFS
114  queue = list(order)
115
116  # Nodos que seran eliminados
117  delel = []
118
119  M = adjacency.shape[0]
120
121  # -----
122  # 4. Proceso iterativo de reduccion
123  # -----
124
125  while M - len(set(delel)) > size and len(queue) > 0:
126
127      # Si ya existen nodos eliminados,
128      # reorganizar los candidatos
129      if delel:
130
131          # Nodos que aun permanecen
132          index_selected = list(set(range(M)) - set(delel))
133          selected_coords = x[index_selected]
134
135          # Calcular el centroide de los
136          # puntos que permanecen
137          centroid = selected_coords.mean(axis=0)

```

```

138
139     # Distancia de cada nodo de la
140     # cola respecto al centroide
141     dists = np.linalg.norm(x[queue] - centroid, axis=1)
142
143     # Priorizar los puntos mas alejados
144     queue = [queue[i] for i in np.argsort(dists)[::-1]]
145
146     # -----
147     # 5. Seleccion del nodo central
148     # -----
149
150     idx = queue.pop(0)
151     h, k, l = x[idx]
152
153     # -----
154     # 6. Construccion de la region espacial
155     # -----
156
157     sphere_points = generaPuntosUniformes(h, k, l, esfera_size)
158
159     # Calcular la distancia de cada nodo
160     # a los puntos de la region generada
161     distances = np.linalg.norm(
162         x[:, None] - sphere_points[None, :], axis=2
163     )
164
165     # Nodos que cumplen la condicion
166     # de proximidad espacial
167     sphere_neighbors = np.where(
168         distances.min(axis=1) <= alcance
169     )[0]
170
171     # -----
172     # 7. Interseccion con la adyacencia
173     # -----
174
175     adjacent_neighbors = np.where(adjacency[idx] == 1)[0]
176
177     # Un nodo se considera candidato unicamente
178     # si cumple proximidad espacial y adyacencia
179     neigs = np.intersect1d(sphere_neighbors, adjacent_neighbors)
180
181     # Ignorar nodos eliminados previamente
182     neigs = np.array([n for n in neigs if n not in delel])
183
184     # Evitar eliminar mas nodos de los
185     # necesarios para alcanzar la medida objetivo
186     remaining = M - len(set(delel)) - size

```

```

187
188     if len(neigs) > remaining:
189         neigs = neigs[:remaining]
190
191     # Eliminar los vecinos seleccionados
192     # de la cola de candidatos
193     queue = list(set(queue).difference(neigs))
194
195     # -----
196     # 8. Redistribucion de adyacencias
197     # -----
198
199     for n in neigs:
200
201         # Vecinos del nodo que sera eliminado
202         nears = list(
203             set(np.where(new_adj[n] == 1)[0]).difference(delel)
204         )
205
206         if len(nears) == 0:
207             continue
208
209         # Reunir las conexiones de
210         # los vecinos del nodo n
211         result = new_adj[nears].sum(axis=0)
212
213         # Transferir las conexiones
214         # hacia el nodo central idx
215         new_adj[idx] += result
216         new_adj.T[idx] += result
217
218         # -----
219         # 9. Eliminacion de los nodos seleccionados
220         # -----
221
222         new_adj[neigs] = 0
223         new_adj.T[neigs] = 0
224
225         delel += list(neigs)
226
227         # Eliminar elementos repetidos
228         delel = list(set(delel))
229
230         # -----
231         # 10. Construccion de la grafica reducida
232         # -----
233
234     new_adj = np.delete(new_adj, delel, axis=0)
235     new_adj = np.delete(new_adj, delel, axis=1)

```

```
236
237     # Binarizar la matriz de adyacencia
238     new_adj = 1 * (new_adj > 0)
239
240     # Eliminar posibles auto-conexiones
241     np.fill_diagonal(new_adj, 0)
242
243     # Indices correspondientes
244     # a los nodos conservados
245     index = list(set(range(M)).difference(delel))
246
247     # Nueva nube de puntos
248     new_x = x[index, :]
249
250     print(
251         '\nOriginal size: {}\nSampling size: {}'.format(
252             x.shape, new_x.shape
253         )
254     )
255
256     return new_x, new_adj
```